408 详解 · 2017 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2017-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

>

读法：先看标题话题与答案，再按 6 节往下读；每题自成体系，前置概念在题内讲清。

>

图题说明：第 5 题、第 8 题的图中细节未完全看清，题内与附录均标注“部分核对”，图中细节以原卷为准。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 含 sum+=++i 的 while 循环时间复杂度（正确答案：B）

题干与选项：

下列函数的时间复杂度是（）。

```
int func(int n){ int i=0, sum=0; while(sum < n) sum += ++i; return i; }
```

- A. O(log n)
- B. O(n^(1/2))
- C. O(n)
- D. O(n log n)

入手点： 别急着套公式，先把 sum 的变化列出来：i 从 1 开始往上加，sum 就是 1、3、6、10……这样长；循环停在 sum 够到 n 的那一刻，问题就变成“加到第几个数能超过 n”。

题干专有名词从零解释：

- 先想“时间复杂度”：衡量程序跑得慢不慢的办法，看的是“输入变大时，基本操作次数跟着怎么变”，不数具体的秒数。
- 再想“大 O 记号”：只留增长最快的那一项，系数和低次项都扔掉；比如次数是 $\sqrt{2n}$，就写成 O(n 的 1/2 次方)。
- 再想“++i”：先把 i 加 1，再用加完的值；所以 sum 第一次加的是 1，第二次加的是 2，第三次加的是 3。

考查知识点：

- 先想 sum 的规律：第 1 次 sum=1，第 2 次 sum=1+2=3，第 3 次 sum=1+2+3=6，第 i 次 sum=1+2+…+i=i(i+1)/2。
- 再想循环停在哪：循环条件是 sum<n，只要 sum 还没够到 n 就继续；停下来时 i(i+1)/2 刚好超过 n，也就是 i 的平方大约是 2n。
- 再想次数：i 大约是 $\sqrt{2n}$，也就是根号 n 的量级，所以时间复杂度是 O(n 的 1/2 次方)。
- 拿生活例子验算：每天存钱，第 1 天存 1 块，第 2 天存 2 块，第 3 天存 3 块……存够 100 块要多少天？前 13 天一共存了 91 块，前 14 天存了 105 块，所以要 14 天；天数大约是总钱数的平方根，不是一天一天线性长。
- 边界情况：如果循环条件改成 sum<=n，次数最多差 1 次，量级不变；如果 i 每次加的是固定值（比如每次加 1 个常数），那才是 O(n)。
- 易混对子：O(log n) 是“每次规模砍半”（比如二分查找）才有的；O(n) 是“每个元素只看一遍”才有的；本题 i 是越加越大的，不是每次砍半，也不是一遍扫完。

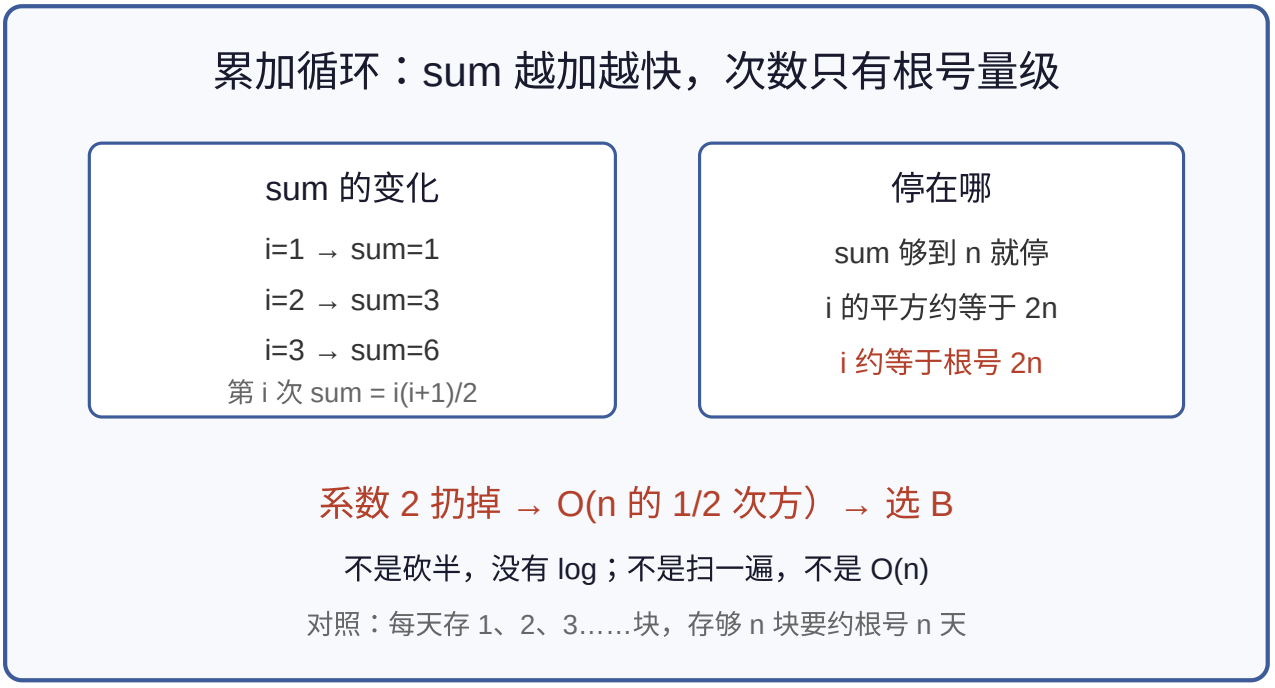
正确选项为什么对： i 的次数大约是 $\sqrt{2n}$，扔掉系数 2，就是 O(n 的 1/2 次方)，就是 B，选 B。

错误选项为什么错＋举例：

- A（O(log n)）：那是每次把问题规模砍掉一半的算法（比如二分查找，每次剩一半）；本题是累加，不是砍半。教训：先看循环里规模怎么变，砍半才有 log。
- C（O(n)）：那是 i 每次只加固定值、扫一遍的情况；本题 i 越加越大，sum 长得比线性快，次数比 n 小得多。教训：先列出 sum 的变化，再定量级。
- D（O(n log n)）：那是“扫 n 遍、每遍再花 log n”（比如归并排序的量级）；本题只有一个循环，没有嵌套。教训：单循环先按单循环算，不要套排序的量级。

笔记本一句话： 第 1 题＋累加循环：sum=1+2+…+i，i 的平方约等于 2n，次数是根号量级。

【图像】



累加循环示意图：i 从 1 往上加，sum 为 1、3、6、10，次数约根号 n

【定位】 数据结构 · 第1章绪论 · 算法的时间复杂度

【易混对比】

- "累加循环"和"砍半循环"是两回事：累加是 sum 越加越快、次数只有根号量级；砍半是每次规模减半、次数才是对数量级——看到循环先看规模怎么变，别上来就套 log。
- "O(根号 n)"和"O(n)"是两回事：前者是 i 越加越大、sum 长得比线性快；后者是 i 每次只加固定值、老老实实把 n 扫一遍。
- "$\sqrt{2n}$"和"n"是两回事：大 O 记号里系数 2 直接扔掉，只留 $n^{1/2}$ 这一项。

sum 第 i 次等于 i(i+1)/2，循环停在 i 的平方约等于 2n 的地方，次数是根号量级，即 O(n 的 1/2 次方)，选 B。

【坑位】

别看到循环就写 O(n)，先把 sum 的变化列出来再定量级。

最容易踩的坑是：数循环体里的语句有几条，就当复杂度是 O(n)。为什么错：复杂度看的是"基本操作执行多少次"，不是"循环体里有几行"；本题循环体只有一句 sum+=++i，但循环跑的次数不是 n 次。正确做法：先列 sum——第 1 次 1，第 2 次 3，第 3 次 6……看出 sum=i(i+1)/2，再让它够到 n，解出 i 约等于根号 2n，量级就是 O(根号 n)。

【套路】

- 第一步，列出累加规律：把 sum 随 i 的变化写出来，认出 sum=1+2+…+i=i(i+1)/2。
- 第二步，找停止条件：循环停在 sum 刚够到 n 的那一刻，即 i(i+1)/2 超过 n。
- 第三步，解出 i 的量级：i 的平方约等于 2n，i 约等于根号 2n。
- 第四步，扔系数写大 O：系数 2 扔掉，写成 O(n 的 1/2 次方)。

累加循环：先列和，再找停点，解出 i 的量级，系数扔掉。

【类比】

像每天存钱：第 1 天存 1 块，第 2 天存 2 块，第 3 天存 3 块……存够 100 块要多少天？前 13 天一共 91 块，前 14 天 105 块，所以要 14 天。天数大约是总钱数的平方根——钱越多、后面每天存得越多，天数长得比钱慢得多。

【口诀】 累加循环先列和，平方约等二倍 n，次数开根别写 n。

【考频】 「累加循环时间复杂度」在 2009–2022 年解析中出现 12 次 · 高频

第 2 题 · 关于栈的四条叙述中错误的一项（正确答案：C）

题干与选项：

下列关于栈的叙述中，错误的是（）。

Ⅰ．采用非递归方式重写递归程序时必须使用栈

Ⅱ．函数调用时，系统要用栈保存必要的信息

Ⅲ．只要确定了入栈次序，即可确定出栈次序

Ⅳ．栈是一种受限的线性表，允许在其两端进行操作

- A. 仅Ⅰ
- B. 仅Ⅰ、Ⅱ、Ⅲ
- C. 仅Ⅰ、Ⅲ、Ⅳ
- D. 仅Ⅱ、Ⅲ、Ⅳ

入手点： 本题问“错误的是”，做法是先把四条叙述一条一条判对错，再看哪个选项正好把所有错的都挑出来、一个对的都不带。

题干专有名词从零解释：

- 先想“栈”：一种只能在一端进出的线性表；后进来的先出去，像一摞盘子，只能从最上面拿。
- 再想“递归程序”：函数自己调用自己的程序；每次调用都要记下“回来后接着干什么”，这个记录通常放在栈里。
- 再想“入栈次序”和“出栈次序”：元素进栈的先后顺序，和元素出栈的先后顺序；出栈顺序不光由入栈顺序定，还由中间什么时候出栈定。
- 再想“受限的线性表”：操作受限制的表；栈只允许在一端插入和删除。

考查知识点：

- 先想第Ⅰ条：非递归重写递归程序，不一定非用栈不可；有些递归（比如尾递归、简单的循环累加）可以直接改成循环，不用显式地建栈。所以第Ⅰ条是错的。
- 再想第Ⅱ条：函数调用时，系统确实要用栈保存返回地址、参数、局部变量这些“回来后接着干什么”的信息。所以第Ⅱ条是对的。
- 再想第Ⅲ条：只知道入栈次序，定不出出栈次序；比如 a、b、c 依次入栈，出栈可以是 c、b、a，也可以是 a、c、b（a 先出，再入 b、c），要看中间插了几次出栈。所以第Ⅲ条是错的。
- 再想第Ⅳ条：栈只允许在一端操作；两端都能操作的叫双端队列，不是栈。所以第Ⅳ条是错的。
- 再想结论：错的是Ⅰ、Ⅲ、Ⅳ，就是 C。
- 边界情况：第Ⅰ条容易误判——“递归要用栈”和“重写递归必须用栈”是两回事；前者说的是递归执行时的机制，后者说的是改写方法的选择，改写时完全可以不用栈。

正确选项为什么对： Ⅰ、Ⅲ、Ⅳ 三条都是错的，Ⅱ 是对的；C 正好是“仅Ⅰ、Ⅲ、Ⅳ”，把错的全挑出来、没带对的，符合“错误的是”的要求，选 C。

错误选项为什么错＋举例：

- A（仅Ⅰ）：只挑了Ⅰ，漏了Ⅲ和Ⅳ；Ⅲ（出栈次序不定）和Ⅳ（两端操作）也是错的，漏挑就不完整。教训：问“错误的是哪几条”，要一条一条全判完再组合。

- B（仅 Ⅰ、Ⅱ、Ⅲ）：把 Ⅱ 也带进来了；Ⅱ（函数调用用栈保存信息）是对的，不能算进错误里。教训：组合里混进对的，就不对了。
- D（仅 Ⅱ、Ⅲ、Ⅳ）：把 Ⅰ 漏了、把 Ⅱ 带进来了，两头都错；Ⅰ（必须用栈）是错的不能漏，Ⅱ 是对的不能带。教训：先逐条判对错，再对组合。

笔记本一句话： 第 2 题＋栈的叙述：重写递归不一定用栈；出栈次序不能只由入栈次序定；栈只在一端操作。

【图像】



栈的四条叙述判定示意图：栈只在一端开口，四条叙述分别打叉打勾，错的是 Ⅰ Ⅲ Ⅳ

【定位】 数据结构 · 第3章栈、队列和数组 · 栈的基本性质

【易混对比】

- "递归执行要用栈"和"重写递归必须用栈"是两回事：前者说的是运行机制——函数调用现场靠栈保存返回信息；后者说的是改写选择——尾递归这类可以直接改成循环，不用显式建栈。
- "入栈次序"和"出栈次序"是两回事：入栈只定进的先后，出栈还看中间插了几次弹出；a、b、c 依次入栈，出栈可以是 c、b、a，也可以是 a、c、b。
- "栈"和"双端队列"是两回事：栈只允许在一端插入删除，两端都能操作的是双端队列，不是栈。

错的是 Ⅰ、Ⅲ、Ⅳ，Ⅱ 是对的；题目问"错误的是"，选把错的全挑出来、一个对的都不带的 C。

【坑位】

问"错误的是"先逐条判对错再组装选项，别凭感觉直接选。

最容易踩的坑是：看到 Ⅰ "重写递归不一定用栈"觉得别扭，就顺手把它当对的。为什么错：递归"跑起来"确实用栈（系统自动的），但"改写"时人可以选择不用栈——尾递归改循环就是例子；"机制"和"选择"是两码事。正确做法：四条叙述一条一条打勾打叉——Ⅰ 错、Ⅱ 对、Ⅲ 错、Ⅳ 错，然后找哪个选项正好等于"Ⅰ、Ⅲ、Ⅳ"的集合，多一条少一条都不行。

【套路】

- 第一步，逐条判对错：Ⅰ（重写递归不一定用栈）错；Ⅱ（函数调用用栈存返回信息）对；Ⅲ（只知道入栈次序定不出出栈次序）错；Ⅳ（栈两端都能操作）错。
- 第二步，集合错的：错的集合是Ⅰ、Ⅲ、Ⅳ。
- 第三步，对选项：找正好等于这个集合的选项——C 是"仅Ⅰ、Ⅲ、Ⅳ"，就是它。

错误的是哪几条：先逐条判，再集合错的，最后对选项。

【类比】

像老师判四道判断题再出"选出所有错题"的单选：必须四道全判完，把错题号抄下来，再去选项里找题号完全一致的那个；漏抄一道、或把对题抄进来，答案都对不上。

【口诀】 递归重写不一定栈，出栈还要看中间，一端操作是栈规，逐条判完再组装。

【考频】 「栈的基本性质」在 2009–2022 年解析中出现 24 次 · 高频

第 3 题 · 压缩存储稀疏矩阵的两种存储结构（正确答案：A）

题干与选项：

适用于压缩存储稀疏矩阵的两种存储结构是（ ）。

- A. 三元组表和十字链表
- B. 三元组表和邻接矩阵
- C. 十字链表和二叉链表
- D. 邻接矩阵和十字链表

入手点： 稀疏矩阵就是“大部分元素都是 0”的矩阵；压缩的意思是“只存非 0 元素，别给 0 占地方”；两种结构都要满足“只存非 0 元素、还能找到它们的位置”。

题干专有名词从零解释：

- 先想“稀疏矩阵”：非 0 元素很少、0 占绝大多数的矩阵；按行列全存下来太浪费。
- 再想“三元组表”：每个非 0 元素存三个数——行号、列号、值；一张表把所有非 0 元素列出来。
- 再想“十字链表”：每个非 0 元素是一个结点，结点上有“指向同行下一个”和“指向同列下一个”的指针；行和列各有一条链，像十字交叉。
- 再想“邻接矩阵”：图用的存储办法，顶点两两之间有没有边都占一个位置；存稀疏矩阵就等于把 0 也全存了。

考查知识点：

- 先想三元组表为什么行：只存非 0 元素的行、列、值，0 不占地方，存下来就是压缩的；按行号顺序排，还能按位置找。
- 再想十字链表为什么行：每个非 0 元素挂在行链和列链上，0 不建结点；行方向、列方向都能顺着链走，适合经常按行或按列操作的情况。
- 再想邻接矩阵为什么不行：它给每个位置（包括 0）都留地方，存稀疏矩阵和全存没区别，起不到压缩作用。
- 再想二叉链表为什么不行：二叉链表是给二叉树用的（每个结点两个孩子指针），不是给矩阵用的；矩阵元素的行列关系用它表达不了。
- 边界情况：三元组表适合“存下来、偶尔访问”的情况；十字链表适合“经常按行列插入、删除非 0 元素”的情况；两个都是压缩存储，只是操作方便程度不同。

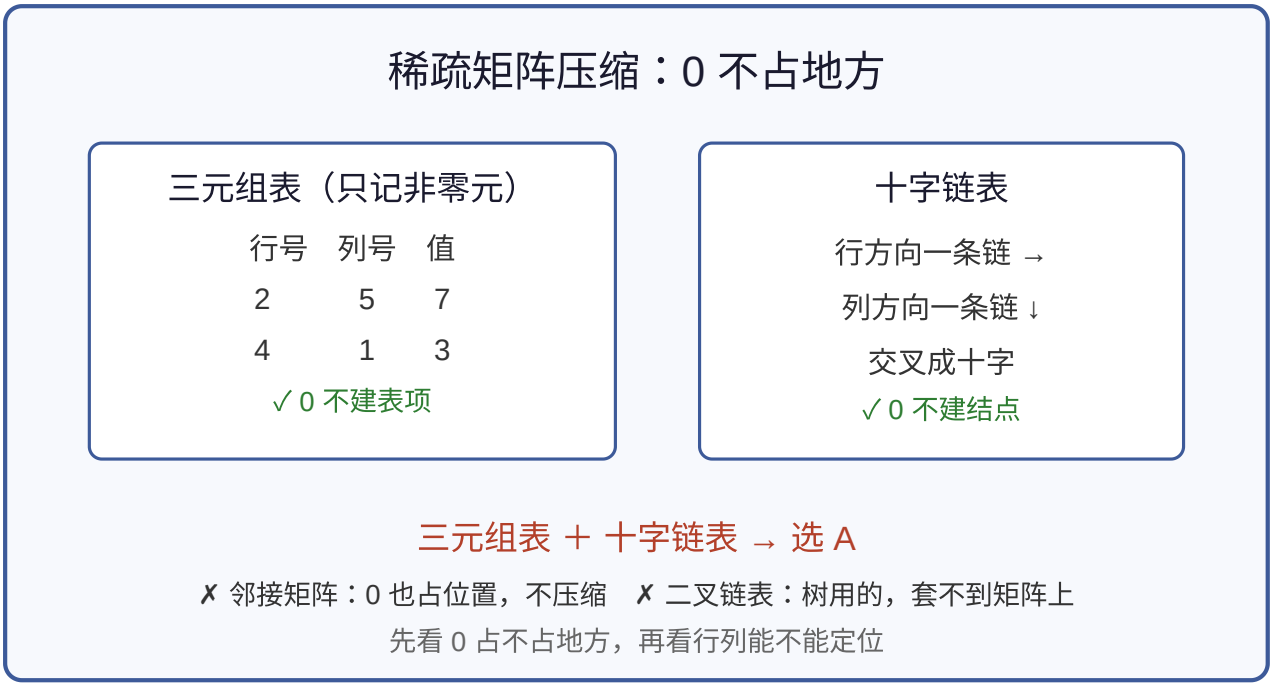
正确选项为什么对： 三元组表和十字链表都只存非 0 元素、都能定位，就是 A，选 A。

错误选项为什么错＋举例：

- B（三元组表和邻接矩阵）：邻接矩阵把 0 也存了，不压缩；比如 100×100 的矩阵只有 10 个非 0 元素，邻接矩阵要存 10000 个位置，白白占地方。教训：带“矩阵”两个字的结构不一定适合压缩存储。
- C（十字链表和二叉链表）：二叉链表是树的结构，不是矩阵的；矩阵的行列关系它表达不了。教训：先看结构是给谁用的，树用的不能硬套到矩阵上。
- D（邻接矩阵和十字链表）：邻接矩阵不压缩，错在这一半；十字链表是对的，但组合里混进错的就不行。教训：组合题要两半都对才行。

笔记本一句话： 第 3 题＋稀疏矩阵压缩：三元组表（行、列、值）＋十字链表（行链列链），0 不占地方。

【图像】



稀疏矩阵压缩存储示意图：左边三元组表只记非零元的行列值，右边十字链表行列各挂一条链，零元素不占地方

【定位】 数据结构 · 第3章栈、队列和数组 · 特殊矩阵的压缩存储

【易混对比】

- "三元组表"和"十字链表"是两回事：都只存非 0 元素，但前者是一张按行号排好的表、适合存下来偶尔访问，后者是行列各挂一条链、适合经常插入删除非 0 元素。
- "邻接矩阵"和"压缩存储"是两回事：邻接矩阵给每个位置（含 0）都留地方，存稀疏矩阵等于全存，起不到压缩作用。
- "十字链表"和"二叉链表"是两回事：前者是矩阵用的（行链加列链），后者是二叉树用的——树用的结构硬套不到矩阵上。

三元组表和十字链表都只存非 0 元素、都能定位行列，就是 A，选 A。

【坑位】

带"矩阵"两字的结构不一定能压缩，邻接矩阵就是反例。

最容易踩的坑是：看到选项里有"邻接矩阵"，觉得它名字里带矩阵、又是二维的，就当它能存稀疏矩阵。为什么错：压缩的意思是"0 不占地方"；邻接矩阵不管是不是 0 都占一个位置，100×100 的矩阵只有 10 个非 0 元素，它也要存 10000 个位置，白占地方。正确做法：先问"0 占不占地方"——占地方的直接排除；再问"行列能不能定位"——三元组表（行、列、值）和十字链表（行链、列链）都过关。

【套路】

- 第一步，看 0 占不占地方：占地方的（邻接矩阵）直接排除，不压缩。
- 第二步，看行列能不能定位：三元组表靠行号列号找，十字链表靠行链列链走，都能定位。
- 第三步，看结构给谁用：二叉链表是树的，矩阵的行列关系它表达不了，排除。

稀疏矩阵压缩：先看 0 占不占地方，再看行列能不能定位。

【类比】

像班级点名册只记到场的人：三元组表像按学号排好的到场名单（行、列、值就是学号、座位、姓名）；十字链表像每排、每列各有一条点名链，增删一个人只改链不断表；邻接矩阵像给全班每人（含缺席的）都留一行，谁没来也占着纸。

【口诀】 稀疏矩阵要压缩，三元组表加十字链，邻接矩阵存全零，二叉链表树上见。

【考频】 「稀疏矩阵压缩存储」在 2009–2022 年解析中出现 1 次 · 低频

第 4 题 · 非空二叉树先序与中序序列相同的非叶结点条件（正确答案：B）

题干与选项：

要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的条件是（）。

- A. 只有左子树
- B. 只有右子树
- C. 结点的度均为 1
- D. 结点的度均为 2

入手点： 先把先序和中序的“报结点顺序”写出来比一比：先序是“结点、左、右”，中序是“左、结点、右”；两个序列要相同，左子树必须不存在——不然左子树的结点会把顺序打乱。

题干专有名词从零解释：

- 先想“先序序列”：先报结点自己，再报左子树，再报右子树；根永远在最前面。
- 再想“中序序列”：先报左子树，再报结点自己，再报右子树；根夹在左和右中间。
- 再想“非叶结点”：至少有一个孩子的结点；叶结点是没有孩子的结点。
- 再想“度”：结点有几个孩子；度为 1 就是只有一个孩子，度为 2 就是有两个孩子。

考查知识点：

- 先想为什么不能有左子树：只要某个结点有左孩子，先序里这个结点排在左孩子前面，中序里这个结点排在左孩子后面；两个序列在这一点上就对不上。所以有左孩子的结点不允许。
- 再想只有右子树为什么行：结点只有右孩子时，先序是“结点、右子树”，中序也是“结点、右子树”；两边顺序一样，整棵树递归下去都一样。
- 再想叶结点：叶结点没有孩子，先序和中序报它时都是它自己，不影响条件；条件只约束非叶结点。
- 拿小例子验算：根 a、右孩子 b、b 的右孩子 c；先序是 a、b、c，中序也是 a、b、c，相同。反过来，根 a、左孩子 b：先序是 a、b，中序是 b、a，不同。
- 边界情况：空树不在讨论范围（题目说非空）；只有一个结点的树既是叶结点，两个序列都是它自己，条件自然满足。

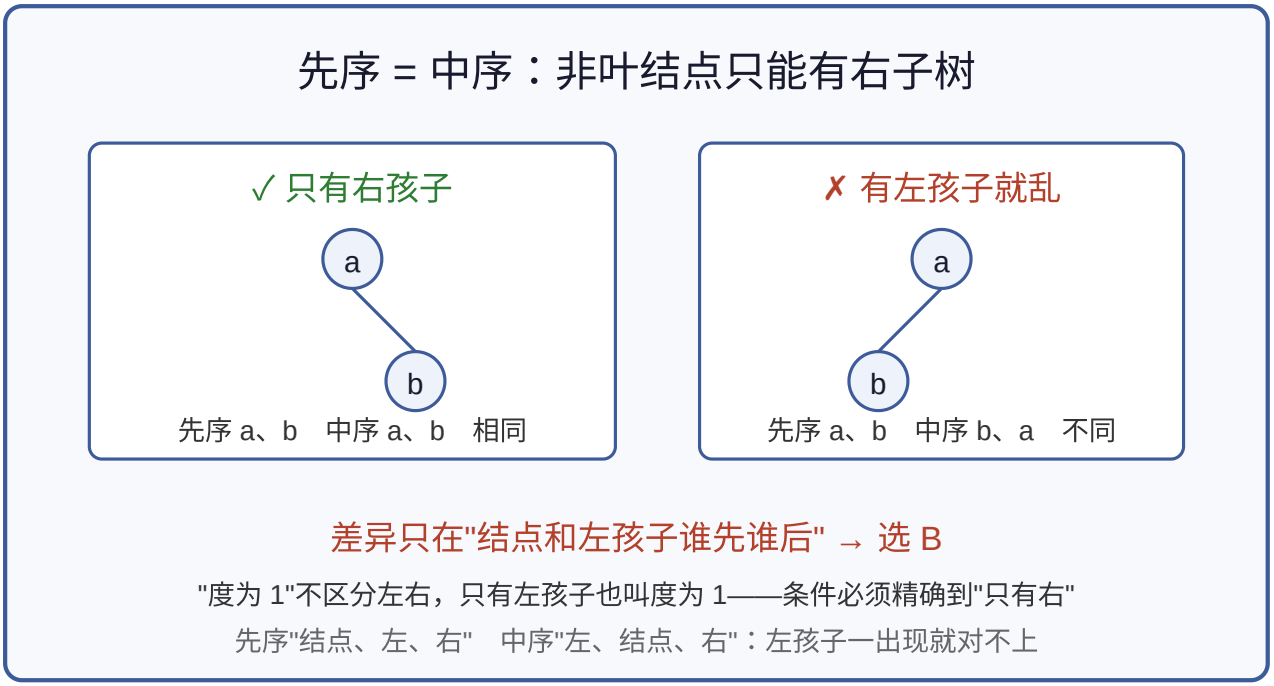
正确选项为什么对： 非叶结点都只有右子树时，先序和中序才处处相同，就是 B，选 B。

错误选项为什么错＋举例：

- A（只有左子树）：有左孩子时，先序里结点在左孩子前面，中序里结点在左孩子后面，两个序列对不上。教训：先序和中序的区别就在“结点和左孩子的先后”，左孩子一出现就乱了。
- C（结点的度均为 1）：度为 1 的结点可能是“只有左孩子”；只要有一个结点是只有左孩子，序列就不相同。教训：度为 1 不区分左右，条件必须精确到“只有右”。
- D（结点的度均为 2）：度为 2 的结点一定有左孩子，序列一定不同。教训：有两个孩子就一定有左孩子，直接排除。

笔记本一句话： 第 4 题＋先序等于中序：非叶结点只能有右子树，左孩子一出现顺序就乱。

【图像】



先序等于中序的条件示意图：只有右孩子的链上先序中序都是 a、b、c，有左孩子时先序 a、b 对中序 b、a

【定位】 数据结构 · 第5章树与二叉树 · 二叉树的遍历序列

【易混对比】

- "只有右子树"和"度为 1"是两回事：度为 1 只说有一个孩子，不说左右——"只有左孩子"也叫度为 1，但序列就对不上了；条件必须精确到"只有右"。
- "先序"和"中序"的区别点是两回事：只差在"结点和左孩子谁先谁后"——先序是结点在左孩子前，中序是结点在左孩子后；左孩子一出现，两处必然对不上。

非叶结点只能有右子树：只要有一个结点有左孩子，先序和中序在该处必然对不上，选 B。

【坑位】

度为 1 不等于只有右子树，条件必须精确到左右。

最容易踩的坑是：看到"度为 1"觉得就是"只有一个孩子、排成一条链"，直接选 C。为什么错：度为 1 的结点可能是"只有左孩子"；比如根 a 只有左孩子 b，先序是 a、b，中序是 b、a，两个序列不同。正确做法：把先序"结点、左、右"和中序"左、结点、右"并排写，找差异点——差异只在结点和左孩子的先后；要让两处处处相同，左子树必须不存在，即非叶结点只能有右孩子。

【套路】

- 第一步，写两种报法：先序"结点、左、右"，中序"左、结点、右"。
- 第二步，找差异点：只差在结点和左孩子谁先谁后。
- 第三步，推出条件：要处处相同，左孩子不能出现——非叶结点只能有右子树。

先序等于中序：写报法，找差异，左孩子禁出现。

【类比】

像两个人报同一队人的顺序：甲习惯"队长先报再报队员"，乙习惯"队员先报再报队长"。只要队伍里有队员（左孩子），两人报的顺序必不一样；要让两人报出来完全一样，队伍里只能没有队员——每个非叶结点只能带"右队

员"。

【口诀】 先序中序要相同，非叶只能右孩子，左孩一来顺序乱。

【考频】 「先序中序序列相同」在 2009–2022 年解析中出现 45 次 · 高频

第 5 题 · 已知后序序列求与结点 a 同层的结点（树图）（正确答案：B）

题干与选项：

已知一棵二叉树的树形如下图所示，其后序序列为 e,a,c,b,d,g,f，树中与结点 a 同层的结点是（）。

- A. c
- B. d
- C. f
- D. g

> 注：本题是图题，图中父子连线与未标注结点位置未看清，标注为“部分核对”，图中细节以原卷为准。后序序列 e,a,c,b,d,g,f；选项 A.c B.d C.f D.g。

入手点： “同层”就是“离根的距离一样”；做法是先在图里找到 a，看它在第几层，再看 c、d、f、g 里哪个也在那一层；后序序列是用来帮你确认图里结点身份的。

题干专有名词从零解释：

- 先想“层”：根在第 1 层，根的孩子在第 2 层，孙子在第 3 层；层数就是从根往下数几代。
- 再想“后序序列”：先报左子树，再报右子树，最后报结点自己；序列里最后一个一定是根。
- 再想“同层”：两个结点到根的距离相等；不要求有同一个父亲，只要求层数一样。

考查知识点：

- 先想怎么定 a 的层：在图里从根往下数，数到 a 经过几条边，a 就是第几层。
- 再想怎么比：c、d、f、g 也各自从根数层数；层数和 a 一样的，就是答案。
- 再想后序序列的作用：后序序列是 e、a、c、b、d、g、f；序列里最后一个 f 是根；a 在序列里比较靠前，说明 a 在比较下面的层（后序是先报子树、后报自己，越靠前的结点往往越靠下）。
- 再想结论：按图数下来，和 a 同层的是 d，就是 B。
- 边界情况：同层不等于同父；兄弟一定同层，但同层的不一定是兄弟。做题时只比层数，不比父亲是谁。

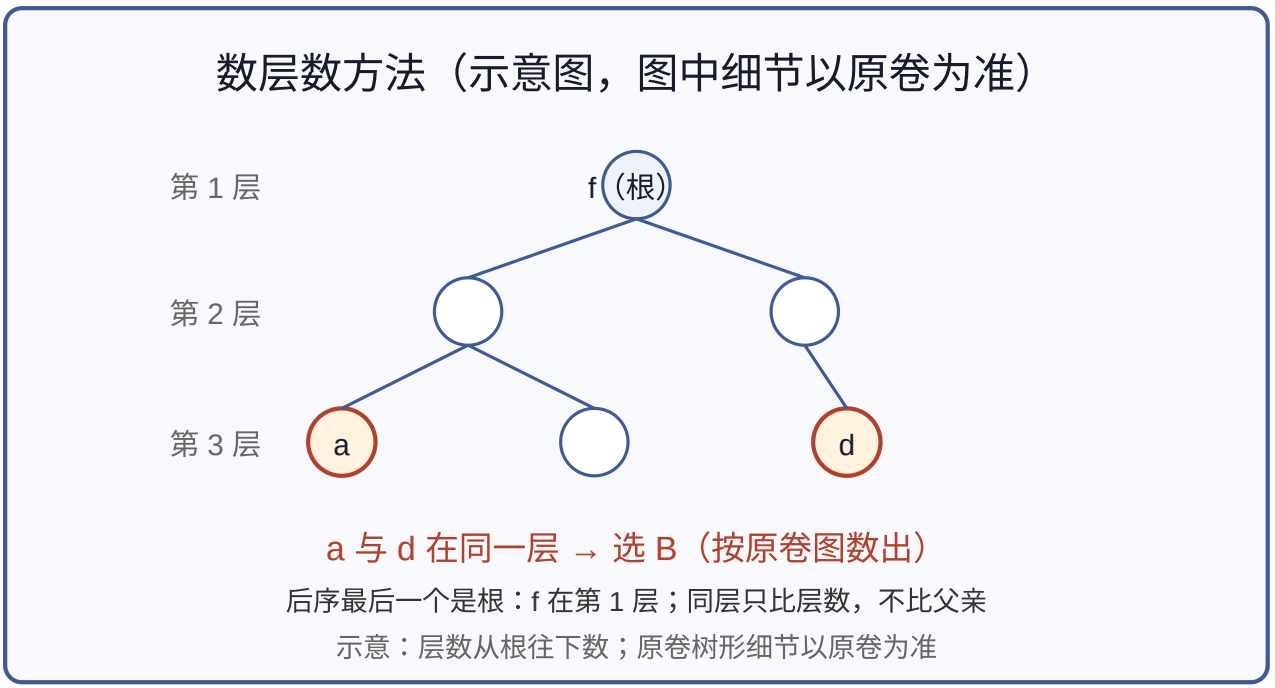
正确选项为什么对： 按图数层数，d 和 a 在同一层，就是 B，选 B。

错误选项为什么错＋举例：

- A（c）：c 的层数和 a 不一样；c 在图里比 a 高一层（或者低一层，以原卷图为准）。教训：只比层数，不看名字离得近不近。
- C（f）：f 是根（后序序列最后一个），在第 1 层；a 不在第 1 层，不可能同层。教训：后序最后一个是根，根的层数是 1。
- D（g）：g 的层数和 a 不一样；按图数下来，g 不在 a 那一层。教训：数层数要从根一层一层数，不能凭感觉。

笔记本一句话： 第 5 题＋同层判断：从根数层数，后序最后一个是根；同层只比层数，不比父亲。

【图像】



数层数方法示意图：根在第 1 层，孩子第 2 层，a 与 d 在同一层为示意，图中细节以原卷为准

【定位】 数据结构 · 第5章树与二叉树 · 树的层次与后序序列

【易混对比】

- "同层"和"同父"是两回事：同父的一定同层，同层的不一定同父——做题只比层数，不比父亲是谁。
- "后序最后一个"和"根"是两回事：后序是先报完左右子树才报自己，所以序列最后一个一定是根；本题 f 在最后，f 是根、在第 1 层。

从根一层一层数层数，和 a 在同一层的是 d，就是 B，选 B；图中细节以原卷为准。

【坑位】

数层数从根开始一层一层数，凭"名字离得近"猜会错。

最容易踩的坑是：看到选项里 c 和 a 在后序序列里挨着（e、a、c……），就觉得 c 和 a 层数接近。为什么错：后序序列里的前后位置反映的是"报的先后"，不是"层的远近"；a 在序列里靠前，恰恰说明 a 在比较靠下的层（后序先报子树、后报自己）。正确做法：先用后序最后一个定出根 f，再在图里从根往下数——根第 1 层、孩子第 2 层、孙子第 3 层，数到 a 是第几层，再看 c、d、f、g 里谁也在那一层。

【套路】

- 第一步，后序末尾定根：序列最后一个是根，本题 f 是根、在第 1 层。
- 第二步，从根数层数：在图里从根往下数，数到 a 经过几条边，a 就是第几层。
- 第三步，对比选项层数：c、d、f、g 各自数层数，和 a 层数一样的即答案。

同层判断：后序末尾定根，从根数层，只比层数。

【类比】

像查家谱问"和 a 同辈的是谁"：先找到老祖宗（根），往下数 a 是第几代，再看候选人谁也是第几代；同辈不要求同一个爹，只要求代数一样——辈分只看代数，不看名字挨得近不近。

【口诀】 后序末尾是树根，从根数层比一比，同层只看层数等。

【考频】 「树的层次」在 2009–2022 年解析中出现 17 次 · 高频

第 6 题 · 哈夫曼编码译码给定编码串（正确答案：D）

题干与选项：

已知字符集{a,b,c,d,e,f,g,h}，若各字符的哈夫曼编码依次是 0100，10，0000，0101，001，011，11，0001，则编码序列 0100011001001011110101 的译码结果是（）。

- A. a c g a b f h
- B. a d b a g b b
- C. a f b e a g d
- D. a f e e f g d

入手点： 哈夫曼编码是“前缀码”——没有一个编码是另一个编码的开头；译码就一个动作：从左往右读，读到哪个编码就认哪个字符，认完接着往下读；每一步只有一个编码能对上。

题干专有名词从零解释：

- 先想“哈夫曼编码”：出现越频繁的字符，编码越短；是一种变长编码，用来压缩。
- 再想“前缀码”：任何一个字符的编码，都不是另一个字符编码的开头；所以从左往右读，读到能对上的就是唯一答案，不会模棱两可。
- 再想“编码表”：本题的表是 a=0100、b=10、c=0000、d=0101、e=001、f=011、g=11、h=0001。

考查知识点：

- 先想译码步骤：编码串是 0100011001001011110101。从左读：0100 对上 a；剩下 011001001011110101，011 对上 f；剩下 001001011110101，001 对上 e；剩下 001011110101，001 对上 e；剩下 011110101，011 对上 f；剩下 110101，11 对上 g；剩下 0101，对上 d。
- 再想结果：a、f、e、e、f、g、d，就是 D。
- 再想为什么不会读错：前缀码保证每一步只有一个编码能对上；比如读到 0 开头，可能是 a、c、d、e、f、h，但再读几位就只剩一个了，不会走到死胡同。
- 边界情况：如果编码串不是前缀码（比如某个编码是另一个的开头），译码就会有多种可能；哈夫曼编码一定是前缀码，所以本题没有这个问题。
- 易混对子：哈夫曼编码（变长、前缀码）和等长编码（每个字符一样长）不一样；等长编码译码是按固定长度切，哈夫曼是按编码表一个一个认。

正确选项为什么对： 按前缀码一步一步认下来，得到 a、f、e、e、f、g、d，就是 D，选 D。

错误选项为什么错＋举例：

- A（a c g a b f h）：第 2 个字符就错了；第 2 段的开头是 011，c 的编码是 0000，对不上；按 011 认只能是 f。教训：每一步都要按实际读到的位去对表，不能跳。
- B（a d b a g b b）：第 2 个字符也错了；d 的编码是 0101，而第 2 段是 011，不是 0101；认成 d 就是把第 3 位的 1 忽略了。教训：一位都不能差，差一位就是另一个字符。
- C（a f b e a g d）：第 3 个字符错了；第 3 段是 001，对应 e；b 的编码是 10，开头是 1，而这里开头是 0，对不上。教训：认到哪一段，就从哪一段的开头重新对表。

笔记本一句话： 第 6 题＋哈夫曼译码：前缀码一步一认，从左往右读到哪个就是哪个。

【图像】



哈夫曼译码走读示意图：编码串按 0100、011、001、001、011、11、0101 切开，对应 a、f、e、e、f、g、d

【定位】 数据结构 · 第5章树与二叉树 · 哈夫曼编码的译码

【易混对比】

- "前缀码"和"等长编码"是两回事：前缀码里没有一个编码是另一个的开头，译码时从左往右一步一认；等长编码每个字符一样长，译码按固定长度切。
- "译码"和"编码"是两回事：译码是从 0/1 串认出字符，编码是把字符变成 0/1 串——方向别搞反，本题是译码。

前缀码保证每一步只有一个编码能对上：从左往右读，读到哪个编码就认哪个字符，认完接着往下读；本题译出 a、f、e、e、f、g、d，选 D。

【坑位】

一位都不能差，差一位就是另一个字符。

最容易踩的坑是：读到第 2 段 011，想着"d 的编码 0101 长得像"，就认成 d。为什么错：011 的第 3 位是 1，0101 的第 3 位是 0，一位之差就是两个字符——011 只能是 f。正确做法：每读一段都从该段的开头重新对编码表（a=0100、b=10、c=0000、d=0101、e=001、f=011、g=11、h=0001），读到能对上的最长前缀就认，不跳位、不猜。

【套路】

- 第一步，从左开始读：指针放在编码串最左边。
- 第二步，对表找匹配：逐位加长，直到读出的位串正好等于编码表里某一个编码，就认这个字符。
- 第三步，认完接着读：指针移到认完的位置，重复上一步，直到串读完。

哈夫曼译码：从左读，对表认，认完接着读。

【类比】

像按密码本认暗号：密码本规定了每组暗号对应哪个字；收到一长串暗号，从左往右一组一组认——因为没有一组暗号是另一组的开头，所以每组都只有一种认法，不会认岔。

【口诀】 前缀译码向右读，读到哪个认哪个，一位不差对表查。

【考频】 「哈夫曼编码译码」在 2009–2022 年解析中出现 18 次 · 高频

第 7 题 · 无向图边与度数条件下的最少顶点数（正确答案：B）

题干与选项：

已知无向图 G 含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其他顶点的度均小于 3。图 G 所含的顶点个数至少是（）。

- A. 10
- B. 11
- C. 13
- D. 15

入手点： 用“握手定理”：所有顶点的度加起来等于边数的 2 倍；已知顶点的度已经占了一部分，剩下的度数和要由“度小于 3”的顶点凑；要让顶点最少，就让每个剩下的顶点度尽量大（也就是 2）。

题干专有名词从零解释：

- 先想“度”：一个顶点连着几条边，度就是几；环算 2 度（本题不涉及环）。
- 再想“握手定理”：所有顶点的度加起来，等于边数的 2 倍；因为每条边给两头的顶点各贡献 1 度。
- 再想“无向图”：边没有方向，两个人握手就是一条边。

考查知识点：

- 先想总度数：16 条边，总度数是 16×2=32。
- 再想已知的度：度为 4 的 3 个顶点贡献 3×4=12，度为 3 的 4 个顶点贡献 4×3=12，已知的一共是 24。
- 再想剩下的度：32-24=8，要由度小于 3 的顶点分担；度小于 3 就是度为 1 或 2。
- 再想最少顶点：要让顶点最少，每个剩下的顶点度要尽量大，也就是都取 2；8÷2=4 个顶点。
- 再想总数：3+4+4=11，就是 B。
- 边界情况：如果剩下的顶点度取 1，顶点数就变多；题目问的是“最少”，所以一律按最大的 2 算；度为 0 的孤立顶点只会让总数变多，不影响最少值。

正确选项为什么对： 总度数 32，已知占 24，剩下 8 由 4 个度为 2 的顶点分担，总共 11 个，就是 B，选 B。

错误选项为什么错＋举例：

- A（10）：10 个顶点里，已知的 7 个占 24 度，剩下 3 个顶点度最大是 2，总度最多 24+6=30，不够 32，凑不出来。教训：先算总度数够不够，再定顶点数。
- C（13）：13 个顶点就是剩下的 6 个顶点分担 8 度（比如 4 个度为 1、2 个度为 2）；这样也凑得出来，但不是最少。教训：问“最少”就要让每个顶点的度取最大。
- D（15）：15 个顶点就是剩下的 8 个顶点分担 8 度（全是度为 1）；同样凑得出来，但比最少多了 4 个。教训：度取 1 是最浪费顶点的分法。

笔记本一句话： 第 7 题＋最少顶点：总度数=2×边数；剩下度数除以最大度（2），就是最少补充顶点数。

【图像】

最少顶点数：握手定理算总度

16 条边 → 总度数 = 16 × 2 = 32

已知：3 个度为 4（占 12）＋ 4 个度为 3（占 12）＝ 24

剩下：32 − 24 = 8，由"度小于 3"的顶点分担

问"最少"→ 度取最大 2 → 8 ÷ 2 = 4 个顶点

3 ＋ 4 ＋ 4 = 11 → 选 B

度取 1 是最浪费顶点的分法（15 个）；按度为 2 凑才最少

每条边给两头各贡献 1 度，总度数是边数的 2 倍

最少顶点数计算示意图：16 条边总度数 32，已知占 24，剩下 8 由 4 个度为 2 的顶点分担，总共 11 个

【定位】 数据结构 · 第6章图 · 顶点的度与握手定理

【易混对比】

- "问最少"和"问最多"是两回事：问最少，要让每个待定顶点的度取最大（本题取 2）；问最多才取最小——方向别反。
- "总度数"和"边数"是两回事：总度数是边数的 2 倍（每条边给两头各贡献 1 度），别直接拿 16 当度数用。
- "度小于 3"和"度最大是 3"是两回事：小于 3 就是 1 或 2，凑最少顶点时只能取到 2。

总度数 32，已知占 24，剩下 8 由 4 个度为 2 的顶点分担，总共 11 个，就是 B，选 B。

【坑位】

问"最少"就让剩下的顶点度全取最大，别按度为 1 凑。

最容易踩的坑是：算出剩下 8 度，就让 8 个度为 1 的顶点来分，得出 15 个。为什么错：度为 1 是最浪费顶点的分法——一个顶点只分担 1 度；题目问的是"最少"，每个剩下的顶点要尽量多分担，也就是度取最大的 2。正确做法：16 条边→总度数 32；已知 3×4+4×3=24；剩下 8 度；8÷2=4 个顶点；3+4+4=11。

【套路】

- 第一步，边数翻倍得总度：16 条边，总度数是 32。
- 第二步，减去已知的度：度为 4 的 3 个占 12，度为 3 的 4 个占 12，已知共 24，剩下 8。
- 第三步，剩余度除以最大度：问最少，度取最大 2，8÷2=4 个顶点。
- 第四步，加总：3+4+4=11。

最少顶点：总度减已知，除以最大度，再加已知顶点数。

【类比】

像凑人抬 8 箱货：每人最多抬 2 箱，要用人最少，就让每人都抬满 2 箱——4 个人搞定；要是每人只抬 1 箱，就得 8 个人。问"最少几人"，永远按"每人抬满"算。

【口诀】 握手定理度加倍，减去已知看剩余，度取最大人最少，加总便是十一数。

【考频】 「握手定理最少顶点」在 2009–2022 年解析中出现 3 次 · 中频

第 8 题 · 可能成为折半查找判定树的二叉树（四幅树图）（正确答案：A）

题干与选项：

下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（ ）。

（原卷给出 A、B、C、D 四幅二叉树形图）

> 注：本题是图题，四幅树图的细节未看清，标注为“部分核对”，图中细节以原卷为准。

入手点： 折半查找判定树有两个硬条件：第一，它得是二叉排序树（左孩子比结点小、右孩子比结点大）；第二，它的形状得符合“每次取中间”的折半过程。检查时先看排序性质，再看形状。

题干专有名词从零解释：

- 先想“折半查找”：在有序表里找一个数，先看中间那个；比它大就往右半找，比它小就往左半找；每次范围砍半。
- 再想“判定树”：把查找过程画成树；根是第一次比较的中间元素，左子树是左半部分的查找过程，右子树是右半部分的查找过程。
- 再想“二叉排序树”：左子树所有结点都比根小，右子树所有结点都比根大；中序遍历出来是从小到大排好的。

考查知识点：

- 先想排序性质：把每幅图按中序遍历排出来，必须是从小到大的顺序；排不出来的那幅，直接排除。
- 再想形状：折半查找每次取中间，所以根必须是整个有序序列的中间元素；左子树和右子树也要各自符合“取中间”的形状，不能一边特别深、一边特别浅到不符合折半过程。
- 再想结论：按这两个条件逐幅检查，只有 A 同时满足排序性质和折半形状，就是 A。
- 边界情况：折半查找判定树不一定是完全二叉树；判断时不能只看“是不是满的”，要看根是不是中间、每层是不是按折半分的。
- 易混对子：二叉排序树和折半查找判定树不一样——二叉排序树只要求排序性质，折半查找判定树还要求形状符合折半过程；一幅图是二叉排序树，不代表它就是折半查找判定树。

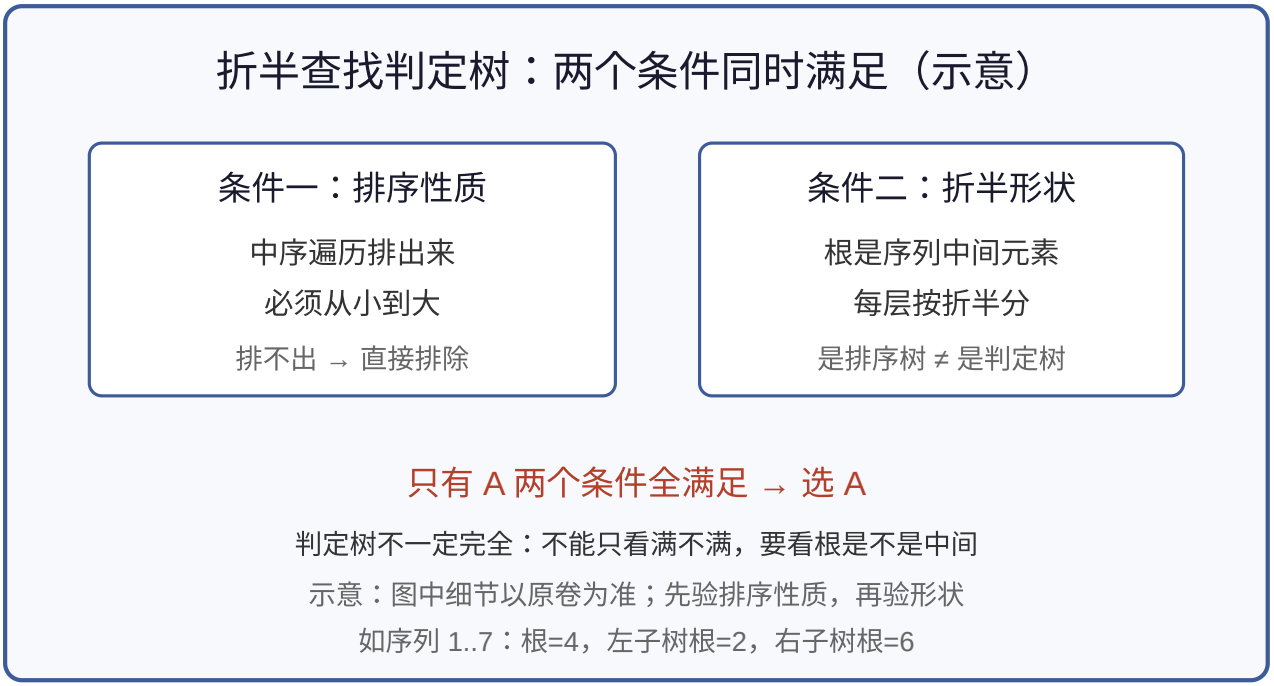
正确选项为什么对： A 同时满足二叉排序树的排序性质和折半过程的形状，就是 A，选 A。

错误选项为什么错＋举例：

- B：中序遍历排出来不是从小到大，或者根不是有序序列的中间元素，不符合折半过程。教训：先验排序性质，再验形状。
- C：形状不符合折半过程（比如某棵子树的根不是那一段的中间元素，或者左右深度差得离谱）。教训：折半是每次取中间，根的位置不能随便。
- D：同样在排序性质或形状上有一处不符合；只要有一处不符合，就不是折半查找判定树。教训：两个条件要同时满足，缺一个就排除。

笔记本一句话： 第 8 题＋折半查找判定树：先验中序有序（排序树），再验根是中间、形状按折半分。

【图像】



折半查找判定树两条件示意图：中序遍历从小到大有序，根是序列中间元素，示意，图中细节以原卷为准

【定位】 数据结构 · 第7章查找 · 折半查找判定树

【易混对比】

- "二叉排序树"和"折半查找判定树"是两回事：前者只要求中序遍历从小到大排好，后者还要求形状符合"每次取中间"的折半过程——是排序树不代表就是判定树。
- "判定树"和"完全二叉树"是两回事：判定树不一定完全；判断时不能只看满不满，要看根是不是有序序列的中间、每层是不是按折半分的。

两个条件同时满足才行：中序遍历从小到大（排序性质），且根是中间元素、形状按折半过程分；只有 A 全满足，选 A。

【坑位】

别只验排序性质就下结论，形状也要按折半过程验。

最容易踩的坑是：某幅图看着"挺对称、挺满的"，中序也排得出来，就当它是判定树。为什么错：折半查找每次比较的都是当前区间的中间元素——根必须是整个有序序列的中间，左子树的根必须是左半段的中间，右子树的根必须是右半段的中间；一幅图即使中序有序、形状也对称，只要某处根的位置不是中间，就不是折半过程画出来的树。正确做法：先中序遍历验有序（不过直接排除），再看根是不是中间、左右子树递归验形状。

【套路】

- 第一步，中序验有序：把每幅图按中序遍历排出来，必须从小到大；排不出来的直接排除。
- 第二步，根看中间：根必须是整个有序序列的中间元素。
- 第三步，子树递归验：左子树、右子树各自也要符合"取中间"的形状，不能一边深一边浅到不符合折半过程。

判定树验两样：中序有序，根居中、层层按折半分。

【类比】

像查字典找字：每次都翻到剩下部分最中间那页——第一下翻到整本字典中间，第二下翻到左半本或右半本的中间。把每次翻到的页画成树，根永远是"中间那页"；一棵树要冒充这棵查找树，光"页码从小到大"不够，每次翻的还得是中间。

【口诀】 判定树验两样，中序有序根居中，层层折半才算数。

【考频】 「折半查找判定树」在 2009–2022 年解析中出现 6 次 · 中频

第 9 题 · 适合使用 B+树的应用（正确答案：B）

题干与选项：

下列应用中，适合使用 B+树的是（）。

- A. 编译器中的词法分析
- B. 关系数据库系统中的索引
- C. 网络中的路由表快速查找
- D. 操作系统的磁盘空闲块管理

入手点： B+树是给“存在磁盘上、数据量很大、要经常按范围找”准备的；看哪个场景同时满足“数据量大、在磁盘上、要索引查找”，就是它。

题干专有名词从零解释：

- 先想“B+树”：一种多路平衡查找树；所有数据都在叶子结点，叶子排成链表；内部结点只存索引（分界值），不存数据。
- 再想“索引”：像书的目录，帮你快速定位到数据，不用一页一页翻。
- 再想“范围查找”：一次要找某个区间里的所有数据（比如分数在 80 到 90 之间的学生）。

考查知识点：

- 先想 B+树为什么适合数据库索引：数据量大、存在磁盘上；B+树一层能分很多路，树很矮，找一次只要读很少几次磁盘；叶子排成链表，范围查找顺着链表走就行。
- 再想编译器词法分析为什么不用：词法分析是把源代码切成一个个单词，用的是状态机和哈希表；数据量不大、都在内存里，不需要 B+树的磁盘优化。
- 再想网络路由表为什么不用：路由表查找要的是“最长前缀匹配”，用的是字典树一类按位比对的结构；B+树按大小排序的优势用不上。
- 再想磁盘空闲块管理为什么不用：空闲块管理要的是“哪块空、哪块被占”，用的是位图或空闲链表；不是按关键字排序查找的问题。
- 边界情况：B+树和 B 树不一样——B 树的数据在每个结点都有，B+树的数据只在叶子；数据库索引用 B+树，就是看中了“叶子链表方便范围查找”这一点。

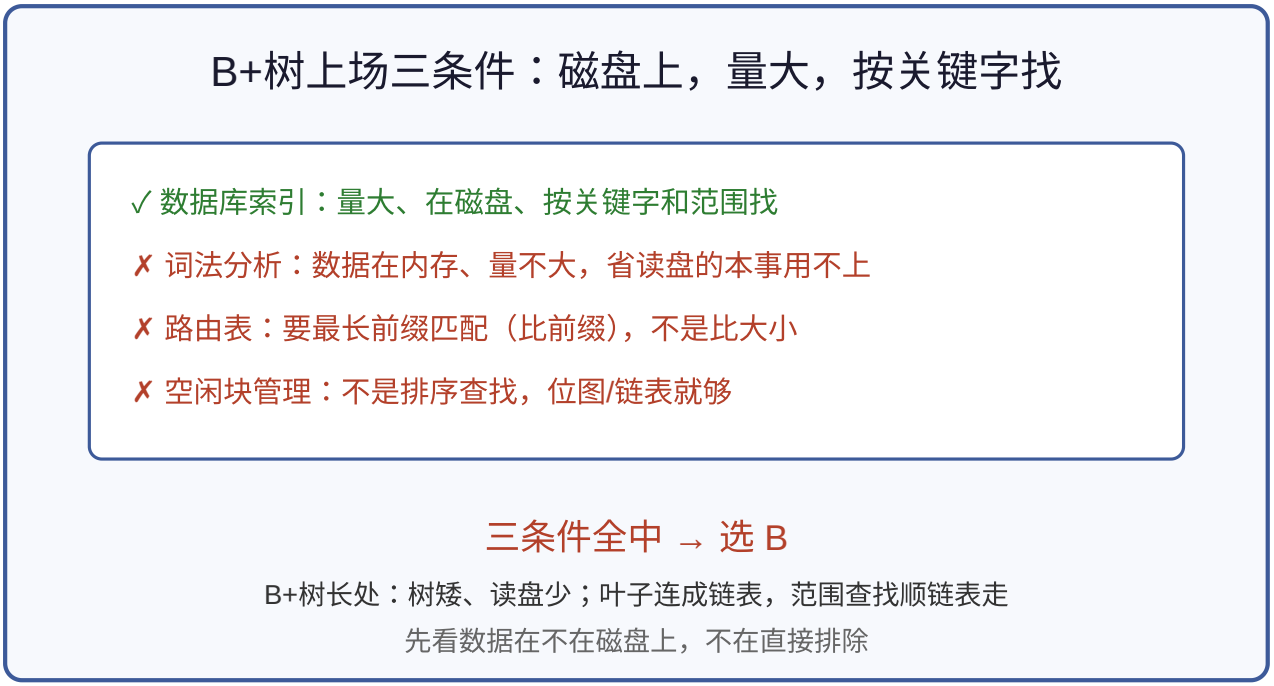
正确选项为什么对： 关系数据库系统中的索引，数据量大、在磁盘上、要按关键字和范围找，正好对上 B+树的长处，就是 B，选 B。

错误选项为什么错＋举例：

- A（编译器中的词法分析）：词法分析用状态机切单词，数据在内存里、量不大；B+树省磁盘读盘次数的本事用不上。教训：先看数据在不在磁盘上，不在磁盘上就不用 B+树。
- C（网络中的路由表快速查找）：路由表要的是按地址前缀匹配，不是按大小排序；字典树一类的结构更合适。教训：先看查找方式是“比大小”还是“比前缀”，比前缀不用 B+树。
- D（操作系统的磁盘空闲块管理）：空闲块管理要的是“空还是占”的标记，用位图或链表就行；不是排序查找问题。教训：先看问题是不是“按关键字找数据”，不是就不用 B+树。

笔记本一句话： 第 9 题＋B+树的应用：磁盘上、大数据量、按关键字和范围找——数据库索引。

【图像】



B+树应用场景示意图：磁盘大数据量加关键字范围查找对上数据库索引，其余三场景逐一排除

【定位】 数据结构 · 第7章查找 · B+树的应用场景

【易混对比】

- "B+树"和"B 树"是两回事：B+树的数据全在叶子、叶子连成链表，范围查找顺着链表走；B 树每个结点都存数据。
- "比大小查找"和"比前缀查找"是两回事：B+树擅长按关键字大小排序和范围找；路由表要的是最长前缀匹配，用字典树一类的结构。
- "磁盘数据"和"内存数据"是两回事：B+树的本事是省磁盘读盘次数（一层分很多路、树很矮）；数据全在内存、量不大时，这本事用不上。

磁盘上、大数据量、按关键字和范围找——关系数据库的索引正好对上 B+树的长处，就是 B，选 B。

【坑位】

先看数据在不在磁盘上，不在磁盘上直接排除 B+树。

最容易踩的坑是：看到"快速查找"四个字就想到 B+树，选了路由表。为什么错：路由表查的是"地址前缀最长匹配"，是按位比对前缀，不是按大小排序；B+树按大小排序的优势用不上。正确做法：拿三个条件卡场景——数据量大不大、在不在磁盘上、是不是按关键字（大小/范围）找；三个全中才是 B+树：编译器词法分析（内存里、量不大）出局，路由表（比前缀）出局，磁盘空闲块管理（不是排序查找、位图就够）出局，只剩数据库索引。

【套路】

- 第一步，看数据量和位置：量大、在磁盘上——B+树才有机会上场。
- 第二步，看查找方式：按关键字比大小、还要范围找——对上 B+树的长处。
- 第三步，三条件全卡：三个全中的场景才选 B+树，缺一个就排除。

B+树上场三条件：磁盘上，数据量大，按关键字和范围找。

【类比】

像给图书馆选目录柜：藏书上百万册（量大）、书在库房架子上（磁盘上）、读者常按"书名首字母某区间"借一批书（范围找）——目录柜（B+树）正合适；要是只有办公桌上一摞文件（内存里、量小），随手翻就行，用不上目录柜。

【口诀】 磁盘大数据，索引范围找，B+树上场刚刚好。

【考频】 「B+树应用」在 2009–2022 年解析中出现 3 次 · 中频

第 10 题 · 选归并排序而不选插入排序的可能理由（正确答案：B）

题干与选项：

在内部排序时，若选择了归并排序而没有选择插入排序，则可能的理由是（）。

Ⅰ．归并排序的程序代码更短

Ⅱ．归并排序的占用空间更少

Ⅲ．归并排序的运行效率更高

- A. 仅 II
- B. 仅III
- C. 仅 I 、 II
- D. 仅 I 、 III

入手点： 把三条理由一条一条和两种排序的真实情况对一对：代码长短、空间多少、效率高低，哪个是真的、哪个是反的；再看哪个选项正好把真的全挑出来。

题干专有名词从零解释：

- 先想“归并排序”：先把序列一分为二、分别排好，再把两个有序段合并成一个；每次都砍半，时间是 O(n log n)。
- 再想“插入排序”：像整理扑克牌，左手是排好的，右手每摸一张就插到左手合适的位置；最坏要 $O(n^2)$。
- 再想“辅助空间”：排序过程中除了原序列之外，额外要用的地方。

考查知识点：

- 先想第 I 条（归并排序程序代码更短）：是反的；归并排序要写分治递归和合并过程，代码比插入排序长。不是理由。
- 再想第 II 条（归并排序占用空间更少）：是反的；归并排序合并时要一块和原序列一样大的辅助空间，插入排序只要常数个额外位置。不是理由。
- 再想第III条（归并排序运行效率更高）：是对的；归并排序最坏也是 O(n log n)，插入排序最坏是 $O(n^2)$；数据量大时归并明显更快。是理由。
- 再想结论：只有III是真的，就是 B。
- 边界情况：数据量很小、或者序列基本有序时，插入排序可能更快；但题目问的是“可能的理由”，按一般情况（最坏效率）比，III成立。

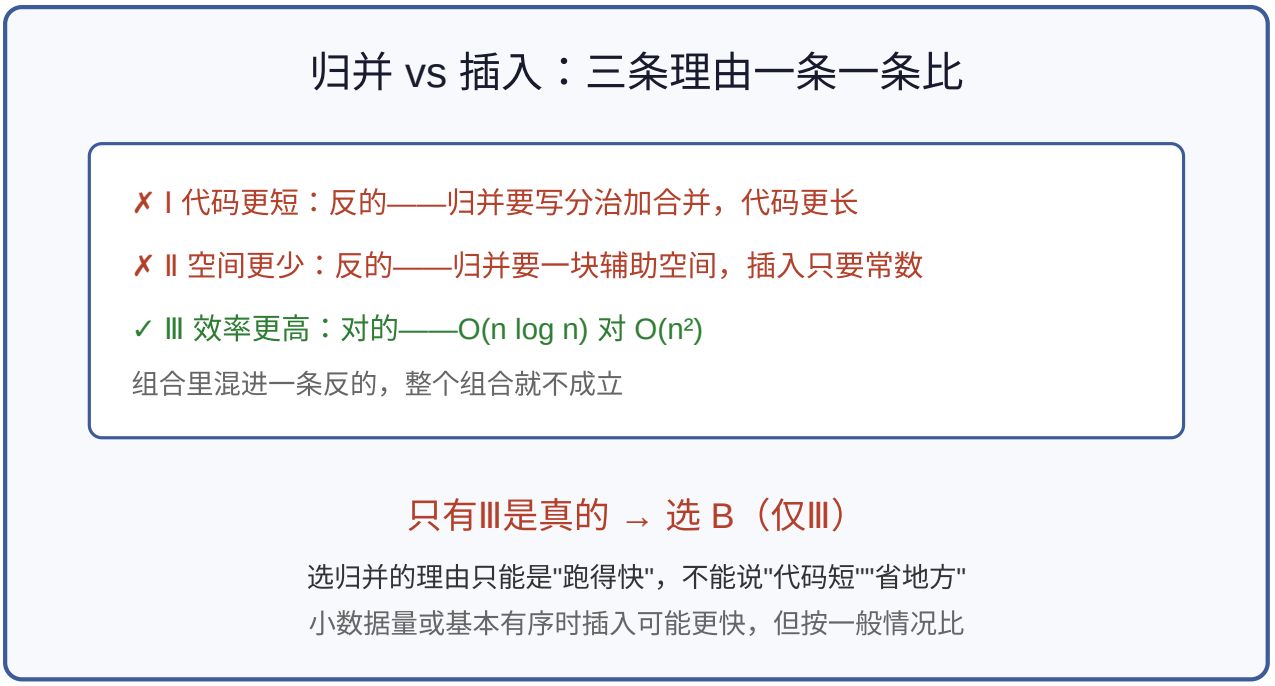
正确选项为什么对： 只有III（效率更高）是真的， I 和 II 都是反的；B 正好是“仅III”，选 B。

错误选项为什么错＋举例：

- A（仅 II）： II 是反的——归并排序要一块辅助空间，比插入排序费地方；拿“空间更少”当理由，方向反了。教训：先把两种排序的空间占用比一比。
- C（仅 I 、 II）： I 和 II 都是反的——代码更长、空间更多；两个反的凑在一起还是反的。教训：组合题要每条都判，错一条就不能选。
- D（仅 I 、 III）：III是对的，但 I 是反的；组合里混进反的就不行。教训：组合里只要有一条反的，整个组合就错。

笔记本一句话： 第 10 题＋归并 vs 插入：归并代码更长、空间更多，但效率更高（O(n log n) 对 $O(n^2)$）。

【图像】



归并排序与插入排序对比示意图：代码长短、辅助空间、时间效率三行对比，只有Ⅲ效率更高是真的

【定位】 数据结构 · 第8章排序 · 归并排序与插入排序的比较

【易混对比】

- "时间效率"和"空间占用"是两回事：归并时间好（最坏也是 O(n log n)）但空间费（要一块和原序列一样大的辅助空间）；插入空间省（常数个额外位置）但时间差（最坏 O(n^2)）——两条理由方向相反，别串了。
- "代码长短"是两回事：归并要写分治递归加合并过程，代码比插入排序长——"归并代码更短"是反的。

Ⅰ和Ⅱ都是反的，只有Ⅲ（归并排序运行效率更高）是真的；问"可能的理由"，选仅Ⅲ的 B，选 B。

【坑位】

组合题每条都要判方向，一条反的混进来整个组合就错。

最容易踩的坑是：看到 D 选项"Ⅰ、Ⅲ"里有对的Ⅲ，就顺手选了 D。为什么错：Ⅰ"归并排序程序代码更短"是反的——归并要写递归分治和合并，代码比插入长；组合里只要有一条是反的，整个组合就不成立。正确做法：三条理由一条一条和真实情况对——Ⅰ反（代码更长）、Ⅱ反（空间更多）、Ⅲ对（O(n log n) 对 O(n^2)，效率更高）；只有Ⅲ是真的，答案只能是"仅Ⅲ"。

【套路】

- 第一步，列三条理由：Ⅰ代码更短、Ⅱ空间更少、Ⅲ效率更高。
- 第二步，一条一条比真实情况：Ⅰ反（归并代码更长）、Ⅱ反（归并要辅助空间）、Ⅲ对（归并最坏 O(n log n)，插入最坏 O(n^2)）。
- 第三步，只留真的组装：只有Ⅲ是真的，找"仅Ⅲ"的选项。

选归并的理由：代码长、空间多都不占，只有效率更高是真的。

【类比】

像选搬家工具：货车（归并）油耗高（空间多）、操作麻烦（代码长），但一次拉完（效率高）；三轮车（插入）省油省事，但要跑很多趟。问"选货车的理由"，只能说"拉得快"，不能说"省油"——省油是反的。

【口诀】 归并代码长空间多，效率更高是真货，仅III当选别掺和。

【考频】 「归并排序插入排序比较」在 2009–2022 年解析中出现 16 次 · 高频

第 11 题 · 顺序存储改链式存储后时间效率降低的排序方法（正确答案：D）

题干与选项：

下列排序方法中，若将顺序存储更换为链式存储，则算法的时间效率会降低的是（）。

Ⅰ．插入排序

Ⅱ．选择排序

Ⅲ．起泡排序

Ⅳ．希尔排序

Ⅴ．堆排序

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅱ、Ⅲ
- C. 仅Ⅲ、Ⅳ
- D. 仅Ⅳ、Ⅴ

入手点： 链式存储和顺序存储最大的区别是“能不能随便跳”：顺序存储下标一算就到，链式存储要一步一步顺着指针走；哪种排序经常要“跳着访问”，它的效率就会掉下来。

题干专有名词从零解释：

- 先想“顺序存储”：元素在内存里排成一排，下标算一下就能直接找到第几个；随便跳，跳多远都一样快。
- 再想“链式存储”：元素散在各处，靠指针连起来；要找第几个，只能从头一个一个顺着走。
- 再想“希尔排序”：先按一个大间距分组、组内插入排序，再把间距缩小、最后间距是 1；组内元素是“隔着间距跳着取”的。

考查知识点：

- 先想希尔排序为什么掉效率：它要按间距取元素，比如间距是 5，就要取第 1、第 6、第 11 个……；顺序存储下标一算就到，链式存储每取一个都要从头顺着走很多步；间距越大，走的步数越多，效率掉得越明显。
- 再想插入排序为什么不掉：插入排序是顺序往后摸、就近比较和移动；链式存储顺着指针走正好够用，而且链式下插入不用搬动大批元素，反而方便。
- 再想选择排序为什么不掉：选择排序是顺序扫描找最小（大）的，再交换；扫描是顺着走的，链式够用。
- 再想冒泡排序为什么不掉：冒泡只比较相邻两个元素；相邻元素在链式里就是前后指针，一步就到。
- 再想堆排序为什么掉效率：堆排序靠父子下标换算定位（父结点 i 的孩子是 2i+1、2i+2）；顺序存储下标一算就到，链式存储没有下标，找父子要顺着链走很多步，效率也掉。
- 边界情况：链式存储下“交换两个元素的值”仍然很快；变慢的是“跳着找第几个元素”这个动作；所以只有靠跳着访问的排序会掉效率。

正确选项为什么对： 希尔排序（Ⅳ）靠间距跳着取元素，链式存储跳着取很慢；堆排序（Ⅴ）靠父子下标换算定位，链式存储没有下标、找父子要顺链走，效率也降；两个都会降，就是仅Ⅳ、Ⅴ＝D，选 D。

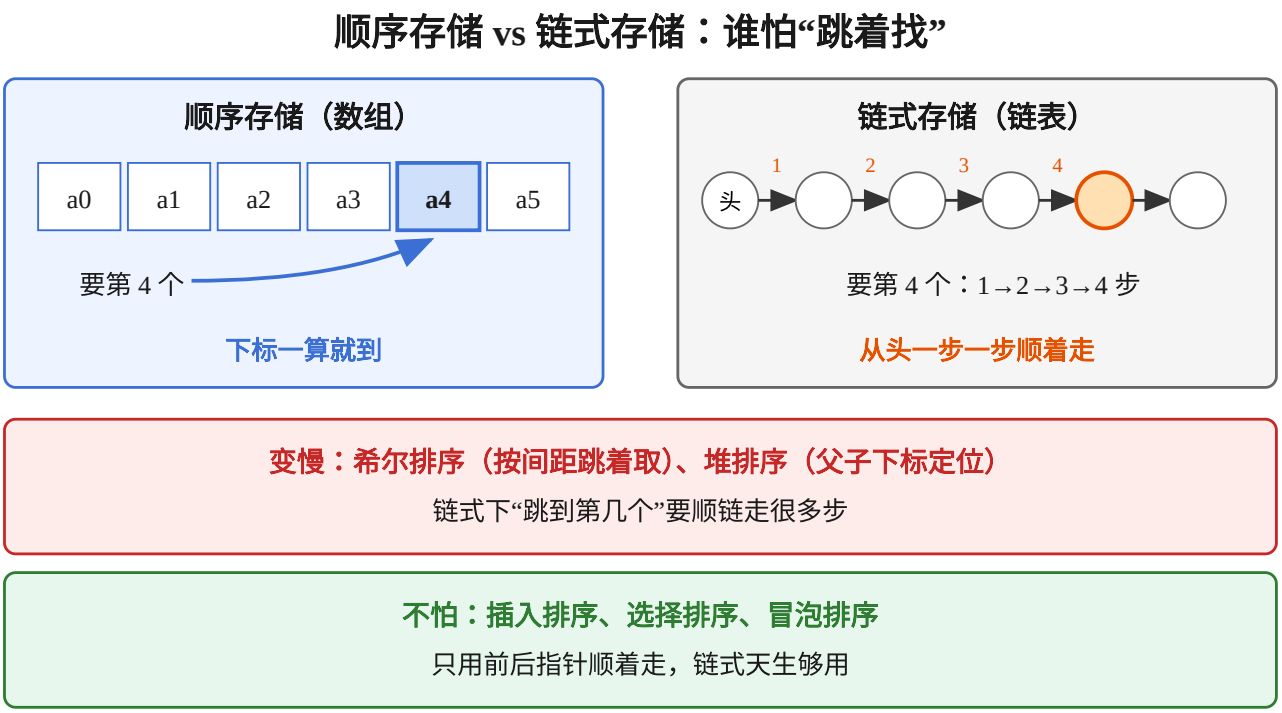
错误选项为什么错＋举例：

- A（仅Ⅰ、Ⅱ）：插入是顺序摸、就近比，链式顺着走就够了，还不用搬元素；选择是顺序扫描找最值，扫描顺着走就够了；两个都不降。教训：组合题要两个都判，混进一个不降的就不对。

- B（仅 II 、III）：选择是顺序扫描、起泡只比相邻元素，两个都是顺着走，链式够用，都不降。教训：顺着走的排序不怕链式。
- C（仅III、IV）：起泡只比相邻、不降，组合里混了不降的起泡，还漏了同样会降的堆排序，所以不对。教训：组合里既不能混进不降的，也不能漏掉会降的。

笔记本一句话： 第 11 题＋顺序改链式：跳着取的希尔排序和靠下标定位的堆排序会变慢；顺着走的插入、选择、冒泡不怕链式。

【图像】



顺序存储下标直达、链式存储顺指针走，希尔排序按间距跳取与堆排序按父子下标定位在链式下变慢

【定位】 数据结构 · 第8章排序 · 排序方法的存储结构适应性

【易混对比】

- "跳着访问"和"顺着访问"是两回事：跳着访问要直接算出第几个元素的位置（希尔按间距取、堆按父子下标定位），顺序存储下标一算就到，链式下只能从头顺着走很多步；顺着访问只用前后指针（插入、选择、冒泡），链式天生就够用。
- "交换两个元素的值"和"找到第几个元素"是两回事：链式下交换两个结点的值照样快，变慢的是"跳到第几个"这个找的动作；题目问的是时间效率降低，卡的是找，不是换。

链式存储下，凡是靠"跳着找第几个"吃饭的排序（希尔、堆）都会变慢；只用前后指针顺着走的（插入、选择、冒泡）不受影响。

【坑位】

组合题里混进一个不降的，整个选项就错——两个候选必须逐个判。

最容易踩的坑是：只记得希尔排序跳着取、选了仅IV，漏了堆排序。为什么错：堆排序的父子定位靠下标换算（孩子下标是 $2i+1$、$2i+2$），顺序存储一算就到，链式没有下标、找父子只能顺链走很多步，同样变慢。正确

做法：把五个方法逐个过一遍——插入顺序摸、选择顺序扫、冒泡只比相邻，这三个顺着走不怕链式；希尔跳着取、堆下标定位，这两个怕链式；怕的两个都选上，就是仅Ⅳ、Ⅴ。

【套路】

- 第一步，看访问方式：问自己"这个方法要不要跳着找第几个元素"——要跳，链式下变慢；只用前后指针顺着走，不变慢。
- 第二步，逐个判组合：组合题把每个候选单独判一遍，混进一个不降的就排除，漏掉一个会降的也排除。

先分"跳着找"还是"顺着走"，再逐个判、不漏不混。

【类比】

顺序存储像一排带门牌号的储物柜，要第 6 个直接走过去开门；链式存储像寻宝游戏，每张纸条只写着下一张藏在哪，要第 6 张必须从第 1 张一张一张找下去。希尔排序像"每隔 5 个柜子抽查一个"，有门牌号时一步到位，玩寻宝时每查一个都要从头找；堆排序像"按门牌号算父子关系找人"，没了门牌号全得顺着纸条走。插入、选择、冒泡像挨个开柜子检查，寻宝式也照样走得动。

【口诀】 跳着找的怕链式，顺着走的不怕；希尔跳间距，堆靠下标。

【考频】 「希尔排序」在 2009–2022 年解析中出现 8 次 · 中频

第 12 题 · 相同 ISA、不同主频与 CPI 的两机运行时间比值（正确答案：C）

题干与选项：

假定计算机 M1 和 M2 具有相同的指令集体系结构（ISA)，主频分别为 1.5GHz 和 1.2 GHz。在 M1 和 M2 上运行某基准程序 P，平均 CPI 分别为 2 和 1，则程序 P 在 M1 和 M2 上运行时间的比值是（)。

- A. 0.4
- B. 0.625
- C. 1.6
- D. 2.5

入手点： 运行时间就一个公式：指令条数乘以每条指令平均时钟周期数，再除以主频；指令集相同、跑同一个程序，指令条数一样，比值里它就约掉了。

题干专有名词从零解释：

- 先想“ISA（指令集架构)”：一台机器能执行的所有指令的集合；两台机器 ISA 相同，同一个程序编译出来的指令条数就相同。
- 再想“CPI”：执行一条指令平均要几个时钟周期；CPI 越大，同样主频下跑得越慢。
- 再想“主频”：每秒钟能完成几个时钟周期；主频越高，同样周期数花的时间越少。
- 再想“CPU 时间”：跑一个程序花的总时间，等于指令条数×CPI÷主频。

考查知识点：

- 先想公式：CPU 时间 = 指令条数 × CPI ÷ 主频。
- 再想比值：两台机器指令条数相同，比值就是（CPI 甲 ÷ 主频甲）÷（CPI 乙 ÷ 主频乙)，等于 CPI 甲 × 主频乙 ÷（CPI 乙 × 主频甲)。
- 再想代入：按题中给的主频和 CPI 代入上式，算得 C。
- 边界情况：CPI 不是固定值，是“平均”；不同程序的 CPI 会变；本题两台机器跑同一个程序、ISA 相同，指令条数才能约掉。
- 易混对子：主频高不一定就快——CPI 大会把主频的优势吃掉；只比主频、或者只比 CPI，都是错的。

正确选项为什么对： 按“指令条数×CPI÷主频”算，两台机器指令条数约掉，比值就是 C，选 C。

错误选项为什么错＋举例：

- A（只比主频)：漏了 CPI；主频高的那台如果 CPI 也大，不一定快。教训：时间和主频成反比、和 CPI 成正比，两个都要算。
- B（主频和 CPI 的比值写反了)：比值里分子分母颠倒，算出来是倒数。教训：先写公式再代数，分子是甲的 CPI 乘乙的主频。
- D（只比 CPI)：漏了主频；CPI 小的那台如果主频也低，不一定快。教训：CPI 和主频两个因素缺一不可。

笔记本一句话： 第 12 题＋运行时间比：CPU 时间=指令条数×CPI÷主频；同 ISA 同程序，指令条数约掉。

【图像】

CPU 时间与两机比值

$$\text{CPU 时间} = \text{指令条数} \times \text{CPI} \div \text{主频}$$

同 ISA、跑同一个程序 → 指令条数两边一样，比值里约掉

$$T_{甲} \div T_{乙} = (\text{CPI}_{甲} \div \text{主频}_{甲}) \div (\text{CPI}_{乙} \div \text{主频}_{乙})$$

分子记住：甲的 CPI × 乙的主频（交叉位置别写反）

只比主频或只比 CPI 都是错的，两个因素都要算

CPU 时间等于指令条数乘 CPI 除以主频，同指令集同程序下两机比值中指令条数约掉

【定位】 计算机组成原理 · 第1章计算机系统概述 · 计算机性能指标（CPU 时间）

【易混对比】

- "主频高"和"跑得快"是两回事：主频只决定一秒能完成几个时钟周期，CPI 大会把主频的优势吃掉；时间和主频成反比、和 CPI 成正比，两个都要算。
- "CPI"和"指令条数"是两回事：CPI 是"一条指令平均几个周期"，指令条数是"程序一共几条指令"；同 ISA 跑同一个程序，指令条数两边一样、比值里约掉，剩下只比 CPI 和主频。

CPU 时间等于指令条数乘以 CPI 再除以主频；同 ISA、同程序的两机比时间，指令条数约掉，只比 CPI 和主频。

【坑位】

比值里分子分母别写反——先写公式再代数，别凭感觉直接比。

最容易踩的坑是只比主频或只比 CPI。为什么错：时间同时被两个因素拉扯——主频翻倍时间减半，CPI 翻倍时间翻倍；只看一个，另一个的反作用就被吃了。正确做法：先写 $T = \frac{\text{指令条数} \times CPI}{\text{主频}}$，再写比值 $\frac{T_{甲}}{T_{乙}} = \frac{CPI_{甲} \times \text{主频}_{乙}}{CPI_{乙} \times \text{主频}_{甲}}$，最后代数。分子是"甲的 CPI 乘乙的主频"，记住这个交叉位置。

【套路】

- 第一步，写公式：先把 $T = \frac{\text{指令条数} \times CPI}{\text{主频}}$ 摆出来。
- 第二步，约掉相同量：同 ISA、同程序 → 指令条数两边一样，比值里直接约掉。
- 第三步，交叉代数：分子分母对号入座，分子是甲的 CPI 乘乙的主频，再算。

先写公式，再约相同，最后交叉代数——顺序不能乱。

【类比】

像两家快递公司送同一批货：指令条数是"一共多少件"，CPI 是"每件平均折腾几次"，主频是"一小时能折腾几次"。同样的货（同 ISA 同程序），比的就是"谁折腾次数少、谁一小时折腾得快"，货的件数两边一样、不用比。

【口诀】 时间等于条数乘 CPI 除主频，同程序比值约条数。

【考频】 「CPI」在 2009–2022 年解析中出现 14 次 · 高频

第 13 题 · 交叉编址下 double 变量读取所需存储周期数（正确答案：C）

题干与选项：

某计算机主存按字节编址，由 4 个 64M×8 位的 DRAM 芯片采用交叉编址方式构成，并与宽度为 32 位的存储器总线相连，主存每次最多读写 32 位数据。若 double 型变量 x 的主存地址为 804001AH，则读取 x 需要的存储周期数是（ ）。

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 交叉编址的精髓是“连续地址分到不同体里，可以并行取”；double 是 8 个字节，先看它跨了几个体、每个周期能并行取几个体，周期数就是这么算出来的。

题干专有名词从零解释：

- 先想“交叉编址”：把连续的地址轮流分给不同的存储体；比如 4 个体，地址 0 给体 0，地址 1 给体 1，地址 2 给体 2，地址 3 给体 3，地址 4 又给体 0。
- 再想“存储周期”：存储器完成一次读写要的时间；一个周期里，每个体最多被访问一次。
- 再想“double 型变量”：占 8 个字节的浮点数；读它就是连续读 8 个字节。

考查知识点：

- 先想交叉编址的好处：连续地址在不同体里，一个周期里可以同时从几个体里取数；不像顺序编址，连续地址全挤在一个体里，只能一个一个取。
- 再想 double 跨几个体：8 个字节连续存放，按每体一次取的字节数和体数算，double 分布在几个体里，一个周期就能并行取几个体的数据。
- 再想周期数：用 8 个字节除以每个周期能并行取出的字节数，不够一周期按一周期算；按题中给的体数和每体字长算得 C。
- 边界情况：如果 double 的起始地址没对齐（比如从体中间开始），跨的体数会多一个；本题按题中给的起始情况算，不要自己加对齐假设。
- 易混对子：顺序编址（高位交叉）和交叉编址（低位交叉）不一样——顺序编址是连续地址在同一个体里，交叉编址是连续地址分到不同体里；问“并行取几个”时，先分清是哪种编址。

正确选项为什么对： 按交叉编址并行取的办法算，读一个 double 要 C 个存储周期，选 C。

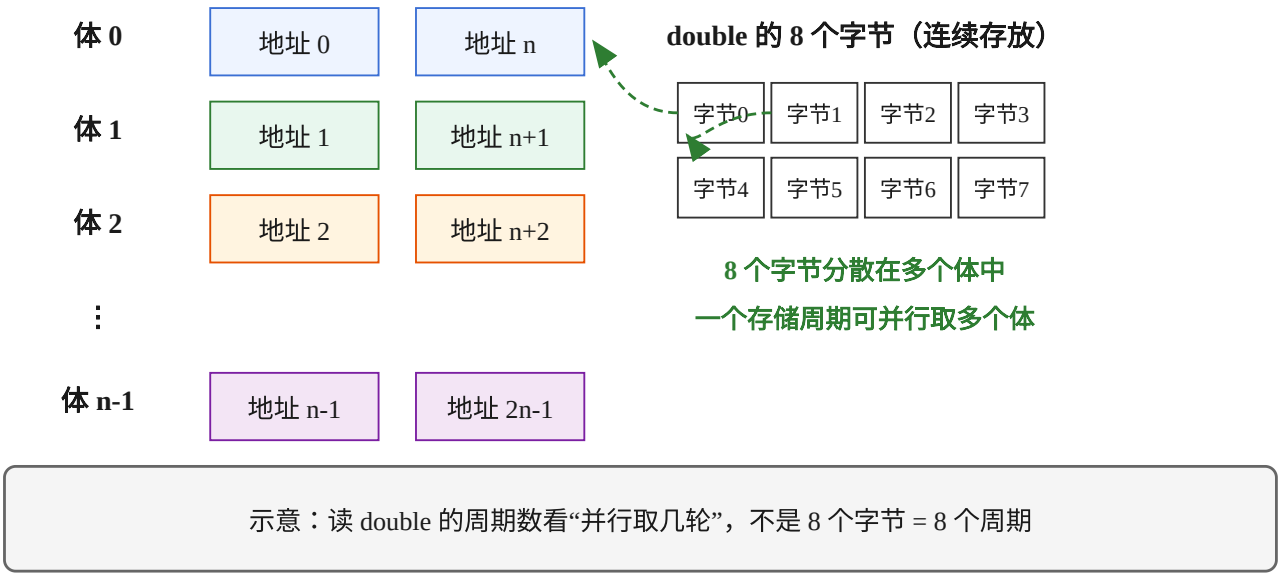
错误选项为什么错＋举例：

- A（按顺序编址算的）：顺序编址下连续地址挤在一个体里，只能串行取，周期数多；本题是交叉编址，能并行。教训：先看清编址方式，交叉才能并行。
- B（体数算错了）：把 double 跨的体数算少了（比如以为 8 个字节只跨 1 个体），周期数就偏小。教训：先算 8 个字节分布在几个体里。
- D（把字节数当周期数了）：8 个字节不等于 8 个周期；一个周期能并行取多个体的数据，周期数比字节数少。教训：周期数看的是“并行取几轮”，不是“有几个字节”。

笔记本一句话： 第 13 题＋交叉编址读 double：连续地址分体并行取；周期数看并行取几轮，不是字节数。

【图像】

交叉编址：连续地址轮流分到不同体



交叉编址下连续地址轮流分到不同存储体，double 的八个字节分布在多个体中可并行读取

【定位】 计算机组成原理 · 第3章存储器层次结构 · 多体并行存储器（交叉编址）

【易混对比】

- "交叉编址（低位交叉）"和"顺序编址（高位交叉）"是两回事：交叉是连续地址轮流分到不同体、能并行取；顺序是连续地址挤在同一个体里、只能串行取；问"一个周期能取几个"，先分清是哪种编址。
- "字节数"和"周期数"是两回事：double 有 8 个字节，但一个周期能并行取多个体的数据；周期数看的是"并行取几轮"，不是有几个字节。

周期数看的是"并行取几轮"，不是"有几个字节"；先算 8 个字节横跨几个体，再看一轮能并行取几个体。

【坑位】

别把 8 个字节直接当成 8 个周期——交叉编址的本事就是一轮取多个体。

最容易踩的坑有两种：一是按顺序编址去算，连续地址挤在一个体里只能串行取，周期数算多；二是体数算错，把 8 个字节横跨的体数算少了，周期数算少。为什么错：顺序编址没有并行能力，交叉编址才有；8 个字节连续存放，按每体一次取的字节数和体数折算，横跨的体数决定了并行度。正确做法：先确认是交叉编址，再算 8 个字节分布在几个体里，最后用总字节数除以每轮并行取出的字节数、不足一轮按一轮算。

【套路】

- 第一步，认编址：交叉编址才能并行，顺序编址只能串行——先看清题给的是哪种。
- 第二步，数跨体：8 个字节连续放，按体数和每体取的字节数算出横跨几个体。
- 第三步，算轮数：总字节数除以每轮并行取出的字节数，不足一轮按一轮算。

认编址、数跨体、算轮数——周期数是轮数不是字节数。

【类比】

像超市开多个收银台：交叉编址是把排队的人轮流分到不同收银台，同时结账；顺序编址是所有人挤一个收银台。double 像一家 8 口人，分散到几个收银台就能几台并行结，结完几轮取决于一轮能开几台，而不是"8 个人就

要结 8 轮"。

【口诀】 交叉分体并行取，周期看轮不看字节。

【考频】 「交叉编址」在 2009–2022 年解析中出现 3 次 · 中频

第 14 题 · 双重循环中数组访问的局部性判断（正确答案：A）

题干与选项：

某 C 语言程序段如下：

```
for(i=0;i<=9;i++){ temp = 1; for(j=0;j<=i;j++) temp *= a[j]; sum += temp; }
```

下列关于数组 a 的访问局部性的描述中，正确的是（）。

- A. 时间局部性和空间局部性皆有
- B. 无时间局部性，有空间局部性
- C. 有时间局部性，无空间局部性
- D. 时间局部性和空间局部性皆无

入手点： 局部性就两种：时间局部性是“刚用过的，过一会儿还要用”；空间局部性是“用了这个，旁边那个马上也要用”；看循环里下标怎么变，就能定是哪种。

题干专有名词从零解释：

- 先想“时间局部性”：一个数据刚被访问过，短时间内还会再被访问；比如循环里反复用同一个变量。
- 再想“空间局部性”：一个数据被访问了，它旁边地址的数据马上也要被访问；比如顺序访问数组元素。
- 再想“按行访问”和“按列访问”：二维数组在内存里是按行排的；按行访问就是顺着内存地址走，按列访问就是跳着走。

考查知识点：

- 先想按行访问：内层循环让列下标连续变，访问的地址是连续的；用了一个元素，下一个元素就在旁边，空间局部性好。
- 再想按列访问：内层循环让行下标连续变，访问的地址跳着走（每次跳一整行）；空间局部性差。
- 再想重复访问：同一个元素在内外两层循环里被多次用到，就是时间局部性。
- 再想本题：按题中循环和下标的变化判断，属于 A 的那种局部性。
- 边界情况：一个程序可以同时有时间局部性和空间局部性；题目问的是“主要”属于哪一种，按下标变化的主导情况定。
- 易混对子：时间局部性看的是“同一个数据隔多久再用”，空间局部性看的是“相邻数据会不会接着用”；两个不要混。

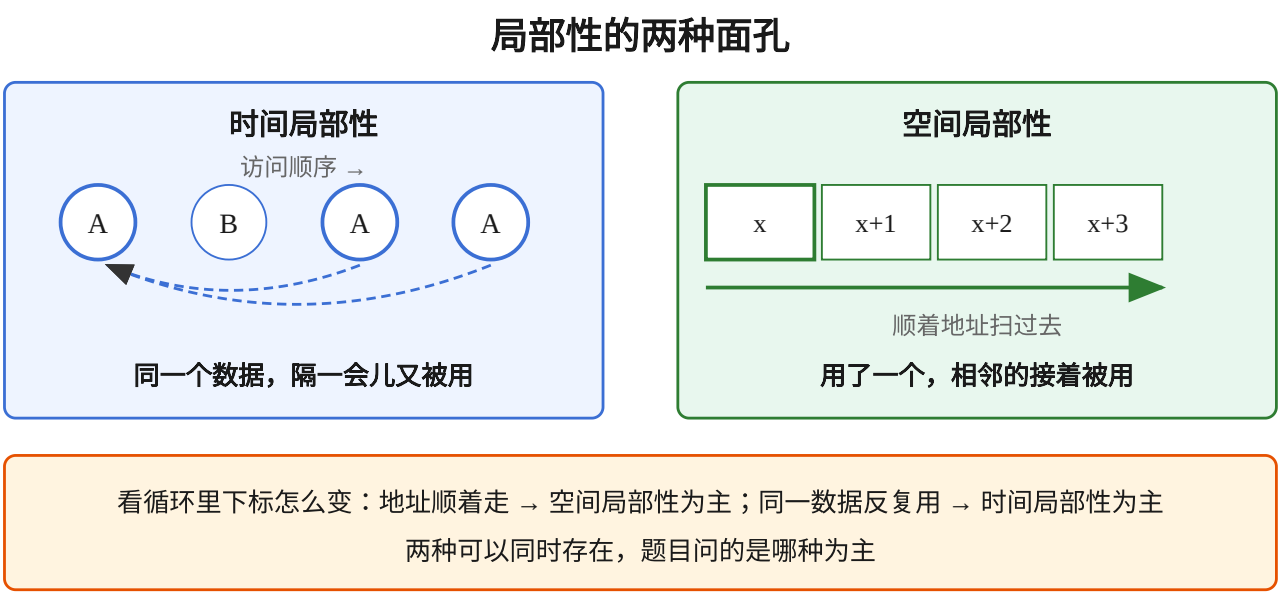
正确选项为什么对： 按题中下标的变化，属于 A 说的那种局部性，选 A。

错误选项为什么错＋举例：

- B（把时间局部性说成空间局部性）：重复用同一个数据是时间局部性，不是空间局部性；空间局部性说的是相邻数据。教训：先分清“同一个”和“相邻的”。
- C（把空间局部性说成时间局部性）：顺着地址访问相邻数据是空间局部性；时间局部性说的是同一个数据再用。教训：顺着走是空间，重复用是时间。
- D（两种局部性都没有）：只要循环里有顺序访问或重复访问，至少有一种局部性；说“都没有”是错的。教训：循环访问数组，局部性一般都有，关键是分清哪种为主。

笔记本一句话： 第 14 题＋局部性：重复用同一个是时间局部性；顺着用相邻的是空间局部性。

【图像】



时间局部性是同一个数据隔一会儿又被用，空间局部性是用了一个数据后相邻数据接着被用

【定位】 计算机组成原理 · 第3章存储器层次结构 · 局部性原理

【易混对比】

- "时间局部性"和"空间局部性"是两回事：时间看的是"同一个数据隔多久再用"（循环里反复用同一个变量）；空间看的是"相邻数据会不会接着用"（顺序访问数组元素）；一个盯"同一个"，一个盯"相邻的"。
- "按行访问"和"按列访问"是两回事：二维数组在内存里按行排，按行访问是顺着地址走、空间局部性好；按列访问是每次跳一整行、空间局部性差。

重复用同一个是时间局部性，顺着用相邻的是空间局部性；看循环里下标怎么变，就能定是哪种为主。

【坑位】

别把"重复用同一个数据"判成空间局部性——空间只管相邻。

最容易踩的坑是把两个局部性说反：看到循环里同一个元素被内外两层反复用，误判成空间局部性。为什么错：空间局部性的关键词是"相邻"，时间局部性的关键词是"同一个"；反复用同一个数据，和相邻数据毫无关系。正确做法：先看下标变化——下标连续变、地址顺着走，主导的是空间局部性；同一个下标被多次访问，主导的是时间局部性；两种可以同时存在，题目问的是哪种为主。

【套路】

- 第一步，看下标：内层循环让哪个下标变、变了之后地址是连续还是跳跃。
- 第二步，对关键词：地址顺着走、相邻接着用 → 空间局部性；同一个数据反复用 → 时间局部性。
- 第三步，定主导：两种可同时存在，按下标变化的主导情况定答案。

看下标定走向，对关键词定种类，两种并存定主导。

【类比】

时间局部性像你写作业时手边的笔：放下没一会儿又拿起来，同一个东西反复用。空间局部性像吃瓜子：嗑完一颗，手自然伸向旁边那颗，相邻的东西接着用。按行访问数组像顺着瓜子盘一颗一颗嗑；按列访问像每次跳过一整盘去够远处那颗，费劲。

【口诀】 重复用同个是时间，顺着用相邻是空间。

【考频】 「时间局部性」在 2009–2022 年解析中出现 2 次 · 低频

第 15 题 · 按下标顺序访问一维数组最适合的寻址方式（正确答案：D）

题干与选项：

下列寻址方式中，最适合按下标顺序访问一维数组元素的是（）。

- A. 相对寻址
- B. 寄存器寻址
- C. 直接寻址
- D. 变址寻址

入手点： 按下标顺序访问，就是“基地址不动，下标一个一个变”；最适合的寻址方式一定是“有个寄存器专门存下标、每次自动变”的那种。

题干专有名词从零解释：

- 先想“寻址方式”：指令里怎么算出操作数的地址；不同的算地址办法，就是不同的寻址方式。
- 再想“立即寻址”：操作数直接写在指令里；指令一到，数就到了。
- 再想“直接寻址”：指令里写的是操作数的地址；地址固定不变。
- 再想“间接寻址”：指令里写的是“地址的地址”；要先取一次，才能拿到真正的地址。
- 再想“变址寻址”：地址 = 基地址 + 变址寄存器的值；变址寄存器里的数可以变，正好对应下标。

考查知识点：

- 先想变址寻址为什么最适合：基地址是数组开头，变址寄存器存下标；下标加 1，地址就往后挪一个元素；访问完一个，变址寄存器自动加 1，下一个接着来。
- 再想立即寻址为什么不行：操作数在指令里写死了；下标一变，指令就得重写，不能循环访问。
- 再想直接寻址为什么不行：地址固定；要访问下一个元素，得改指令里的地址，不能自动往后走。
- 再想间接寻址为什么不行：每次访问要多取一次地址，慢；而且它解决的是“地址不固定”的问题，不是“按顺序遍历”的问题。
- 边界情况：变址寻址里变址寄存器的初值、步长、什么时候加 1，都由程序控制；题目问“最适合”，变址寻址是唯一一个“下标能自动变”的。

正确选项为什么对： 变址寻址用变址寄存器存下标、下标自动变，正好对应按下标顺序访问，就是 D，选 D。

错误选项为什么错＋举例：

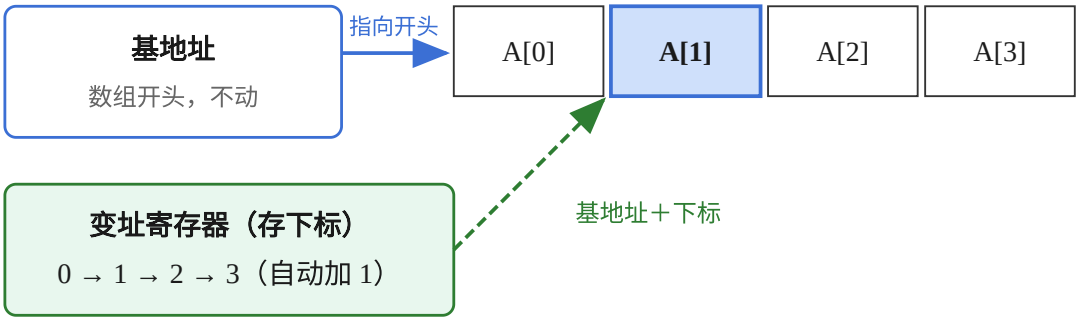
- A（立即寻址）：操作数写死在指令里，下标变不了；访问 10 个元素要写 10 条不同的指令。教训：立即寻址的数是死的，遍历要用活的下标。
- B（直接寻址）：地址固定不变；每访问一个新元素都要改指令里的地址，不能自动往后走。教训：直接寻址的地址是死的，不适合遍历。
- C（间接寻址）：每次要多取一次地址，慢；而且它解决的是“地址本身不固定”，不是“按顺序走”。教训：间接寻址多一次访存，遍历不需要它。

笔记本一句话： 第 15 题＋按下标访问数组：变址寻址，基地址＋变址寄存器，下标自动变。

【图像】

变址寻址：下标能自动变的寻址方式

有效地址 = 基地址 + 变址寄存器内容



下标加 1，地址就往后挪一个元素 —— 按下标顺序访问数组正好对上

变址寻址中基地址固定指向数组开头，变址寄存器存下标并自动加一，地址等于基地址加下标

【定位】 计算机组成原理 · 第5章中央处理器 · 指令的寻址方式

【易混对比】

- "变址寻址"和"直接寻址"是两回事：变址的地址 = 基地址 + 变址寄存器的值，下标存在寄存器里、能自动变；直接寻址的地址直接写在指令里，是死的，访问下一个元素得改指令。
- "变址寻址"和"间接寻址"是两回事：变址解决的是"按顺序遍历"，下标自动往后走；间接解决的是"地址本身不固定"，每次访问要多取一次地址，遍历不需要它。

按下标顺序访问数组，下标是"活"的，只有变址寻址的变址寄存器能跟着自动变。

【坑位】

别选间接寻址——它多一次访存，还解决不了"按顺序走"的问题。

最容易踩的坑是看到"地址要变"就选间接寻址。为什么错：间接寻址的指令里写的是"地址的地址"，每次访问先取一次才能拿到真地址，多一次访存、还慢；它对付的是地址不固定的情况，不是按顺序遍历。正确做法：按下标顺序访问 = 基地址不动、下标一个一个变 → 只有变址寻址（基地址 + 变址寄存器，下标自动加 1）对得上。

【套路】

- 第一步，看访问规律：按下标顺序、一个一个来 → 需要"下标能自动变"的寻址方式。
- 第二步，对号入座：立即寻址数写死、直接寻址地址写死、间接寻址多一次访存还不对路 → 只剩变址寻址。

先看访问规律要"活下标"，再排除三个"死的"或"不对路的"。

【类比】

像查一栋宿舍楼：基地址是楼号（不动），变址寄存器是手里的房号牌（101、102、103……自动往后翻）。变址寻址就是"楼号 + 房号牌"，翻一页牌就找到下一间。直接寻址是每间房的地址都刻在石头上，要换房得重新刻；间接寻址是石头上刻的是"去问门卫"，每次多跑一趟。

【口诀】 基址加变址，下标自动走；遍历数组它最配。

【考频】 「变址寻址」在 2009–2022 年解析中出现 5 次 · 中频

第 16 题 · 两种指令格式下的指令字长至少（正确答案：A）

题干与选项：

某计算机按字节编址，指令字长固定且只有两种指令格式，其中三地址指令 29 条，二地址指令 107 条，每个地址字段为 6 位，则指令字长至少应该是（ ）。

- A. 24 位
- B. 26 位
- C. 28 位
- D. 32 位

入手点： 两种格式都要能装下，字长就得取两种格式里较大的那个；再注意指令是按字节编址的，字长必须是字节的整数倍，不够要往上凑。

题干专有名词从零解释：

- 先想“指令格式”：一条指令里，操作码占几位、地址码占几位、怎么排布；不同的指令用不同的格式。
- 再想“指令字长”：一条指令占多少位；字长越长，能表示的操作码和地址越多。
- 再想“按字节取整”：指令在内存里按字节存放；字长如果不是 8 的倍数，要往上凑到 8 的倍数，不然没法对齐存放。

考查知识点：

- 先想每种格式要几位：操作码要几位看有几种操作，地址码要几位看地址范围；两种格式分别算出所需位数。
- 再想取较大值：两种格式共用一个字长，字长必须同时装得下两种格式，所以取较大的那个；按题中数据，较大的那个是 23 位。
- 再想按字节取整：23 位不是 8 的倍数，往上凑到 24 位；所以指令字长至少是 24 位，就是 A。
- 边界情况：如果两种格式算出来一样大，取那个值再取整；如果较大的那个正好是 8 的倍数，就不用凑。
- 易混对子：“理论最少位数”和“实际字长”不一样——理论是 23 位，实际按字节对齐是 24 位；题目问“至少”，要的是实际能用的 24 位。

正确选项为什么对： 两种格式里较大的是 23 位，按字节取整到 24 位，就是 A，选 A。

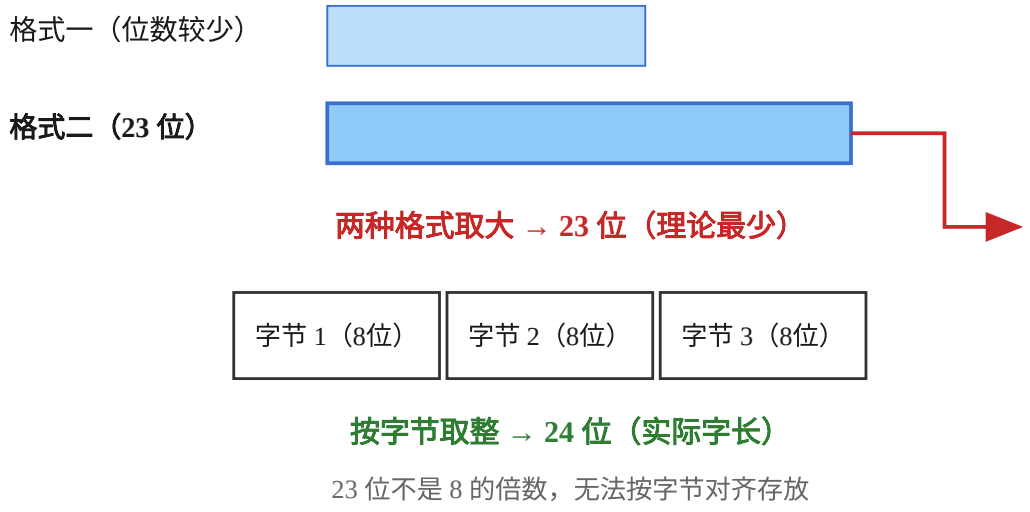
错误选项为什么错＋举例：

- B（23 位）：那是理论最少位数，没按字节取整；23 位没法按字节对齐存放。教训：算完理论值，别忘了按字节凑整。
- C（取了较小格式的位数）：字长要同时装得下两种格式，取小的那个，大的格式就装不下了。教训：两种格式取大不取小。
- D（操作码或地址位数算错了）：操作码位数按操作种类算、地址位数按地址范围算；哪边算错，总位数就错。教训：操作码和地址码分开算，再相加。

笔记本一句话： 第 16 题＋指令字长：两种格式取大值，再按字节取整（23 位→24 位）。

【图像】

指令字长：取大，再按字节凑整



两种指令格式分别算出所需位数，取较大值二十三位，再按字节取整到二十四位

【定位】 计算机组成原理 · 第5章中央处理器 · 指令格式与指令字长

【易混对比】

- "理论最少位数"和"实际指令字长"是两回事：理论是两种格式里较大的那个（23 位）；实际要按字节对齐存放，不够 8 的倍数往上凑，所以是 24 位；题目问"至少"，要的是实际能用的。
- "取大"和"取小"是两回事：两种格式共用一个字长，取小的，大的格式装不下；必须取较大的那个再取整。

两种格式取大不取小，算出理论位数后再按字节取整——23 位理论，24 位实际。

【坑位】

算出 23 位就停笔是最常见的丢分点——别忘了按字节凑整。

最容易踩的坑是选 23 位。为什么错：23 位不是 8 的倍数，指令在内存里按字节存放，没法对齐；"至少"问的是真正能用的字长，必须往上凑到 24 位。另一个坑是取了较小格式的位数——字长要同时装得下两种格式，取小的大的就装不下了。正确做法：操作码位数按操作种类算、地址码位数按地址范围算，两种格式各算总位数，取大的，再按 8 取整。

【套路】

- 第一步，分开算：每种格式，操作码按操作种类算位数，地址码按地址范围算位数，相加得总位数。
- 第二步，取大：两种格式的总位数取较大的那个（23 位）。
- 第三步，凑整：按字节取整，不够 8 的倍数往上凑（23 → 24）。

分开算、取大值、按字节凑整——三步缺一不可。

【类比】

像给两种不同尺寸的画配同一个画框：框必须装得下较大的那幅（取大）；画框还只能按整寸卖，23 寸没有，只能买 24 寸的（按字节取整）。买 23 寸的框不存在，买小一号的框大画装不下。

【口诀】 两格式取大，按字节凑整；二十三理论，二十四实际。

【考频】 「指令字长」在 2009–2022 年解析中出现 8 次 · 中频

第 17 题 · 超标量流水线特性正误（正确答案：C）

题干与选项：

下列关于超标量流水线特性的叙述中，正确的是（）。

Ⅰ．能缩短流水线功能段的处理时间

Ⅱ．能在一个时钟周期内同时发射多条指令

Ⅲ．能结合动态调度技术提高指令执行并行性

- A. 仅Ⅱ
- B. 仅Ⅰ、Ⅲ
- C. 仅Ⅱ、Ⅲ
- D. Ⅰ、Ⅱ和Ⅲ

入手点： 超标量就一个核心特征：“一个时钟周期里发射多条指令”；凡是说到“多发射、多流水线并行”的，就是它；凡是说到“加深流水线、缩短时钟周期”的，那是超流水线，不是超标量。

题干专有名词从零解释：

- 先想“流水线”：把一条指令的执行拆成取指、译码、执行、访存、写回几段，像工厂流水线一样排着干；不同指令的不同段可以同时干。
- 再想“超标量”：一个时钟周期里同时发射多条指令；机器里有多条流水线并行工作；每个周期完成的指令数可以大于 1。
- 再想“超流水线”：把流水线分得更细、段数更多；时钟周期变短，但每个周期还是只发射一条指令。
- 再想“发射”：把指令送进流水线开始执行；发射几条，就是同时开始执行几条。

考查知识点：

- 先想超标量的标志：多发射；一个周期发射 2 条、4 条指令，指令级并行度大于 1；需要多套取指、译码、执行部件。
- 再想超流水线的区别：超流水线是把一段拆成两段、三段，周期变短；但发射还是 1 条；它提高的是时钟频率，不是单周期指令数。
- 再想长指令字（VLIW）的区别：长指令字是编译器把多条指令打包成一条长指令；超标量是硬件在运行时动态决定发射几条；一个靠编译器打包，一个靠硬件调度。
- 再想本题：按“多发射”这个标准去对，符合的说法就是 C。
- 边界情况：超标量和超流水线可以同时用（一台机器既多发射、又深流水）；但题目问的是“超标量”的特性，对上多发射才是正解。
- 易混对子：超标量（多发射）vs 超流水线（深流水）vs 长指令字（编译器打包）——三个名字像，机制完全不同。

正确选项为什么对： C 说的是超标量的多发射特征，和定义对得上，选 C。

错误选项为什么错＋举例：

- A（把超流水线说成超标量）：加深流水线、缩短时钟周期，那是超流水线；超标量说的是单周期发射多条，不是周期变短。教训：先分清“多发射”和“深流水”。

- B（把长指令字说成超标量）：编译器打包长指令是长指令字；超标量是硬件运行时多发射。教训：先分清“编译器打包”和“硬件多发射”。
- D（说超标量每个周期只能发射一条）：那是普通标量流水线；超标量恰恰是能发射多条。教训：超标量的“超”就超在多发射上。

笔记本一句话： 第 17 题＋超标量：单周期多发射；超流水线是深流水、长指令字是编译器打包。

【图像】



锚定“多发射”逐项对照 —— 说多发射的才是超标量

超标量每周期发射多条指令，超流水线段数更多周期更短但每周期仍发射一条，长指令字靠编译器打包

【定位】 计算机组成原理 · 第5章中央处理器 · 指令流水线（超标量）

【易混对比】

- "超标量"和"超流水线"是两回事：超标量是"一个周期发射多条指令"（多发射，单周期指令数大于 1）；超流水线是"把流水线分得更细、周期变短"，但每个周期还是只发射一条。
- "超标量"和"长指令字（VLIW）"是两回事：超标量是硬件在运行时动态决定一个周期发射几条；长指令字是编译器提前把多条指令打包成一条长指令，一个靠硬件调度，一个靠编译器打包。

超标量的"超"就超在多发射上；凡是说到"加深流水线、缩短周期"的是超流水线，说到"编译器打包"的是长指令字。

【坑位】

名字像不代表一回事——"超"字头的三个概念，机制完全不同。

最容易踩的坑是把"加深流水线、缩短时钟周期"判成超标量。为什么错：那是超流水线的特征（段数多、周期短、每周期仍发射一条）；超标量的身份证只有一张，叫"单周期多发射"。正确做法：先锚定"多发射"这个关键词，逐个选项对照——说多发射、多流水线并行的，是超标量；说周期变短、段数变多的，是超流水线；说编译器打包的，是长指令字。

【套路】

- 第一步，锚定关键词：超标量 = 单周期多发射，先把这张身份证记牢。
- 第二步，逐项对照：每个选项问一句"它说的是多发射吗"——是就对，不是就按超流水线/长指令字/普通流水线归类排除。

先锚定"多发射"，再逐项问"是不是多发射"。

【类比】

超标量像超市同时开多个收银台，一波结好几个人（多发射）；超流水线像把结账拆成"扫码、装袋、收钱"更多小步骤，每步更快，但一次还是只结一个人；长指令字像顾客自己提前把东西按"一单"打包好，收银员整单扫。三个都快，但快的办法完全不同。

【口诀】 超标量，多发射；超流水，段加深；长指令，编译包。

【考频】 「超标量」在 2009–2022 年解析中出现 15 次 · 高频

第 18 题 · 主存储器与控制存储器叙述错误项（正确答案：B）

题干与选项：

下列关于主存储器（MM）和控制存储器（CS）的叙述中，错误的是（ ）。

- A. MM 在 CPU 外，CS 在 CPU 内
- B. MM 按地址访问，CS 按内容访问
- C. MM 存储指令和数据，CS 存储微指令
- D. MM 用 RAM 和 ROM 实现，CS 用 ROM 实现

入手点： 本题问“错误的是”，做法是先把主存和控存的“存放内容、读写特性、实现介质”三张名片对清楚，再看哪个选项把名片贴错了。

题干专有名词从零解释：

- 先想“主存储器（主存）”：CPU 直接访问的内存；存放正在运行的程序的指令和数据；可以读也可以写。
- 再想“控制存储器（控存）”：存放微程序的存储器；微程序是控制 CPU 干活的底层命令序列。
- 再想“微程序”：一条机器指令拆成的一串微操作；微程序存在控存里，CPU 执行指令时按微程序一步一步干。
- 再想“只读”：写进去之后一般不改；控存常用只读存储器实现。

考查知识点：

- 先想主存的名片：存放指令和数据；可读可写；容量大；速度比控存慢；用随机存储器实现。
- 再想控存的名片：存放微程序（微指令）；一般只读；容量小；速度快；用只读存储器实现。
- 再想容易贴错的地方：把“按内容访问”贴到控存头上；控存是按微地址访问的，按内容访问的是相联存储器；访问方式贴错，就是错误项。
- 再想本题：B 说“主存按地址访问、控存按内容访问”，错在后半句：控存也是按地址访问（按微地址读微指令），按内容访问的是相联存储器，不是控存；所以 B 是错误项。
- 边界情况：有的机器控存可写（可写控存），但那是特例；按一般概念和本题设定，控存按只读处理。

正确选项为什么对： B 说控存按内容访问，是错的：控存按微地址访问（给出微地址、读出微指令），和主存一样是按地址访问；按内容访问的是相联存储器，不是控存。本题问的就是错误项，选 B。

错误选项为什么错＋举例：

- A（主存 CPU 外、控存 CPU 内）：说的是位置区分，对的；主存是独立的存储器、在 CPU 外，控存是 CPU 内部的控制部件。本题问错误项，不能选它。教训：问“错误的是”，先把对的排除掉。
- C（主存存指令数据、控存存微指令）：说的是存放内容区分，对的；主存放指令和数据，控存放微程序。不能选它。教训：对的叙述再像答案也不能选。
- D（主存用 RAM 和 ROM 实现、控存用 ROM 实现）：说的是实现技术区分，对的；主存可读写（RAM）也可只读（ROM），控存一般用 ROM 实现。不能选它。教训：实现技术对上就是对的。

笔记本一句话： 第 18 题＋主存 vs 控存：主存存指令数据、可读写、容量大；控存存微程序、只读、容量小。

【图像】

主存 vs 控存：三张名片

主存储器（主存）

存放：正在运行的指令和数据

读写：可读可写

容量：大

位置：在 CPU 外

实现：RAM 为主

访问：按地址访问

控制存储器（控存）

存放：微程序（控制 CPU 干活的底层命令序列）

读写：一般只读

容量：小

位置：在 CPU 内

实现：ROM

陷阱：“控存按内容访问”是错的！

控存按微地址访问；按内容访问的是相联存储器，别贴错名片

主存储器存指令和数据可读写容量大，控制存储器存微程序只读容量小，按内容访问的是相联存储器

【定位】 计算机组成原理 · 第3章存储器层次结构 · 主存储器与控制存储器

【易混对比】

- "主存"和"控存"是两回事：主存存正在运行的指令和数据、可读可写、容量大、在 CPU 外；控存存微程序（控制 CPU 干活的底层命令序列）、一般只读、容量小、在 CPU 内；存放内容、读写特性、位置三张名片全不同。
- "按内容访问"和"按地址访问"是两回事：控存是按微地址访问（给微地址、读微指令），和主存一样是按地址；按内容访问的是相联存储器，别把它的名片贴到控存头上。

问"错误的是"，先把主存和控存的三张名片对清楚；看到"控存按内容访问"直接判错——那是相联存储器的名片。

【坑位】

问"错误的是"时，对的叙述再像答案也不能选——先排除对的。

最容易踩的坑是被"主存按地址访问、控存按内容访问"这种半对半错的说法带走，觉得后半句也对。为什么错：控存按微地址读微指令，是按地址访问；按内容访问的是相联存储器（按内容找数据的那种存储器），和控存没关系。正确做法：逐项核对名片——位置（主存 CPU 外、控存 CPU 内）、内容（主存指令数据、控存微指令）、实现（主存 RAM/ROM、控存 ROM），名片贴错的那项就是答案。

【套路】

- 第一步，列名片：主存（指令数据、可读写、容量大、CPU 外）vs 控存（微程序、只读、容量小、CPU 内）。
- 第二步，逐项排除：问"错误的是"，先把名片对得上的对项排除，剩下名片贴错的就是答案。

先列两张名片，再排除对的——贴错名片的就是错项。

【类比】

主存像办公室的资料柜：放正在办的文件（指令和数据），随取随放（可读写），柜子大，放在办公室外面。控存像贴在机器内部的操作规程小卡片：写着机器每一步该怎么动（微程序），印好就不改（只读），卡片小，贴在机器肚子里。有人说"小卡片是按内容找的"——不对，按内容找的是另一本"按特征查的通讯录"（相联存储器）。

【口诀】 主存指令数据能读写，控存微程序只读小。

【考频】 「控制存储器」在 2009–2022 年解析中出现 4 次 · 中频

第 19 题 · 指令流水线数据通路叙述错误项（正确答案：A）

题干与选项：

下列关于指令流水线数据通路的叙述中，错误的是（ ）。

- A. 包含生成控制信号的控制部件
- B. 包含算术逻辑运算部件（ALU）
- C. 包含通用寄存器组和取指部件
- D. 由组合逻辑电路和时序逻辑电路组合而成

入手点： 本题问“错误的是”，做法是先把流水线五段（取指、译码、执行、访存、写回）每段干什么、用到哪个部件列清楚，再看哪个选项把某段的活说错了。

题干专有名词从零解释：

- 先想“数据通路”：指令执行过程中，数据流过的部件和路线；包括程序计数器、寄存器堆、运算器、存储器这些部件。
- 再想“流水线五段”：取指（从指令存储器取指令）、译码（读寄存器、解码）、执行（运算器算）、访存（读写数据存储器）、写回（结果写回寄存器）。
- 再想“程序计数器（PC）”：存着下一条要取的指令的地址；取指阶段用它找指令，取完指向下一条。

考查知识点：

- 先想取指段：按 PC 到指令存储器取指令；PC 加 4（或指向下一条）；不涉及数据存储器、不涉及运算。
- 再想译码段：读寄存器堆、产生控制信号；不访问数据存储器。
- 再想执行段：运算器算地址或算结果；分支判断也在这一段。
- 再想访存段：只有访存指令（读/写内存）才访问数据存储器；普通运算指令这一段空转。
- 再想写回段：结果写回寄存器堆；访存指令写回的是从内存读到的数据。
- 再想本题：A 说数据通路包含生成控制信号的控制部件，错在“控制部件”：生成控制信号的控制部件不属于数据通路，数据通路只含数据流过的部件；所以 A 是错误项。

正确选项为什么对： A 错在把控制部件算进数据通路：数据通路是数据流过的部件（取指部件、寄存器组、ALU、数据存储器等），生成控制信号的控制部件是独立的控制逻辑、不属于数据通路。本题问的就是错误项，选 A。

错误选项为什么错＋举例：

- B（包含 ALU）：对的；ALU 在执行段做运算，是数据通路的核心部件。不能选它。教训：问“错误的是”，对的先排除。
- C（包含通用寄存器组和取指部件）：对的；寄存器组供译码读数、写回写数，取指部件按 PC 取指令，都是数据通路部件。不能选它。教训：每段的部件要认准。
- D（由组合逻辑和时序逻辑组合而成）：对的；组合逻辑做运算和选择，时序逻辑（段间寄存器）保存每段的结果。不能选它。教训：数据通路就是组合逻辑加时序逻辑。

笔记本一句话： 第 19 题＋流水线数据通路：取指、译码、执行、访存、写回，每段的活和部件要分清。

【图像】

流水线数据通路：五段的活



✗ 控制部件（生成控制信号）不属于数据通路

数据通路只收“数据流过”的部件；控制部件只发控制信号，数据没流过它

数据通路 = 组合逻辑（ALU 运算、多路选择）+ 时序逻辑（段间寄存器存每段结果）

访存段只有访存指令才访问数据存储器，普通运算指令该段空转

流水线五段取指译码执行访存写回与各自部件，生成控制信号的控制部件不属于数据通路

【定位】 计算机组成原理 · 第5章中央处理器 · 指令流水线的数据通路

【易混对比】

- "数据通路"和"控制部件"是两回事：数据通路是数据流过的部件（取指部件、寄存器堆、ALU、数据存储器）；生成控制信号的控制部件是独立的控制逻辑，不属于数据通路——这正是本题的错项。
- "访存段"和"执行段"是两回事：访存段只有访存指令（读/写内存）才访问数据存储器，普通运算指令这一段空转；执行段是 ALU 算数、分支判断的地方。

数据通路只收"数据流过的部件"，生成控制信号的控制部件不算——这是"错误的是"类题目的经典坑位。

【坑位】

看到"控制部件"四个字出现在数据通路的选项里，直接警觉。

最容易踩的坑是觉得"控制信号也是流水线的一部分，控制部件当然算数据通路"。为什么错：数据通路的定义卡得很死——数据流过才算；控制部件产生的是控制信号，指挥数据怎么走，自己不走数据，所以不算。正确做法：把五段每段的活和部件列清楚（取指：PC＋指令存储器；译码：寄存器堆；执行：ALU；访存：数据存储器；写回：寄存器堆），哪个选项塞进了"数据没流过"的部件，哪个就是错项。

【套路】

- 第一步，列五段：取指、译码、执行、访存、写回，每段的活和用的部件写清楚。
- 第二步，查"数据流过"：逐项问"数据流过这个部件吗"——控制部件数据没流过，判错；问"错误的是"就选它。

列清五段的活，再问一句"数据流过吗"。

【类比】

数据通路像工厂的传送带：零件（数据）流过传送带、机械臂（ALU）、仓库（存储器）。控制部件像车间的调度员：他拿对讲机指挥哪条传送带开、哪条停，但他自己不站在传送带上被运走。把调度员算成传送带的一部分，就错了。

【口诀】 五段活计要分清，控制部件不算数。

【考频】 「数据通路」在 2009–2022 年解析中出现 19 次 · 高频

第 20 题 · 多总线结构叙述错误项（正确答案：D）

题干与选项：

下列关于多总线结构的叙述中，错误的是（）。

- A. 靠近 CPU 的总线速度较快
- B. 存储器总线可支持突发传送方式
- C. 总线之间须通过桥接器相连
- D. PCI-Express×16 采用并行传输方式

入手点： 本题问“错误的是”，做法是先把多总线的三条线（系统总线、存储总线、I/O 总线）各连谁、各干什么列清楚，再看哪个选项把某条线的对象或作用说错了。

题干专有名词从零解释：

- 先想“总线”：连接各个部件、传数据和控制信号的公共通道；像一条路，数据在上面跑。
- 再想“系统总线”：连接 CPU、主存和 I/O 的总线；是大家都要走的主干道。
- 再想“存储总线”：CPU 和主存之间的专线；只给 CPU 和主存用，不让 I/O 来挤。
- 再想“I/O 总线”：连接 I/O 设备和 I/O 接口的总线；外设的数据走这条线。

考查知识点：

- 先想多总线为什么好：把路分开，CPU 和主存走存储总线、外设走 I/O 总线，两边可以同时传，互不挤；比单总线（大家挤一条路）快。
- 再想每条线的对象：存储总线只连 CPU 和主存；I/O 总线连 I/O 设备；系统总线是总的协调通道。
- 再想容易说错的地方：把存储总线的对象说成 I/O 设备，或者把 I/O 总线的作用说成连 CPU 和主存，或者说多总线比单总线慢；说错的就是错误项。
- 再想本题：D 说 PCI-Express×16 采用并行传输，错在“并行”：PCI-Express 是串行传输（差分串行通道），×16 是 16 条串行通道，不是 16 位并行；所以 D 是错误项。

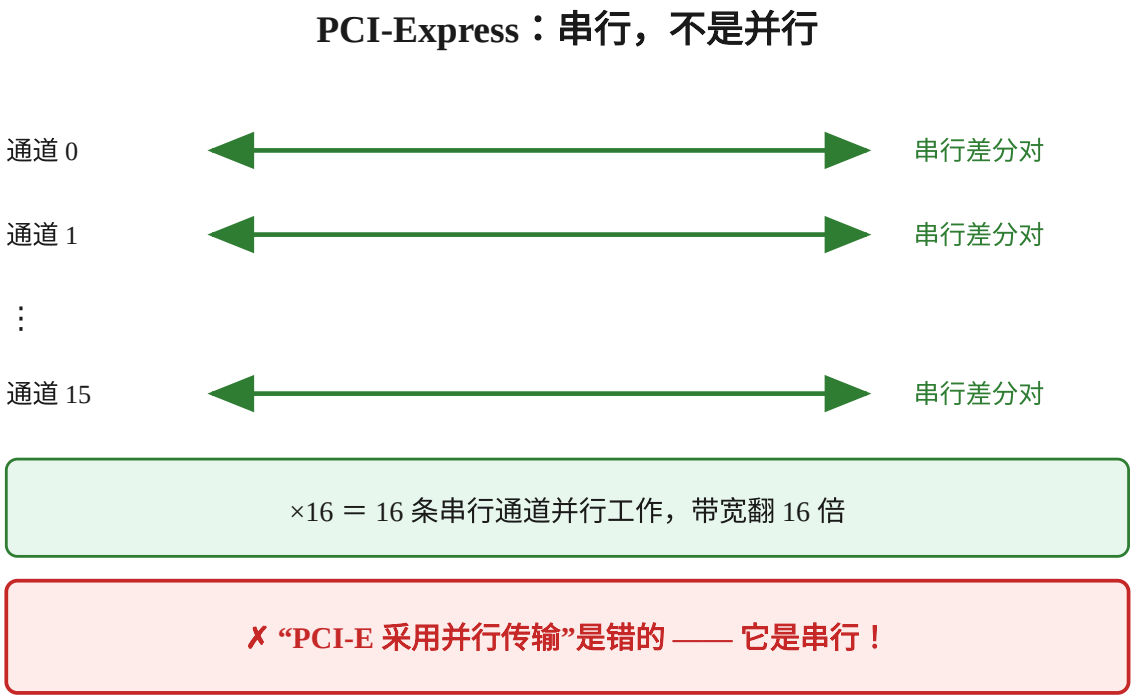
正确选项为什么对： D 错在传输方式：PCI-Express 采用串行传输，×16 表示 16 条串行通道并行工作，不是 16 位并行传输。本题问的就是错误项，选 D。

错误选项为什么错＋举例：

- A（靠近 CPU 的总线速度较快）：对的；CPU 总线、存储器总线离 CPU 近、速度快，I/O 总线离得远、速度慢。不能选它。教训：问“错误的是”，对的先排除。
- B（存储器总线可支持突发传送）：对的；突发传送一次给出地址、连续传多个数据，存储器总线支持这种方式。不能选它。教训：突发传送是存储器总线的典型方式。
- C（总线之间须通过桥接器相连）：对的；不同速度、不同协议的总线之间要经桥接器转换互连。不能选它。教训：桥接器就是不同总线之间的翻译。

笔记本一句话： 第 20 题＋多总线：存储总线连 CPU 和主存，I/O 总线连外设；分开走、能并行。

【图像】



PCI-Express 采用串行传输，乘十六表示十六条串行通道并行工作而非十六位并行传输

【定位】 计算机组成原理 · 第6章总线 · 总线结构与总线标准

【易混对比】

- "串行传输"和"并行传输"是两回事：PCI-Express 是串行（差分串行通道），一次 1 位但速度快、抗干扰；并行是一次多位但线一多干扰大；×16 是 16 条串行通道并行工作，不是 16 位并行。
- "存储总线"和"I/O 总线"是两回事：存储总线是 CPU 和主存的专线，不让外设挤；I/O 总线连外设；分开走、两边能同时传，这是多总线比单总线快的根源。

PCI-Express 的"×16"是 16 条串行通道，不是 16 位并行——看到"PCI-E 并行"直接判错。

【坑位】

"×16"最容易被读成"16 位并行"——它是 16 条串行通道。

最容易踩的坑是想当然：看到 ×16 就以为是 16 根线同时传 16 位（并行）。为什么错：PCI-Express 从设计上就是串行传输，每条通道是差分串行对；×16 表示 16 条这样的串行通道并排工作，带宽翻 16 倍，但传输方式依然是串行。正确做法：记住"PCI-E ＝ 串行"，再核对其他选项——靠近 CPU 的总线速度快、存储总线支持突发传送、总线之间经桥接器相连，这三条都是对的，问"错误的是"就锁定 D。

【套路】

- 第一步，列三条线：存储总线（CPU–主存专线）、I/O 总线（连外设）、系统总线（主干道），各连谁、各干什么。
- 第二步，记一个反常识：PCI-Express 是串行，×16 是 16 条串行通道。
- 第三步，逐项排除：问"错误的是"，对的三项排除，剩下说"并行"的就是答案。

列清三条线，记住 PCI-E 是串行，排除对的锁定错的。

【类比】

PCI-Express ×16 像 16 条高速公路车道：每条车道上的车都是单列纵队走（串行），16 条车道一起走运力大，但不能说"16 辆车并排走"（并行）。老式的并行总线像 16 匹马拉一辆车并排跑，绳子一多就互相绊（干扰大），所以被串行取代了。

【口诀】 PCI-E 走串行，乘十六是十六条通道。

【考频】 「PCI-Express」在 2009–2022 年解析中出现 25 次 · 高频

第 21 题 · I/O 指令数据传送的发生位置（正确答案：D）

题干与选项：

I/O 指令实现的数据传送通常发生在（ ）。

- A. I/O 设备和 I/O 端口之间
- B. 通用寄存器和 I/O 设备之间
- C. I/O 端口和 I/O 端口之间
- D. 通用寄存器和 I/O 端口之间

入手点： I/O 指令是 CPU 直接指挥外设的办法；CPU 手里只有寄存器，数据一定是从 CPU 的寄存器出发、到外设的接口；凡是说到“主存”的，那是 DMA，不是 I/O 指令。

题干专有名词从零解释：

- 先想“I/O 指令”：CPU 发给外设的专用指令（比如 IN、OUT）；CPU 用它直接读写外设的接口寄存器。
- 再想“I/O 接口”：CPU 和外设之间的中转站；外设的数据先到接口，CPU 再从接口取；CPU 的数据先到接口，接口再给外设。
- 再想“DMA”：直接存储器访问；外设和主存之间直接传数据，CPU 不参与搬运；和 I/O 指令是两回事。

考查知识点：

- 先想 I/O 指令的传送路径：CPU 的通用寄存器 ←→ I/O 接口的数据寄存器；CPU 执行 IN 指令，把接口寄存器的数据读到 CPU 寄存器；执行 OUT 指令，把 CPU 寄存器的数据写到接口寄存器。
- 再想为什么不是主存：I/O 指令是程序控制的传送，CPU 全程参与；数据经过 CPU 寄存器，不直接进主存；主存和外设直接传，那是 DMA 的活。
- 再想为什么不是 CPU 和主存之间：CPU 和主存之间传数据用的是访存指令（读/写内存），不是 I/O 指令。
- 边界情况：I/O 指令传送的数据最终可能来自主存（比如程序先从主存读到寄存器、再用 OUT 送出去），但 I/O 指令本身这一步的传送位置是寄存器和接口之间；题目问的是这一步的位置。

正确选项为什么对： I/O 指令的数据传送发生在 CPU 通用寄存器和 I/O 接口之间，就是 D，选 D。

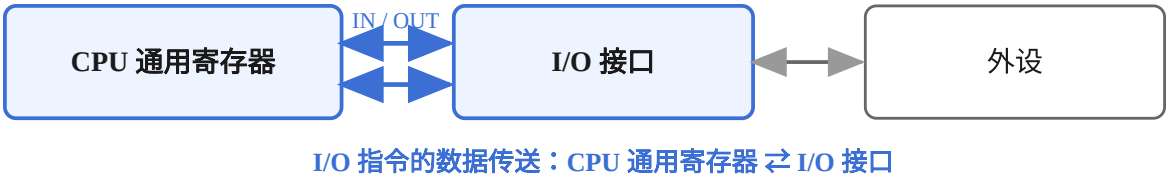
错误选项为什么错＋举例：

- A（主存和外设之间）：那是 DMA 的传送路径；DMA 不经过 CPU 寄存器，直接在主存和外设之间搬。教训：先分清 I/O 指令和 DMA，DMA 才走主存。
- B（CPU 和主存之间）：那是访存指令的路径；I/O 指令不访问主存。教训：I/O 指令的访问对象是接口，不是主存。
- C（外设和外设之间）：外设之间不直接传数据；数据一定经过 CPU（I/O 指令）或主存（DMA）中转。教训：外设之间没有直连的数据通路。

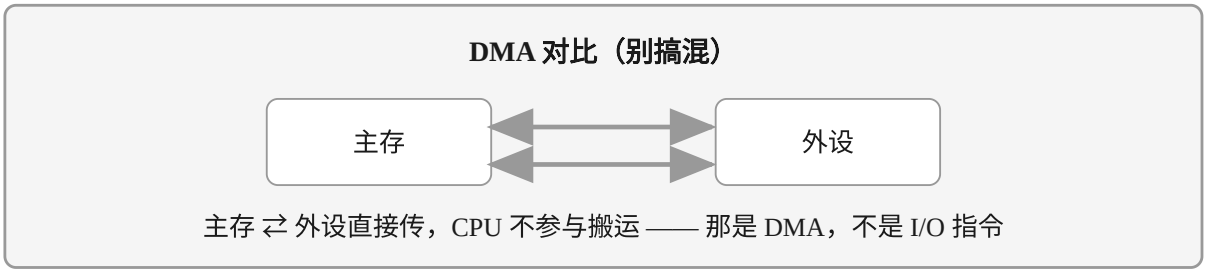
笔记本一句话： 第 21 题＋I/O 指令传送：CPU 寄存器 ←→ I/O 接口；主存和外设直传是 DMA。

【图像】

I/O 指令：谁搬，数据走哪



I/O 指令的数据传送：CPU 通用寄存器 ⇄ I/O 接口



I/O 指令的数据在 CPU 通用寄存器与 I/O 接口之间传送，主存与外设之间直传的是 DMA

【定位】 计算机组成原理 · 第7章输入输出系统 · I/O 方式（程序控制 I/O）

【易混对比】

- "I/O 指令"和"DMA"是两回事：I/O 指令是 CPU 亲自搬（IN/OUT），数据走 CPU 通用寄存器 ←→ I/O 接口；DMA 是外设和主存直传，CPU 不参与搬运。
- "I/O 指令"和"访存指令"是两回事：I/O 指令访问的是外设的接口寄存器；访存指令访问的是主存；访问对象完全不同。

I/O 指令这一步，数据只在 CPU 通用寄存器和 I/O 接口之间走；凡是说到"主存"的，那是 DMA 的活。

【坑位】

看到"主存和外设"就选，是 DMA 的坑——I/O 指令不碰主存。

最容易踩的坑是把 I/O 指令和 DMA 搞混，选了"主存和外设之间"。为什么错：I/O 指令是程序控制的传送，CPU 全程参与，数据一定经过 CPU 的寄存器；主存和外设直接传、CPU 不参与，那是 DMA 的定义。正确做法：先分清"谁搬"——CPU 亲自搬（I/O 指令）还是外设和主存直传（DMA）；再定路径——CPU 亲自搬，起点只能是 CPU 手里的通用寄存器，终点是外设的接口。

【套路】

- 第一步，看谁搬：题目说"用 I/O 指令传送" → CPU 亲自搬，不是 DMA。
- 第二步，定两端：CPU 手里只有寄存器，另一端是外设的接口 → 通用寄存器 ←→ I/O 接口。

先看谁搬，再定两端——CPU 亲自搬，两端就是寄存器和接口。

【类比】

I/O 指令像你亲自去前台取快递：你（CPU）伸手（寄存器）从前台（I/O 接口）接过包裹。DMA 像快递柜：快递员（外设）直接把包裹塞进你家门口的柜子（主存），你全程不用出面。问"亲自取"时答"快递柜"，就答错了。

【口诀】 I/O 指令 CPU 搬，寄存器到接口间；主存直传是 DMA。

【考频】 「I/O 指令」在 2009–2022 年解析中出现 19 次 · 高频

第 22 题 · 多重中断系统叙述错误项（正确答案：B）

题干与选项：

下列关于多重中断系统的叙述中，错误的是（）。

- A. 在一条指令执行结束时响应中断
- B. 中断处理期间 CPU 处于关中断状态
- C. 中断请求的产生与当前指令的执行无关
- D. CPU 通过采样中断请求信号检测中断请求

入手点： 本题问“错误的是”，做法是先把多重中断的规矩列清楚：能不能嵌套、谁能打断谁、要不要保护现场；再看哪个选项把规矩说反了。

题干专有名词从零解释：

- 先想“中断”：外设或程序向 CPU 发的“有事要处理”的请求；CPU 停下手头的事，先去处理中断。
- 再想“多重中断”：处理一个中断的过程中，又来了新的中断；允许新的中断打断正在处理的中断，叫中断嵌套。
- 再想“优先级”：中断有高低；高优先级的中断可以打断低优先级中断的处理，反过来不行。
- 再想“保护现场”：中断处理前，把 CPU 的状态（程序计数器、寄存器）保存起来；处理完恢复，才能接着干原来的事。

考查知识点：

- 先想嵌套的规矩：多重中断允许嵌套；但只有高优先级能打断低优先级，低优先级不能打断高优先级。
- 再想现场保护：每次进入中断处理，都要保护现场（断点和寄存器）；嵌套几次，就保护几次；处理完一层一层恢复。
- 再想开中断和关中断：处理中断时，CPU 要开中断（允许新的高优先级中断进来）才能嵌套；关中断期间，新的中断只能等着。
- 再想本题：B 说中断处理期间 CPU 处于关中断状态，错在“关中断”：多重中断处理期间 CPU 处于开中断状态，才能让高优先级中断嵌套进来；单重中断才是处理期间关中断；所以 B 是错误项。

正确选项为什么对： B 错在中断期间的开关状态：多重中断处理期间 CPU 处于开中断状态（以便高优先级中断嵌套打断），不是关中断；关中断是单重中断的做法。本题问的就是错误项，选 B。

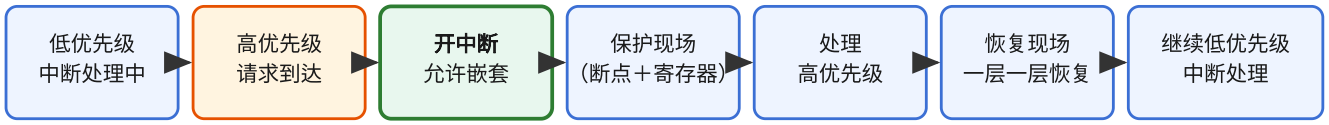
错误选项为什么错＋举例：

- A（一条指令执行结束时响应中断）：对的；中断响应在指令边界，不在指令中间。不能选它。教训：问“错误的是”，对的先排除。
- C（中断请求的产生与当前指令的执行无关）：对的；中断请求来自外设或内部事件，和当前执行哪条指令无关。不能选它。教训：请求什么时候来，和当前指令无关。
- D（CPU 通过采样中断请求信号检测中断请求）：对的；CPU 在指令执行末尾采样中断请求线，有请求就响应。不能选它。教训：采样检测是中断响应的入口。

笔记本一句话： 第 22 题＋多重中断：允许嵌套；高优先级能打断低优先级；每次都要保护现场。

【图像】

多重中断：开中断才能嵌套



多重中断：处理期间开中断，允许嵌套

只许高优先级打断低优先级，反过来不行；嵌套几次，保护现场几次

中断响应在指令边界；CPU 在指令末尾采样中断请求信号

单重中断：处理期间关中断，新请求只能等着

“处理期间关中断”是单重中断的名片，贴到多重中断头上就是错项

中断请求的产生与当前执行哪条指令无关

多重中断处理期间 CPU 开中断，高优先级可打断低优先级并逐层保护现场，单重中断处理期间才关中断

【定位】 计算机组成原理 · 第7章输入输出系统 · 中断（多重中断/中断嵌套）

【易混对比】

- "多重中断"和"单重中断"是两回事：多重允许嵌套，处理期间 CPU 开中断（高优先级能打断低优先级）；单重不允许嵌套，处理期间关中断，新的请求只能等着。
- "谁能打断谁"是单向的：只有高优先级能打断低优先级，低优先级不能打断高优先级；嵌套几次就保护几次现场，处理完一层一层恢复。

多重中断处理期间是开中断，不是关中断——"处理期间关中断"那是单重中断的名片，贴错了就是错项。

【坑位】

问"错误的是"时，"单重中断的做法"经常被贴到"多重中断"头上。

最容易踩的坑是觉得"处理中断时 CPU 肯定关中断，不然不乱套了"。为什么错：多重中断的设计目标就是允许高优先级插队，所以处理期间必须开中断；关中断是单重中断的做法。正确做法：先列多重中断三条规矩——允许嵌套、高打断低（反过来不行）、每次进中断都保护现场；再看选项哪条把规矩说反了（比如"处理期间关中断"），那条就是答案。

【套路】

- 第一步，列规矩：多重中断三条——允许嵌套、只许高打断低、每次保护现场。
- 第二步，找说反的：问"错误的是"，逐项对照三条规矩，说反的那条就是答案（如"处理期间关中断"）。

先列三条规矩，再找说反的那条。

【类比】

多重中断像急诊室：正在处理一个普通病人（低优先级），来了个大出血的（高优先级），医生立刻放下手头的、先抢救重的，抢救完回来继续——这叫嵌套。单重中断像只有一个窗口的银行：办着业务时卷帘门拉下（关中

断），后面的人只能等。把"拉下卷帘门"说成急诊室的做法，就错了。

【口诀】 多重能嵌套，高打低不反；处理开中断，单重才关门。

【考频】 「多重中断」在 2009–2022 年解析中出现 3 次 · 中频

第 23 题 · 作业表、t=2 开始调度，FCFS 与 SJF 各选中的作业（表题）（正确答案：D）

题干与选项：

假设 4 个作业到达系统的时刻和运行时间如下表所示。

| 作业 | 到达时刻 t | 运行时间 |

|------|-----------|---------|

| J1 | 0 | 3 |

| J2 | 1 | 3 |

| J3 | 1 | 2 |

| J4 | 3 | 1 |

系统在 t=2 时开始作业调度。若分别采用先来先服务和短作业优先调度算法，则选中的作业分别是（ ）。

- A. J2、J3
- B. J1、J4
- C. J2、J4
- D. J1、J3

> 注：本题是表题，表格数据如下：J1 到达 0、运行 3；J2 到达 1、运行 3；J3 到达 1、运行 2；J4 到达 3、运行 1；t=2 开始调度。选项 A.J2、J3 B.J1、J4 C.J2、J4 D.J1、J3。

入手点： 关键是“t=2 时谁已经到了”：只有到了的作业才能被选中；J4 是 3 时刻才到，t=2 时还没到，直接排除；剩下的三个里，FCFS 挑最早到的，SJF 挑运行最短的。

题干专有名词从零解释：

- 先想“到达时间”：作业提交到系统的时间；没到时间的作业不在队列里，调度时看不见它。
- 再想“运行时间”：作业需要 CPU 跑多久；短作业优先比的就是这个。
- 再想“先来先服务（FCFS）”：按到达先后选；谁先到就先选谁，不管运行长短。
- 再想“短作业优先（SJF）”：按运行长短选；谁运行最短就先选谁，不管到达先后（但只看已经到了的）。

考查知识点：

- 先想 t=2 时队列里有谁：J1（0 时刻到）、J2（1 时刻到）、J3（1 时刻到）都到了；J4（3 时刻到）还没到，不在队列里。
- 再想 FCFS 选谁：队列里最早到的是 J1（0 时刻），所以 FCFS 选中 J1。
- 再想 SJF 选谁：队列里运行时间最短的是 J3（2，小于 J1 和 J2 的 3），所以 SJF 选中 J3。
- 再想结论：FCFS 是 J1，SJF 是 J3，就是 D。
- 边界情况：J2 和 J3 都是 1 时刻到，FCFS 里如果两个同时到、又要比运行时间，那是另一回事；本题 FCFS 的答案是 J1（0 时刻最早），不存在并列问题。SJF 里如果运行时间并列，一般按到达先后或先来先服务打破并列；本题 J3 的 2 是唯一的最小值，没有并列。

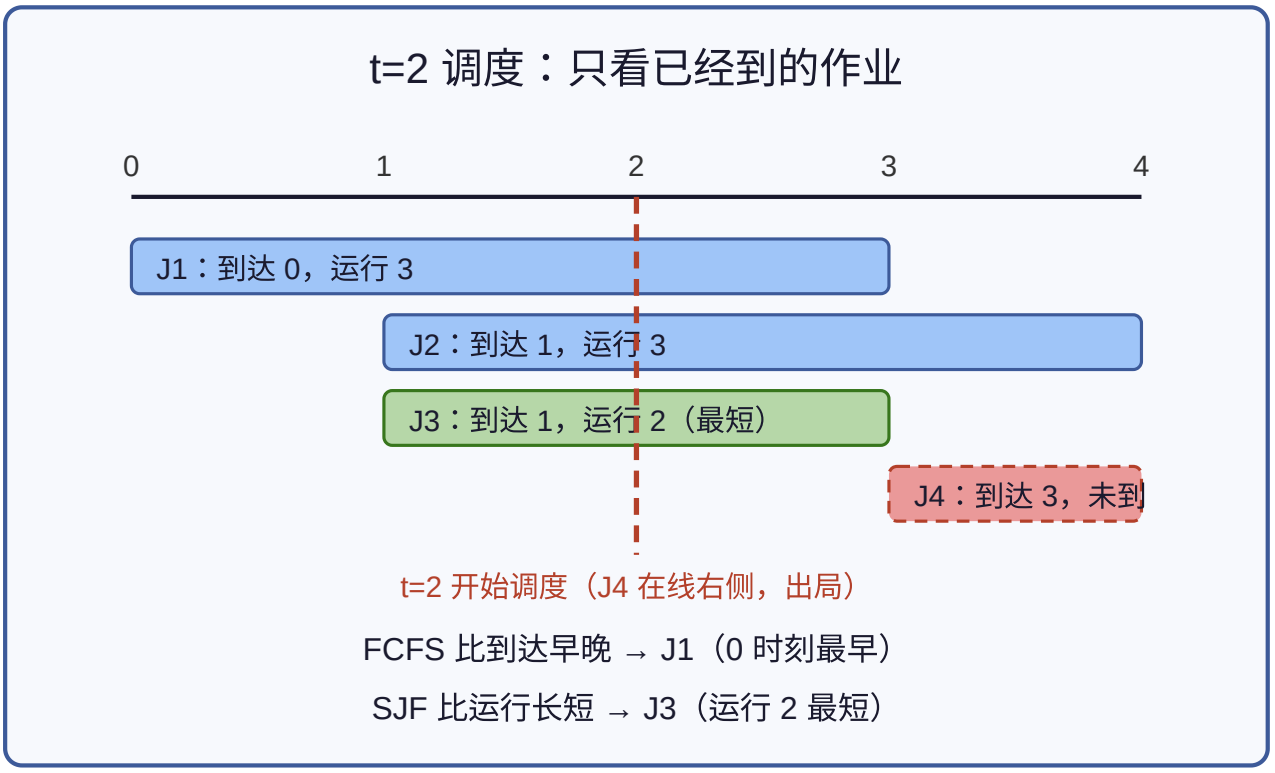
正确选项为什么对： t=2 时队列里是 J1、J2、J3；FCFS 挑最早到的 J1，SJF 挑最短的 J3，就是 D，选 D。

错误选项为什么错＋举例：

- A（J2、J3）：SJF 的 J3 是对的，但 FCFS 的 J2 是错的；FCFS 挑最早到的，J1 是 0 时刻到，比 J2 的 1 时刻早。教训：FCFS 比到达时间，不看运行长短。
- B（J1、J4）：FCFS 的 J1 是对的，但 SJF 的 J4 是错的；J4 是 3 时刻才到，t=2 时还没进队列，根本不在候选里。教训：没到的作业不能选，先看 t=2 时谁在队列里。
- C（J2、J4）：两个都错；FCFS 应该是 J1（最早到），SJF 应该是 J3（最短），J4 还没到。教训：FCFS 和 SJF 的挑法不一样，要分开算。

笔记本一句话： 第 23 题＋t=2 调度：先看谁到了（J4 没到）；FCFS 挑最早到的，SJF 挑运行最短的。

【图像】



时间线：J1 到达0运行3、J2 到达1运行3、J3 到达1运行2、J4 到达3运行1；t=2 处画竖线，J4 在线右侧未到

【定位】 操作系统 · 第2章进程管理 · 处理机调度（FCFS、SJF）

【易混对比】

- "先来先服务"和"短作业优先"是两回事：前者只比到达时刻谁早，后者只比运行时间谁短；一个作业可以早到但运行长，两种算法选中的可以不一样。
- "调度时刻"和"到达时刻"是两回事：调度时刻是系统开始选人的时间点，到达时刻是作业进门的时间点；进门比选人晚的作业，这次轮不到它。
- "在场"和"被选中"是两回事：J2 和 J3 都在场，但 FCFS 只看最早到的 J1；在场只是拿到入场券，被选中还要再按规则比。

调度时刻没到的作业直接出局；FCFS 比到达早晚，SJF 比运行长短，两套规则分开算。

【坑位】

J4 运行时间最短，但 t=2 时它还没到，不能选——"最短"不等于"在场"。

最容易踩的坑是看到 J4 运行时间只有 1，SJF 就直接选 J4。为什么错：SJF 是"在已经到的作业里"挑最短的，J4 是 3 时刻才到，t=2 开始调度时它连队都没排上。正确做法：第一步先圈出 t=2 时在场的作业（J1、J2、J3），把 J4 划掉；第二步再在圈里按规则挑——FCFS 挑到达最早的 J1，SJF 挑运行最短的 J3。

【套路】

- 第一步，圈出场内作业：把到达时刻晚于调度时刻的作业划掉，剩下的才是候选。
- 第二步，FCFS 挑最早到的：候选里比到达时刻，最早的那个中选。
- 第三步，SJF 挑最短的：候选里比运行时间，最短的那个中选；两套规则各算各的。

先圈在场，再分开比：FCFS 比早到，SJF 比短。

【类比】

像早上 8 点开始叫号的窗口：8 点前排进队伍的人才有号，8 点 10 分才到的人，再着急也得等下一轮；轮到叫号时，"先来先服务"叫排得最早的人，"短事优先"叫手里活最少的人——"人到了"和"被叫到"是两码事。

【口诀】 调度先看谁在场，没到的作业先靠边；先来比早到，短作业比长短。

【考频】 「进程调度」在 2009–2022 年解析中出现 15 次 · 高频

第 24 题 · 执行系统调用主要操作的正确顺序（正确答案：C）

题干与选项：

执行系统调用的过程包括如下主要操作：

①返回用户态 ②执行陷入（trap）指令 ③传递系统调用参数 ④执行相应的服务程序

正确的执行顺序是（ ）。

- A. ②→③→①→④
- B. ②→④→③→①
- C. ③→②→④→①
- D. ③→④→②→①

入手点： 系统调用就四步：用户程序先准备好参数，再发“陷入”信号进核心态，核心态的服务程序干活，干完回到用户态；顺序不能乱——参数没准备好，服务程序就不知道要干什么。

题干专有名词从零解释：

- 先想“系统调用”：用户程序请操作系统帮忙干的事（比如读文件、申请内存）；用户程序自己没权限干，得请操作系统。
- 再想“陷入指令”：用户程序发给 CPU 的“切换到核心态”的信号；执行它，CPU 就从用户态转到核心态。
- 再想“用户态和核心态”：用户态是普通权限，核心态是最高权限；操作系统在核心态干活，用户程序在用户态干活。
- 再想“服务程序”：操作系统里专门处理某类系统调用的代码；比如读文件的系统调用，就有对应的读文件服务程序。

考查知识点：

- 先想第一步：用户程序准备参数（比如系统调用号、文件名、缓冲区地址），放在约定的寄存器或栈里。
- 再想第二步：执行陷入指令，CPU 从用户态转到核心态，跳到系统调用处理入口。
- 再想第三步：核心态的服务程序按参数干活（比如真的去读文件）。
- 再想第四步：服务程序干完，返回用户态，用户程序接着往下跑。
- 再想结论：准备参数→陷入→服务→返回，就是 C。
- 边界情况：陷入指令本身不干活，它只负责“切换状态和跳转”；真正的活是服务程序干的；返回时也要恢复用户态的现场。

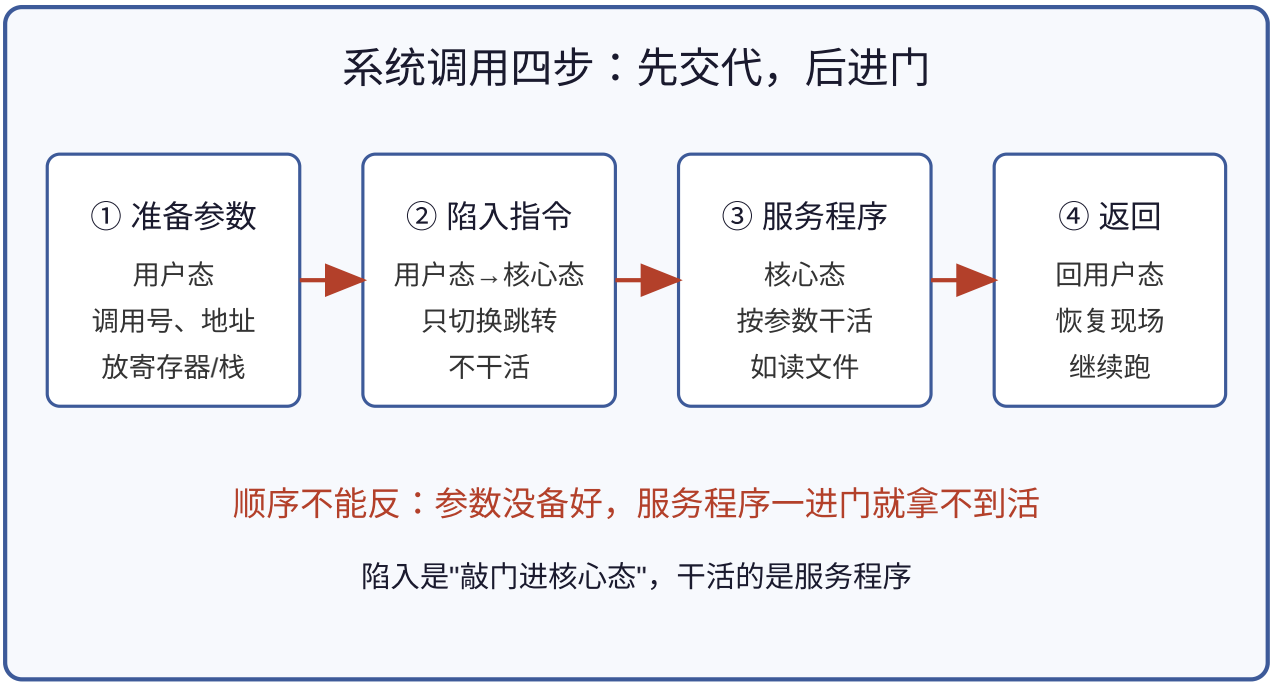
正确选项为什么对： 准备参数、陷入、服务、返回，这个顺序和系统调用的流程对得上，就是 C，选 C。

错误选项为什么错＋举例：

- A（先服务、后陷入）：服务程序在核心态，陷入是进核心态的门；门还没进，服务程序跑不起来。教训：先陷入进核心态，再服务。
- B（先陷入、后准备参数）：参数是服务程序要用的；陷入之后才准备，服务程序一开始拿不到参数。教训：参数要在陷入之前准备好。
- D（服务完了才返回用户态、但把返回放在服务之前）：返回是最后一步；服务没干完就返回，用户程序拿不到结果。教训：服务干完，才能返回。

笔记本一句话： 第 24 题＋系统调用顺序：准备参数→陷入进核心态→服务程序干活→返回用户态。

【图像】



四步流程：用户程序准备参数，执行陷入指令进核心态，服务程序干活，返回用户态

【定位】 操作系统 · 第1章计算机系统概述 · 系统调用

【易混对比】

- "陷入指令"和"服务程序"是两回事：陷入指令只负责"敲门进核心态"，不干活；服务程序负责"真正干活"，在核心态里跑。
- "用户态"和"核心态"是两回事：用户态是普通权限，用户程序待的地方；核心态是最高权限，操作系统干活的地方；陷入指令就是从前者跳到后者的开关。
- "准备参数"和"执行陷入"是两回事：准备参数是用户程序在用户态先把活交代清楚，执行陷入是交代完之后才敲门；顺序反了，服务程序一进门就拿不到活。

参数先备好，陷入是进门，服务是干活，返回是收尾——顺序是：准备参数→陷入→服务→返回。

【坑位】

别把"陷入"当"干活"——陷入只是切换状态和跳转，真正的活是服务程序干的。

最容易踩的坑是把顺序排成"先陷入、后准备参数"。为什么错：服务程序一进核心态就要用参数，参数还没准备好，它就不知道要干什么、给谁干。正确做法：记住"先交代、后进门"——用户程序在用户态把系统调用号、文件名、缓冲区地址这些参数放进约定的寄存器或栈里，然后才执行陷入指令进核心态，服务程序按参数干活，干完返回用户态。

【套路】

- 第一步，用户程序准备参数：把系统调用号和要用的参数放进约定的寄存器或栈里。
- 第二步，执行陷入指令：CPU 从用户态转到核心态，跳到系统调用处理入口。
- 第三步，服务程序干活：核心态的服务程序按参数执行，比如真的去读文件。
- 第四步，返回用户态：活干完，恢复现场，回到用户态继续跑用户程序。

先交代后进门，进门干活再回家：参数→陷入→服务→返回。

【类比】

像去银行办业务：先在家里把身份证和要办的事准备好（准备参数），再到银行取号进门（陷入指令），柜员按你的材料办理（服务程序），办完拿着结果回家（返回用户态）。没带材料就进门，柜员也办不了。

【口诀】 参数先备好，陷入进核心；服务干完活，返回用户态。

【考频】 「系统调用」在 2009–2022 年解析中出现 19 次 · 高频

第 25 题 · 最佳适应算法下回收分区后的空闲分区数与链首（表题）（正确答案：B）

题干与选项：

某计算机按字节编址，其动态分区内存管理采用最佳适应算法，每次分配和回收内存后都对空闲分区链重新排序。当前空闲分区信息如下表所示。

分区起始地址	20K	500K	1000K	200K
分区大小	40KB	80KB	100KB	200KB

回收起始地址为 60K、大小为 140KB 的分区后，系统中空闲分区的数量、空闲分区链第一个分区的起始地址和大小分别是（ ）。

- A. 3、20K、380KB
- B. 3、500K、80KB
- C. 4、20K、180KB
- D. 4、500K、80KB

> 注：本题是表题，空闲分区（起始，大小）：20K/40KB、500K/80KB、1000K/100KB、200K/200KB；回收 60K/140KB。选项 A.3、20K、380KB B.3、500K、80KB C.4、20K、180KB D.4、500K、80KB。

入手点： 回收就两步：第一步看回收的分区左右有没有邻居，有就合并；第二步按最佳适应的规矩重排——最佳适应是按大小从小到大排，链首就是最小的那个。

题干专有名词从零解释：

- 先想“空闲分区”：内存里没被占用的连续区域；用“起始地址、大小”两个数表示。
- 再想“回收”：一个程序用完内存，把它的分区还给系统；还回去的分区要和相邻的空闲分区合并（如果相邻的话）。
- 再想“最佳适应算法”：分配内存时，找“刚好够用、最小的那个”空闲分区；为了找得快，空闲分区按大小从小到大排成链。
- 再想“链首”：链的第一个分区；最佳适应算法的链首就是最小的空闲分区。

考查知识点：

- 先想回收分区的位置：回收的是 60K/140KB（从 60K 开始、140KB 大小）。
- 再想左边邻居：20K/40KB 从 20K 开始、40KB 大小，到 20K+40K=60K 为止；正好连着回收分区的开头 60K，所以左边可以合并。
- 再想右边邻居：回收分区从 60K 开始、140KB 大小，到 60K+140K=200K 为止；200K/200KB 正好从 200K 开始，所以右边也可以合并。
- 再想合并结果：左边 40KB + 回收 140KB + 右边 200KB = 380KB，从 20K 开始；合并后是 20K/380KB。
- 再想剩下几个：原来 4 个，合并了 3 个成 1 个，剩下 20K/380KB、500K/80KB、1000K/100KB，一共 3 个。
- 再想最佳适应的链首：按大小从小到大排，80KB（500K/80KB）最小、100KB 次之、380KB 最大；链首是 500K/80KB。
- 再想结论：3 个分区、链首 500K、80KB，就是 B。

- 边界情况：回收分区只和“地址相邻”的空闲分区合并；地址不挨着（中间隔着被占用的分区）不能合并。合并后要重新按大小排，链首会变。

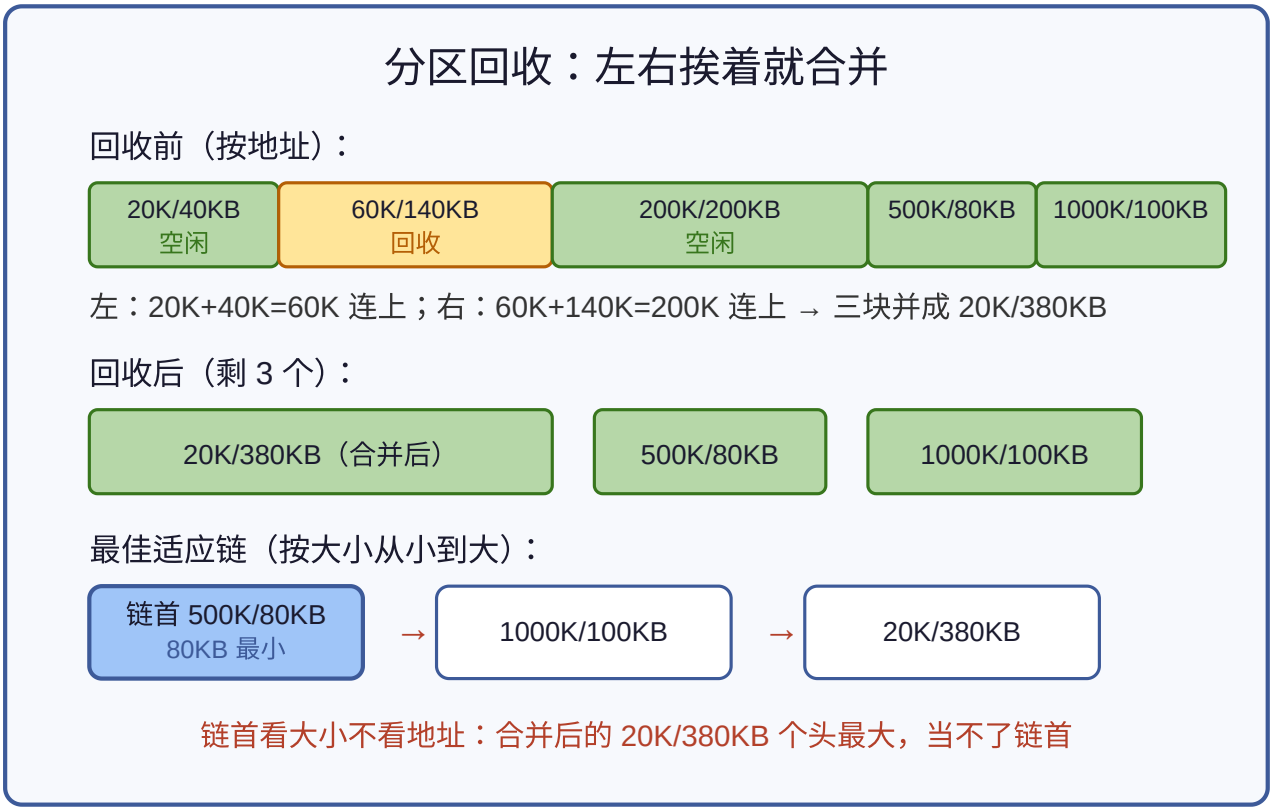
正确选项为什么对： 左右都合并成 20K/380KB，剩 3 个分区；按大小排，链首是 500K、80KB，就是 B，选 B。

错误选项为什么错＋举例：

- A（3、20K、380KB）：分区数 3 是对的，但链首错了；最佳适应按大小排，链首是最小的 80KB，不是合并后的 380KB。教训：最佳适应的链首看大小，不看地址。
- C（4、20K、180KB）：分区数 4 是错的；它只合并了左边（20K/40KB + 60K/140KB = 20K/180KB），漏了右边 200K/200KB 的合并。教训：回收要左右两边都看，挨着的都要合并。
- D（4、500K、80KB）：链首 500K、80KB 是对的，但分区数 4 是错的；同样是漏了右边的合并。教训：分区数和链首要同时对，错一个就不行。

笔记本一句话： 第 25 题＋回收分区：左右挨着的都合并；最佳适应按大小排，链首是最小的。

【图像】



内存条示意：回收前 20K/40KB 空闲、60K/140KB 待回收、200K/200KB 空闲；回收后三块合并成 20K/380KB；最佳适应链按大小排序，链首是 500K/80KB

【定位】 操作系统 · 第3章内存管理 · 动态分区分配（最佳适应、分区回收）

【易混对比】

- "地址相邻"和"大小相近"是两回事：回收合并只看地址挨不挨着，不看大小像不像；地址连上了才能并成一块。
- "最佳适应"和"首次适应"是两回事：最佳适应按大小从小到大排链，找"刚好够用的最小块"；首次适应按地址从小到大排链，找"第一个够用的块"。

- "链首"是两回事：最佳适应链的链首是大小最小的分区，不是地址最小的分区；合并后的 20K/380KB 地址最小，但个头最大，当不了链首。

回收先看左右地址挨不挨着，挨着就并；最佳适应链按大小从小到大排，链首是最小的那个。

【坑位】

回收要左右两边都看——只合并一边，会把分区数算成 4 个。

最容易踩的坑是只合并左边：20K/40KB 从 20K 开始、40KB 长，到 60K 为止，正好连着回收分区 60K/140KB 的开头，于是并成 20K/180KB 就停了。为什么错：回收分区从 60K 开始、140KB 长，到 200K 为止；200K/200KB 正好从 200K 开始，右边也挨着，必须一起并。正确做法：左右都验一遍地址是否相连——左边 20K+40K=60K 连上，右边 60K+140K=200K 连上，三块并成 20K/380KB；剩下 20K/380KB、500K/80KB、1000K/100KB 共 3 个；按大小排 80KB<100KB<380KB，链首是 500K/80KB。

【套路】

- 第一步，验左邻居：左邻居的"起始+大小"等于回收分区起始，就合并。
- 第二步，验右邻居：回收分区的"起始+大小"等于右邻居起始，就合并。
- 第三步，数分区：合并后还剩几个空闲分区就数几个。
- 第四步，按大小定链首：最佳适应链按大小从小到大排，排第一的就是链首。

左右都验地址，并完数个数，再按大小排链首。

【类比】

像停车场里三个连着的空车位被同一辆车退出来：左右挨着的空位可以打通成一个大车位；打通之后，停车场按"车位从小到大"挂牌子找车位，最小的车位永远挂在第一个——牌子按大小排，不按位置排。

【口诀】 回收看左右，挨着就合并；最佳按大小，链首是最小。

【考频】 「最佳适应算法」在 2009–2022 年解析中出现 2 次 · 低频

第 26 题 · 文件大小与簇大小下的分配磁盘空间（正确答案：D）

题干与选项：

某文件系统的簇和磁盘扇区大小分别为 1KB 和 512B。若一个文件的大小为 1026B，则系统分配给该文件的磁盘空间大小是（）。

- A. 1026B
- B. 1536B
- C. 1538B
- D. 2048B

入手点： 磁盘分配不是按字节给的，是按“簇”给的；文件再小也要占满一整个簇，最后一簇用不完也算一簇；所以先算要几簇，再乘以簇大小。

题干专有名词从零解释：

- 先想“簇”：磁盘分配的最小单位；一块磁盘分成很多簇，每簇由若干扇区组成；文件按簇分配，不按字节分配。
- 再想“分配空间”：文件实际占的磁盘空间；等于簇数乘以簇大小。
- 再想“向上取整”：有零头也要算一整个；比如 2.3 簇要算 3 簇。

考查知识点：

- 先想簇数：文件大小除以簇大小，向上取整；比如文件 10KB、簇 4KB，10÷4=2.5，向上取整是 3 簇。
- 再想分配空间：簇数乘以簇大小；上例就是 3×4KB=12KB。
- 再想为什么比文件大：最后一簇没用完的部分也算进去了；这部分叫内部碎片。
- 再想本题：按题中给的文件大小和簇大小算，簇数向上取整再乘以簇大小，得 D。
- 边界情况：文件大小正好是簇大小的整数倍时，簇数就是商，不用多加一簇；空文件占 0 簇（按本题设定，不占簇）。

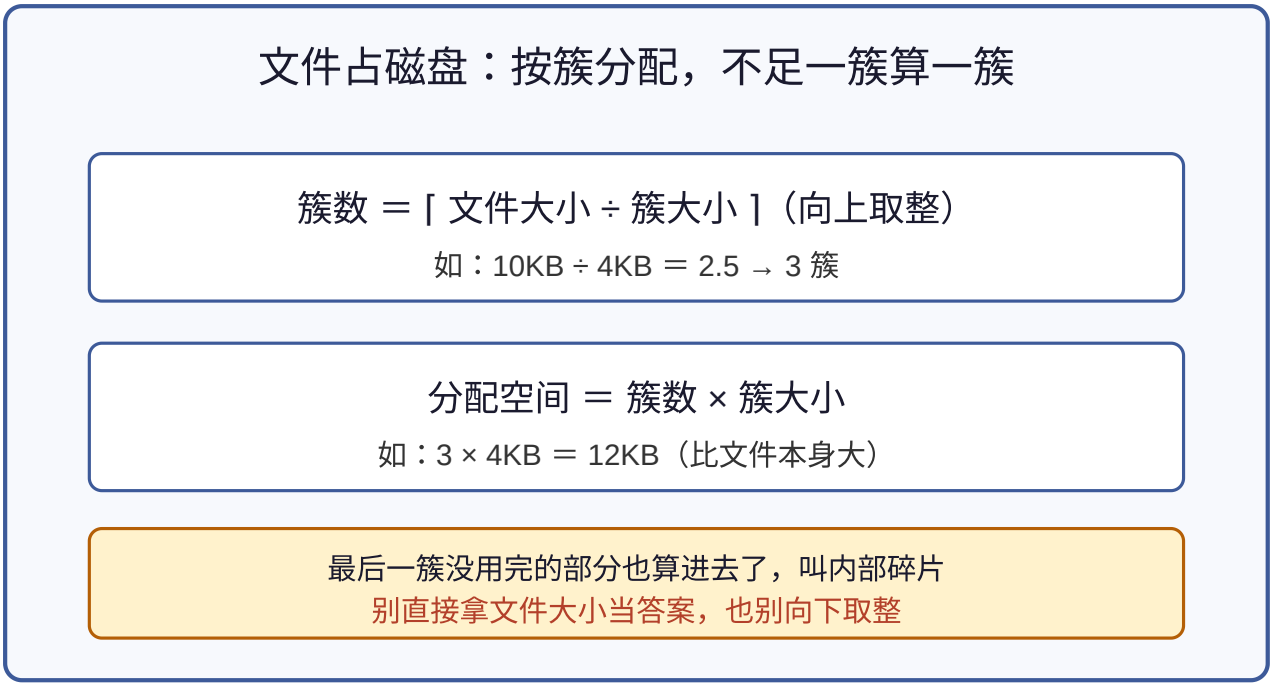
正确选项为什么对： 簇数向上取整再乘以簇大小，就是 D，选 D。

错误选项为什么错＋举例：

- A（直接用文件大小）：那是文件本身的大小，不是分配空间；分配按簇算，最后一簇的零头也要算。教训：分配空间看簇数，不看文件字节数。
- B（簇数向下取整了）：向下取整会少算一簇，最后一簇的数据就没地方放。教训：有零头就要向上取整。
- C（簇数算对了、但乘错了簇大小）：簇数对、簇大小错，总空间也错；两个数都要用题中给的。教训：簇数和簇大小两个数都要对上。

笔记本一句话： 第 26 题＋文件占磁盘：簇数=文件大小÷簇大小向上取整；分配空间=簇数×簇大小。

【图像】



公式卡：簇数等于文件大小除以簇大小向上取整，分配空间等于簇数乘以簇大小；最后一簇没用完也算一整簇

【定位】 操作系统 · 第4章文件管理 · 文件的物理结构（簇）

【易混对比】

- "文件大小"和"分配空间"是两回事：文件大小是文件本身有多少字节，分配空间是磁盘实际划给它多少地方；后者按簇算，一般比前者大。
- "向上取整"和"向下取整"是两回事：有零头必须向上进一簇，向下取整会少算一簇，最后一簇的数据就没地方放。
- "簇数"和"簇大小"是两回事：簇数是"占了几块地"，簇大小是"每块地多大"；分配空间是两者相乘，两个数都要用题里给的。

磁盘按簇分配，不足一簇也算一整簇：簇数向上取整，再乘以簇大小。

【坑位】

别直接拿文件大小当答案——分配空间看的是"占了几簇"，不是文件本身几个字节。

最容易踩的坑是算出文件 10KB、簇 4KB，就答 10KB。为什么错：磁盘分配的最小单位是簇（文件按簇分配，不按字节分配），$\frac{10}{4}=2.5$ 簇，有零头就要进成 3 簇；分配空间是 $3\times 4KB=12KB$。正确做法：先算簇数 $=\lceil\frac{\text{文件大小}}{\text{簇大小}}\rceil$（向上取整），再算分配空间 $=$ 簇数 $\times$ 簇大小；多出来的那点没用完的簇内空间叫内部碎片，照样算在分配空间里。

【套路】

- 第一步，除：文件大小除以簇大小。
- 第二步，向上取整：有零头就进一簇，正好整除就不用加。
- 第三步，乘：簇数乘以簇大小，就是分配空间。

先除再进一，最后乘簇大小。

【类比】

像买鸡蛋按盒卖：一盒 4 个，你要 10 个，就得买 3 盒（2.5 盒没人卖你）；你带走的是 10 个鸡蛋，但占的是 3 盒的"分配空间"，多出来的 2 个位置就是内部碎片。

【口诀】 文件多大不算数，按簇分配才作数；有零头就进一，再乘簇大小。

【考频】 「簇」在 2009–2022 年解析中出现 5 次 · 中频

第 27 题 · 时间片进程调度叙述错误项（正确答案：B）

题干与选项：

下列有关基于时间片的进程调度的叙述中，错误的是（）。

- A. 时间片越短，进程切换的次数越多，系统开销也越大
- B. 当前进程的时间片用完后，该进程状态由执行态变为阻塞态
- C. 时钟中断发生后，系统会修改当前进程在时间片内的剩余时间
- D. 影响时间片大小的主要因素包括响应时间、系统开销和进程数量等

入手点： 本题问“错误的是”，做法是先把时间片轮转的规矩列清楚：人人轮流、时间片大小的影响、切换开销；再看哪个选项把规矩说错了。

题干专有名词从零解释：

- 先想“时间片”：每个进程一次能连续用 CPU 的最长时间；用完就换下一个，不管活干没干完。
- 再想“时间片轮转”：进程排成队，每人按时间片轮流上 CPU；一个人的时间片用完，就排到队尾等下一轮。
- 再想“切换开销”：换进程时保存旧现场、恢复新现场花的时间；这段时间 CPU 不干正事。

考查知识点：

- 先想公平性：时间片轮转让每个进程都有机会上 CPU，不会有人长期饿着；适合分时系统。
- 再想时间片大小的影响：时间片太小，切换太频繁，开销占大头；时间片太大，轮转退化成先来先服务，交互响应变差。
- 再想切换开销：每次换进程都有开销；时间片越小，单位时间里切换次数越多，开销占比越大。
- 再想本题：B 说时间片用完后进程由执行态变为阻塞态，错在“阻塞态”：时间片用完是轮转到期，进程变为就绪态（重新排队等下一轮）；阻塞态是进程等待某事件（如 I/O 完成）才进的；所以 B 是错误项。

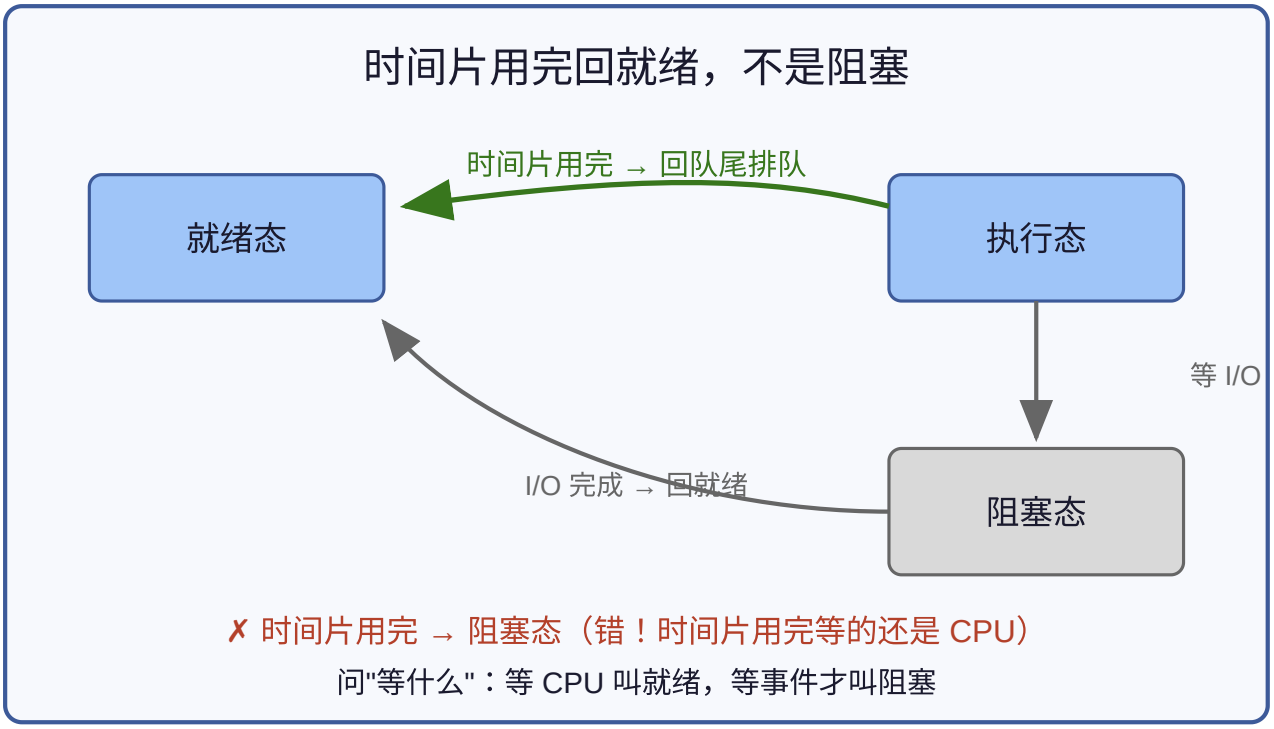
正确选项为什么对： B 错在状态名：时间片用完后进程由执行态变为就绪态（重新排队），不是阻塞态；阻塞态是等事件（如 I/O）才进的。本题问的就是错误项，选 B。

错误选项为什么错＋举例：

- A（时间片越短切换越多、开销越大）：对的；时间片越小，单位时间里切换次数越多，开销占比越大。不能选它。教训：问“错误的是”，对的先排除。
- C（时钟中断后修改剩余时间）：对的；时钟中断处理会扣减当前进程在时间片内的剩余时间。不能选它。教训：剩余时间是时钟中断维护的。
- D（影响因素含响应时间、系统开销、进程数量）：对的；时间片大小要兼顾响应、开销和进程数。不能选它。教训：时间片大小是多因素折中。

笔记本一句话： 第 27 题＋时间片轮转：人人轮流、公平；时间片太小开销大、太大退化成先来先服务。

【图像】



进程状态转换：执行态时间片用完回到就绪态排队尾，执行态等 I/O 才进阻塞态，I/O 完成回就绪态；选项 B 的错误说法画叉

【定位】 操作系统 · 第2章进程管理 · 进程调度（时间片轮转）

【易混对比】

- "就绪态"和"阻塞态"是两回事：就绪态是"万事俱备、只等 CPU"，时间片用完就回这里排队；阻塞态是"在等某件事（如 I/O 完成）"，事件没到、给了 CPU 也跑不了。
- "时间片太小"和"时间片太大"是两回事：太小则切换频繁、开销占大头；太大则轮转退化成先来先服务，交互响应变差；大小是折中出来的。
- "问错误的是"和"问正确的是"是两回事：本题要挑的是说法错的那一项，对的选项要先排除，不能顺手选个对的。

时间片用完是"轮到下一位"，进程回就绪态排队尾；只有等事件才进阻塞态。

【坑位】

题干问"错误的是"——先把三个对的划掉，剩下的才是答案，别顺手选了个正确的。

最容易踩的坑是把"时间片用完→阻塞态"当成对的。为什么错：阻塞态是进程主动等某个事件（比如等 I/O 完成）才进的；时间片用完只是"这一轮的时间到了"，进程什么都不缺，直接回就绪队列尾等下一轮。正确做法：看到状态转换先问"它在等什么"——等 CPU 叫就绪，等事件才叫阻塞；时间片用完等的还是 CPU，所以是就绪态，B 的说法错，选 B。

【类比】

像课堂轮流发言：每人 2 分钟，时间到了就坐下等下一轮（就绪态）；只有被老师叫去拿粉笔、暂时离席的，才是"有事等着"（阻塞态）。时间到不等于有事等，坐下和离席是两码事。

【口诀】 时间片用完回就绪，等事件才叫阻塞；问错先排对，别把对的当错的。

【考频】 「时间片轮转」在 2009–2022 年解析中出现 11 次 · 高频

第 28 题 · 多道程序系统相对单道程序的优点（正确答案：D）

题干与选项：

与单道程序系统相比，多道程序系统的优点是（）。

Ⅰ．CPU 利用率高

Ⅱ．系统开销小

Ⅲ．系统吞吐量大

Ⅳ．I/O 设备利用率高

- A. 仅Ⅰ、Ⅲ
- B. 仅Ⅰ、Ⅳ
- C. 仅Ⅱ、Ⅲ
- D. 仅Ⅰ、Ⅲ、Ⅳ

入手点： 单道的问题是“一个人干活、一个人等”：程序等 I/O 时 CPU 空着；多道就是“几个人轮流干”：一个等 I/O 时，另一个用 CPU；优点一定落在“CPU 不空着、单位时间干的活多”上。

题干专有名词从零解释：

- 先想“单道程序系统”：内存里一次只放一个程序；这个程序等 I/O 时，CPU 就空闲。
- 再想“多道程序系统”：内存里同时放多个程序；一个程序等 I/O 时，CPU 切换去跑另一个程序。
- 再想“CPU 利用率”：CPU 真正干活的时间占总时间的比例；空闲越少，利用率越高。
- 再想“系统吞吐量”：单位时间里系统完成的作业数。

考查知识点：

- 先想 CPU 利用率：多道让 CPU 在某个程序等 I/O 时去跑别的程序，空闲少了，利用率上去了。
- 再想系统吞吐量：CPU 不空着，单位时间完成的作业就多，吞吐量上去了。
- 再想资源利用率：内存和 I/O 设备也一样——多个程序分着用，设备空闲也少了。
- 再想不是优点的地方：单个作业的周转时间不一定变短（多个程序抢 CPU，单个作业可能等得更久）；内存需求也不是变少（多道要同时放多个程序，内存需求更大）。
- 再想结论：优点是 CPU 利用率、系统吞吐量、资源利用率提高，就是 D。

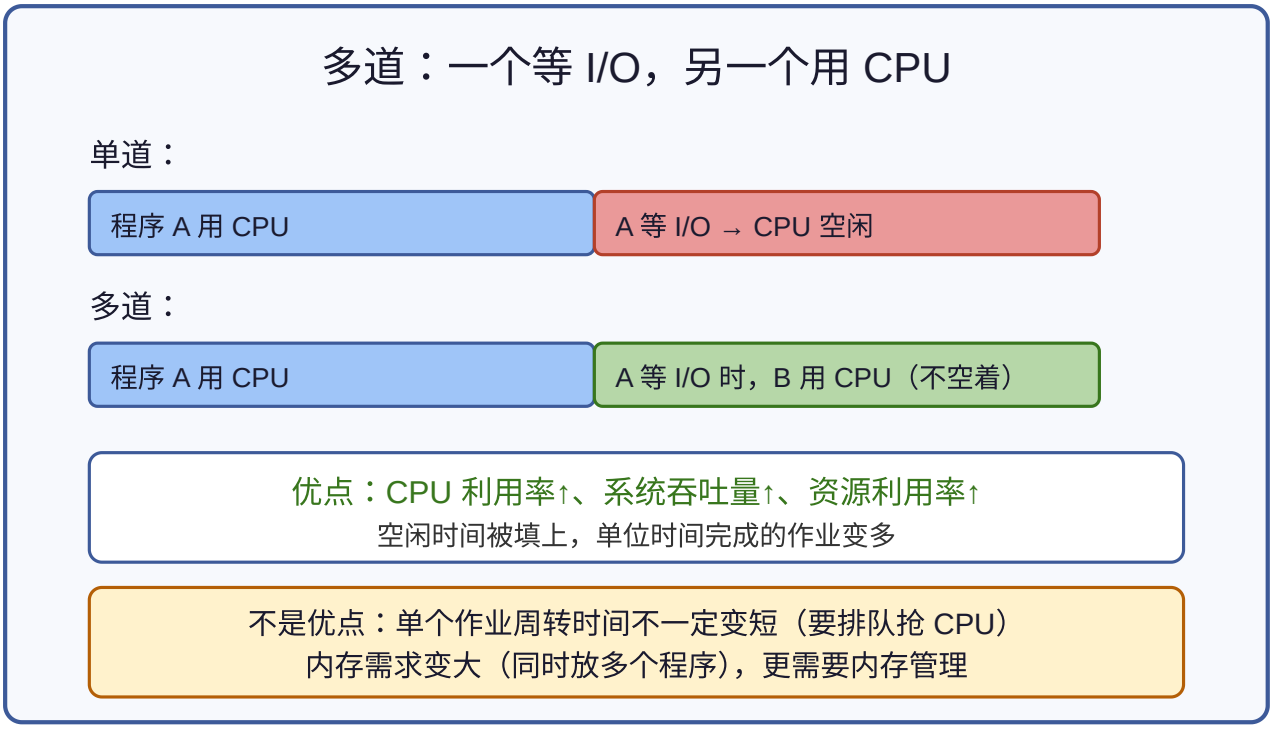
正确选项为什么对： 多道让 CPU 不空着，利用率和吞吐量都上去了，就是 D，选 D。

错误选项为什么错＋举例：

- A（单个作业周转时间变短）：多道下单个作业要和别人抢 CPU，周转时间可能变长；变短的不是单个作业，是整体效率。教训：整体效率和单个作业时间是两回事。
- B（内存需求变少）：多道要同时放多个程序在内存里，内存需求是变大，不是变少。教训：多道的前提就是内存够放多个程序。
- C（不需要内存管理了）：多道下多个程序共用内存，更需要内存管理（分区、保护、地址转换）；不是不需要，是更需要。教训：程序一多，管理只会更复杂。

笔记本一句话： 第 28 题＋多道优点：CPU 不空着，利用率和吞吐量上去；单个作业时间不一定变短。

【图像】



对比条：单道程序等 I/O 时 CPU 空闲；多道程序一个等 I/O 时另一个用 CPU，CPU 利用率和吞吐量上升

【定位】 操作系统 · 第1章计算机系统概述 · 多道程序设计

【易混对比】

- "系统吞吐量"和"单个作业周转时间"是两回事：多道让 CPU 不空着，单位时间完成的作业变多（吞吐量上升）；但单个作业要和别人抢 CPU，它的周转时间可能变长。
- "资源利用率提高"和"内存需求减少"是两回事：多道把内存、CPU、I/O 设备都用得更满（利用率提高）；但内存里要同时放多个程序，内存需求是变大。
- "需要内存管理"和"不需要内存管理"是两回事：程序一多，共用内存反而更需要分区、保护、地址转换这些管理手段，只会更复杂。

多道的好处是"大家轮着用、谁也不闲着"：CPU 利用率、系统吞吐量、资源利用率都上去；单个作业不一定更快，内存反而要得更多。

【坑位】

别选"单个作业周转时间变短"——多道快的是整体，不是某一个作业。

最容易踩的坑是觉得"系统变快了，每个作业肯定也变快"。为什么错：单道时一个作业独占 CPU，多道时它要和别的作业排队分 CPU，自己等的总时间可能更长。正确做法：多道的优点永远落在"不让资源闲着"上——一个程序等 I/O 时，CPU 切去跑另一个，空闲时间被填上，利用率和吞吐量上升；问优点就找"利用率/吞吐量提高"这类整体效率的说法。

【类比】

像一个厨师同时做三道菜：切菜的间隙把汤炖上，炖汤的间隙去炒菜，灶台和人都不闲着，一小时出的菜更多；但单看那道汤，它等灶台的时间反而比"只做一道汤"时更长。快的是整桌菜，不是某一道。

【口诀】 一个等菜一个炒，CPU 从此不闲着；整体快了单个慢，内存反而要更多。

【考频】 「多道程序」在 2009–2022 年解析中出现 5 次 · 中频

第 29 题 · 磁盘逻辑格式化程序的工作（正确答案：B）

题干与选项：

下列选项中，磁盘逻辑格式化程序所做的工作是（）。

Ⅰ．对磁盘进行分区

Ⅱ．建立文件系统的根目录

Ⅲ．确定磁盘扇区校验码所占位数

Ⅳ．对保存空闲磁盘块信息的数据结构进行初始化

- A. 仅 II
- B. 仅 II 、IV
- C. 仅III、IV
- D. 仅 I 、 II 、IV

入手点： 格式化分两级：低级格式化是“划格子”（磁道、扇区），逻辑格式化是“建账本”（文件系统）；逻辑格式化程序干的就是建账本的活。

题干专有名词从零解释：

- 先想“低级格式化”：在磁盘上划出磁道和扇区；是磁盘出厂前或最底层的准备工作。
- 再想“逻辑格式化”：在已经划好格子的磁盘上，建立文件系统；让操作系统能按文件来存取。
- 再想“文件系统”：管理文件的办法；包括根目录、文件分配表（或位图）、空闲块信息这些账本。
- 再想“分区”：把一块磁盘分成几个逻辑盘；分区和格式化是两步，分区在前、格式化在后。

考查知识点：

- 先想逻辑格式化建什么：根目录（文件的目录项）、文件分配表或位图（哪块空、哪块被占）、空闲块管理信息；建好这些，文件才能存取。
- 再想低级格式化建什么：磁道、扇区、扇区头；那是低级格式化程序的活，不是逻辑格式化的活。
- 再想分区是干什么：把磁盘切成几个区；分区程序干这个，不是逻辑格式化程序。
- 再想结论：逻辑格式化程序的工作是建立文件系统，就是 B。
- 边界情况：逻辑格式化一般不破坏低级格式化的格子；它只是在格子之上建账本。

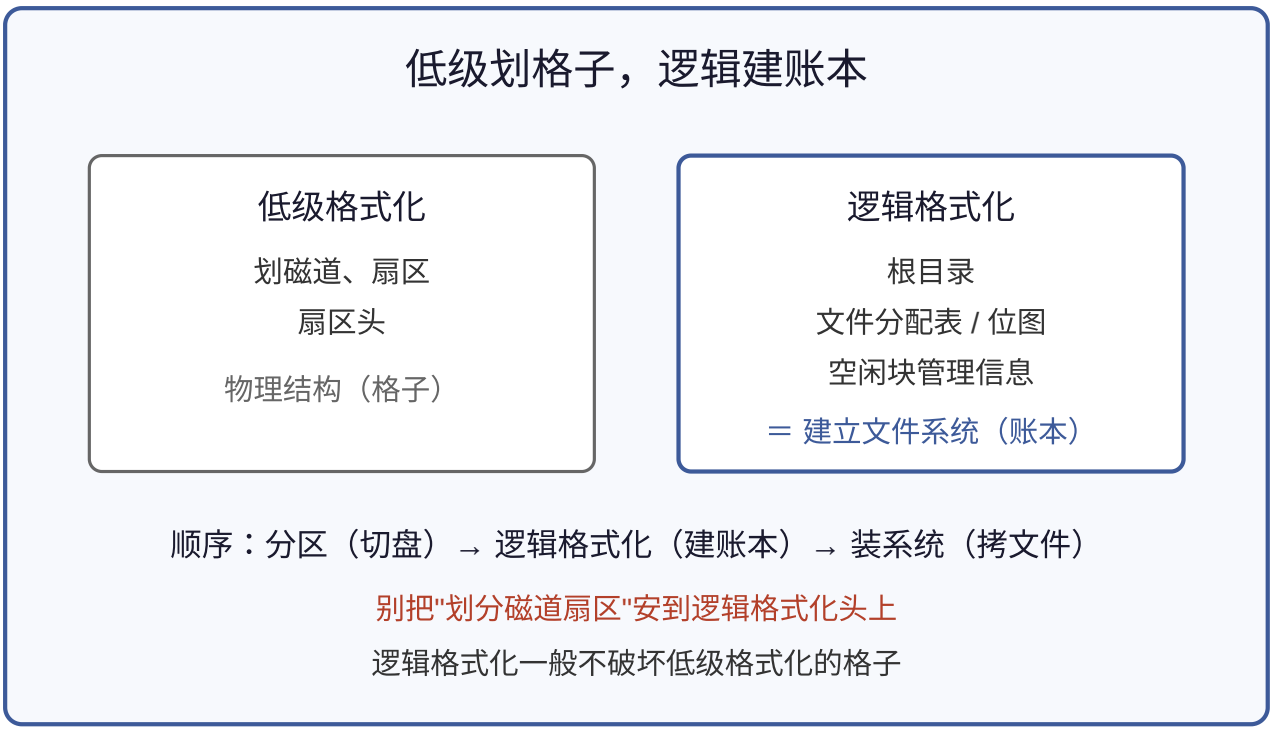
正确选项为什么对： 逻辑格式化就是建立文件系统（根目录、分配表、空闲信息），就是 B，选 B。

错误选项为什么错＋举例：

- A（划分磁道和扇区）：那是低级格式化的活；逻辑格式化不管格子，只管账本。教训：先分清低级和逻辑，低级划格子、逻辑建账本。
- C（给磁盘分区）：分区是把磁盘切成几个区，是分区程序的活；逻辑格式化是在分区之后、对某个分区建文件系统。教训：分区和格式化是两步，别混。
- D（安装操作系统）：装系统是把系统文件拷进磁盘，那是安装程序的活；逻辑格式化只建文件系统结构，不装系统。教训：建账本和装系统是两回事。

笔记本一句话： 第 29 题＋逻辑格式化：建文件系统（根目录、分配表、空闲信息）；划格子是低级格式化。

【图像】



两栏对比：低级格式化划磁道和扇区是划格子；逻辑格式化建根目录、分配表位图、空闲信息是建账本；分区在前、装系统在后

【定位】 操作系统 · 第4章文件管理 · 文件系统（逻辑格式化）

【易混对比】

- "低级格式化"和"逻辑格式化"是两回事：低级是在盘上划磁道和扇区（划格子），逻辑是在格子之上建文件系统（建账本）。
- "分区"和"格式化"是两回事：分区是把一块盘切成几个逻辑盘，分区程序干；逻辑格式化是对某个分区建文件系统，在分区之后做。
- "建文件系统"和"装操作系统"是两回事：建文件系统是搭好根目录、分配表这些空账本，装系统是把系统文件拷进去；账本搭好不等于东西搬进去。

低级划格子，逻辑建账本：逻辑格式化程序的工作就是建立文件系统。

【坑位】

看到"格式化"就选"划分磁道和扇区"——那是低级格式化的活，逻辑格式化不管格子。

最容易踩的坑是把两级格式化混成一团。为什么错：磁道、扇区、扇区头是低级格式化在出厂或最底层划好的；逻辑格式化拿到的是已经划好格子的盘，它只建让操作系统能按文件存取的账本——根目录（文件的目录项）、文件分配表或位图（哪块空、哪块被占）、空闲块管理信息。正确做法：先看定语——"逻辑"格式化对应"逻辑"结构（文件系统），"低级"格式化对应"物理"结构（磁道扇区）。

【类比】

像新房交付：低级格式化是开发商砌墙隔出房间（格子有了）；逻辑格式化是物业来装门牌、建住户花名册、画车位管理图（账本有了）；分区是先把一栋楼切成几个单元，装操作系统是最后把家具搬进去。四步各有各的活。

【口诀】 低级划格子，逻辑建账本；分区在前，装系统在后。

【考频】 「逻辑格式化」在 2009–2022 年解析中出现 7 次 · 中频

第 30 题 · 文件权限位串的最少位数（正确答案：D）

题干与选项：

某文件系统中，针对每个文件，用户类别分为 4 类：安全管理员、文件主、文件主的伙伴、其他用户；访问权限分为 5 种：完全控制、执行、修改、读取、写入。若文件控制块中用二进制位串表示文件权限，为表示不同类别用户对一个文件的访问权限，则描述文件权限的位数至少应为（）。

- A. 5
- B. 9
- C. 12
- D. 20

入手点： 权限就三个动作（读、写、执行），主体就三类人（属主、同组、其他）；每类人、每个动作都要一位；乘起来就是最少位数。

题干专有名词从零解释：

- 先想“访问权限”：谁能对文件干什么；读（看内容）、写（改内容）、执行（运行它）。
- 再想“位串”：用 0 和 1 的串表示信息；一位表示“有”或“没有”。
- 再想“三类用户”：文件属主（文件的主人）、同组用户（和属主同一组的人）、其他用户（剩下所有人）。

考查知识点：

- 先想每类用户要几位：读、写、执行三个动作，每个动作一位，一类用户要 3 位。
- 再想三类用户要几位：3 类用户 × 每类 3 位 = 9 位。
- 再想为什么是最少：每类用户的每个动作都要独立表示，少一位就表示不全；9 位不多不少，正好。
- 再想结论：最少 9 位，就是 D。
- 边界情况：如果系统还有别的权限（比如粘着位、特殊位），位数会更多；本题问的是基本读写执行三类用户的最少位数，就是 9 位。

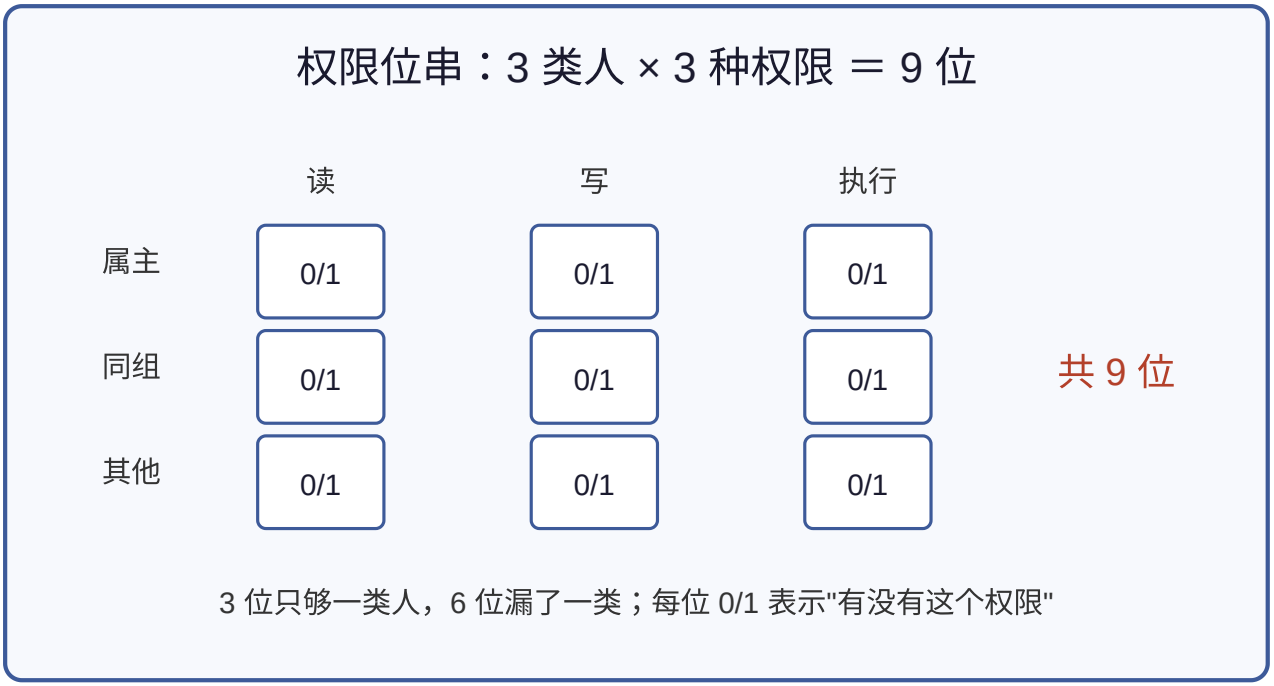
正确选项为什么对： 3 类用户 × 3 种权限 = 9 位，就是 D，选 D。

错误选项为什么错＋举例：

- A（3 位）：那只是一类用户的位数；三类用户都要表示，3 位不够。教训：先数主体有几类，再乘。
- B（6 位）：那是两类用户的位数；三类用户要 9 位，6 位少了一类。教训：属主、同组、其他，三类一个不能少。
- C（12 位）：多算了 3 位；读写执行是 3 种，不是 4 种；3×3=9，不是 12。教训：权限动作是读、写、执行三种，别多算。

笔记本一句话： 第 30 题＋权限位串：3 类用户 × 读、写、执行 3 种 = 9 位。

【图像】



三行三列权限表：属主同组其他三类用户，读写作执行三种权限，每格一位，共九位

【定位】 操作系统 · 第4章文件管理 · 文件保护（访问权限）

【易混对比】

- "3 位"和"9 位"是两回事：3 位只够表示一类用户的权限，三类用户都要表示就得 $3 \times 3 = 9$ 位。
- "用户类别"和"权限动作"是两回事：类别是"谁"（属主、同组、其他），动作是"干什么"（读、写、执行）；位数是两者相乘，不是一边。
- "最少位数"和"实际位数"是两回事：9 位是读写执行三类用户的基本盘；系统再加特殊位（比如粘着位），实际会更多，但"最少"就是 9。

先数有几类人，再数每人几个动作，相乘就是最少位数：三类乘三种得九位。

【坑位】

漏数一类用户就少 3 位——属主、同组、其他，三类一个不能少。

最容易踩的坑是答 3 或 6。为什么错：答 3 是只算了一类用户（比如只算属主），同组和其他的权限就没地方表示了；答 6 是漏了一类（比如把同组和其他并成一类），三类用户权限分开管是基本要求。正确做法：先把"谁"列全——文件属主（文件的主人）、同组用户（和属主同一组的人）、其他用户（剩下所有人），共 3 类；再把"干什么"列全——读、写、执行，共 3 种；$3 \times 3 = 9$ 位，每位 0/1 表示"有没有这个权限"。

【套路】

- 第一步，数用户类别：属主、同组、其他，共几类。
- 第二步，数权限动作：读、写、执行，共几种。
- 第三步，相乘：类别数乘以动作数，就是最少位数。

类别乘动作，9 位是基本盘。

【类比】

像 3 栋楼、每栋 3 层，每层装一盏灯表示"这层有人"：一共要 $3 \times 3 = 9$ 盏灯；少一栋楼的灯，那栋楼的情况就表示不出来了。

【口诀】 三类人三种权，三三得九不能减。

【考频】 「访问权限」在 2009–2022 年解析中出现 4 次 · 中频

第 31 题 · 硬链接文件的读写指针与内存索引结点正误（正确答案：B）

题干与选项：

若文件 f1 的硬链接为 f2，两个进程分别打开 f1 和 f2，获得对应的文件描述符为 fd1 和 fd2，则下列叙述中，正确的是（ ）。

Ⅰ．f1 和 f2 的读写指针位置保持相同

Ⅱ．f1 和 f2 共享同一个内存索引结点

Ⅲ．fd1 和 fd2 分别指向各自的用户打开文件表中的一项

- A. 仅 Ⅰ
- B. 仅 Ⅱ、Ⅲ
- C. 仅 Ⅰ、Ⅱ
- D. Ⅰ、Ⅱ和Ⅲ

入手点： 硬链接的精髓是“多个名字、同一个文件”：目录项有多个，但索引结点只有一个；读写指针是“每次打开”的东西，打开几次就有几个；索引结点是“文件本身”的东西，只有一个。

题干专有名词从零解释：

- 先想“硬链接”：给同一个文件起多个名字；多个目录项指向同一个索引结点；删一个名字，文件还在（除非名字全删光）。
- 再想“索引结点”：存文件信息的结构（大小、位置、权限）；一个文件只有一个索引结点。
- 再想“内存索引结点”：索引结点调入内存后的副本；同一个文件的内存索引结点也只有一个（大家共用）。
- 再想“读写指针”：每次打开文件时，操作系统给这次打开配的位置标记；读到哪、写到哪，由它记着；打开几次，就有几个。

考查知识点：

- 先想读写指针：每次打开文件，操作系统都给一次独立的文件偏移量；两个进程（或两次打开）各读各的，互不干扰；所以读写指针是独立的。
- 再想内存索引结点：多个硬链接指向同一个索引结点，调入内存后也是同一个内存索引结点；大家共用它，文件大小、位置的改动，所有链接都看得到。
- 再想结论：读写指针独立、内存索引结点共享，就是 B。
- 边界情况：软链接不一样——软链接有自己的索引结点（存的是目标路径）；本题说的是硬链接，不要套软链接的情况。

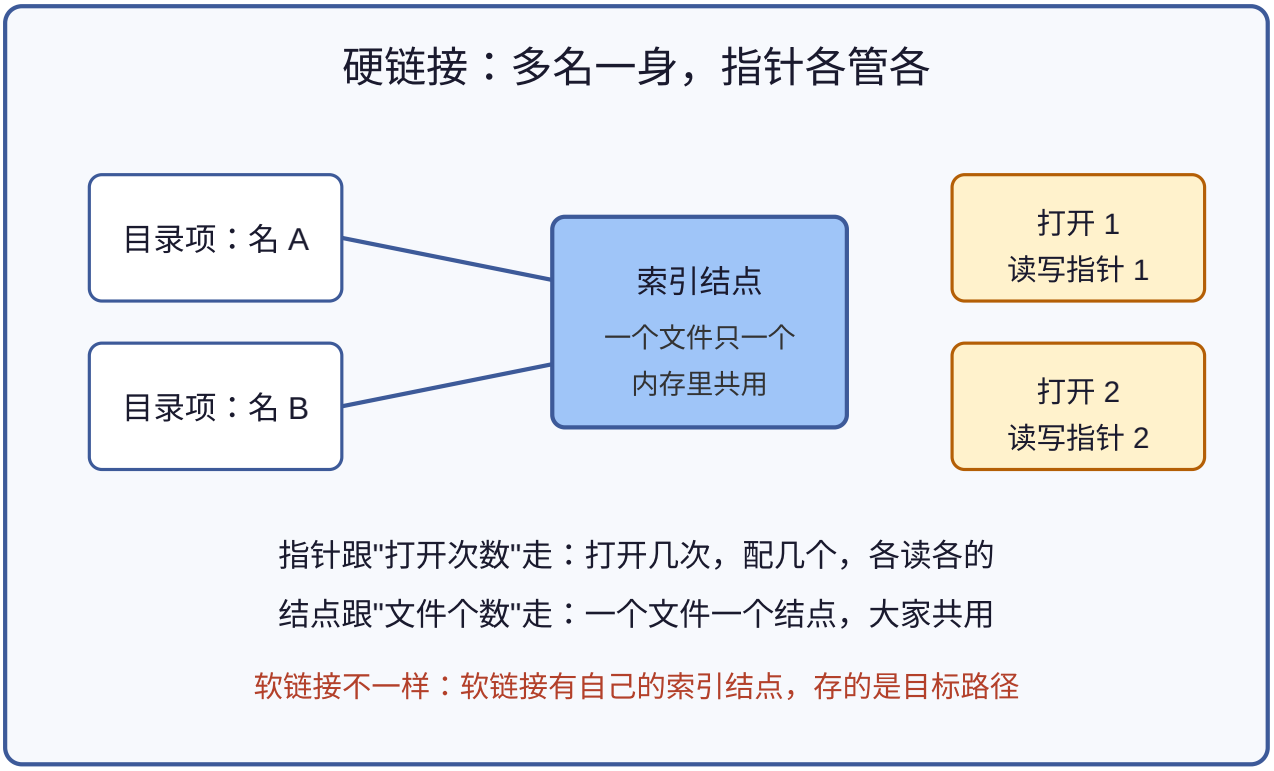
正确选项为什么对： 读写指针每次打开独立配，内存索引结点同一个文件只一个、大家共用，就是 B，选 B。

错误选项为什么错＋举例：

- A（读写指针共享）：两次打开共用一个指针，一个进程读了 10 个字节，另一个进程再读就从第 11 个字节开始，那就乱了；实际上各读各的。教训：指针是“每次打开”的东西，不是“每个文件”的东西。
- C（内存索引结点独立）：硬链接指向同一个索引结点，内存里也是同一个；独立的话，改动文件大小就不同步了。教训：索引结点是“每个文件”的东西，不是“每次打开”的东西。
- D（两者都共享或都独立）：两个正好说反了；指针独立、结点共享，一个不能反。教训：指针看“打开次数”，结点看“文件个数”。

笔记本一句话： 第 31 题＋硬链接：读写指针每次打开独立；内存索引结点同一个文件只一个、共用。

【图像】



硬链接示意：两个目录项名A名B指向同一个磁盘索引结点，调入内存后同一个内存索引结点；两次打开各配一个独立的读写指针

【定位】 操作系统 · 第4章文件管理 · 文件共享（硬链接）

【易混对比】

- "硬链接"和"软链接"是两回事：硬链接是给同一个文件起多个名字，多个目录项指向同一个索引结点；软链接有自己的索引结点，存的是目标路径。
- "读写指针"和"索引结点"是两回事：读写指针是"每次打开"配的位置标记，打开几次就有几个；索引结点是"每个文件"的身份信息，一个文件只有一个。
- "删除一个名字"和"删除文件"是两回事：硬链接删掉一个名字，文件还在（引用计数减一）；名字全删光，文件才真正消失。

指针跟打开次数走，结点跟文件个数走：硬链接下读写指针独立、内存索引结点共享。

【坑位】

别以为"同一个文件"就"共用一个读写指针"——指针是每次打开独立配的。

最容易踩的坑是觉得两个硬链接打开的是同一个文件，读写位置肯定共享。为什么错：读写指针记录的是"这次打开读到哪、写到哪"；两个进程各打开一次，各读各的、互不干扰——如果共用一个指针，一个进程读了 10 个字节，另一个再读就从第 11 个字节开始，全乱了。正确做法：分清"打开"和"文件"——打开几次就有几个指针（独立）；一个文件只有一个索引结点，调入内存后也是同一个内存索引结点（共享），文件大小、位置的改动所有链接都看得到。

【类比】

像一本书被两个人各夹了一个书签：书签（读写指针）各是各的，你看到第 10 页不影响我看到第 3 页；书本身（索引结点）只有一本，书里改了一个字，两个人看到的都是改过的。

【口诀】 指针跟打开走，结点跟文件走；硬链接多名一身。

【考频】 「硬链接」在 2009–2022 年解析中出现 7 次 · 中频

第 32 题 · 磁盘读到内存的 DMA 操作正确顺序（正确答案：B）

题干与选项：

系统将数据从磁盘读到内存的过程包括以下操作：

①DMA 控制器发出中断请求 ②初始化 DMA 控制器并启动磁盘 ③从磁盘传输一块数据到内存缓冲区 ④执行"DMA 结束"中断服务程序

正确的执行顺序是（ ）。

- A. ③→①→②→④
- B. ②→③→①→④
- C. ②→①→③→④
- D. ①→②→④→③

入手点： DMA 的流程就五步：CPU 先交代（地址、长度），DMA 接管总线去干活，设备读数据，DMA 把数据写进内存，干完发中断告诉 CPU；顺序就是“交代→干活→写内存→报告”。

题干专有名词从零解释：

- 先想“DMA（直接存储器访问）”：外设和内存之间直接传数据，CPU 不参与搬运；CPU 只负责开始和结束。
- 再想“DMA 控制器”：专门管 DMA 传送的硬件；CPU 把任务交代给它，它接管总线干活。
- 再想“中断”：任务干完后，DMA 控制器发给 CPU 的“干完了”的信号；CPU 收到才来处理后续。

考查知识点：

- 先想第一步：CPU 向 DMA 控制器交代任务——内存地址、磁盘地址、传送长度。
- 再想第二步：DMA 控制器接管总线，向磁盘发读命令；磁盘开始读数据。
- 再想第三步：磁盘读出的数据经 DMA 控制器写入内存；这个过程 CPU 不参与。
- 再想第四步：数据全部传完，DMA 控制器向 CPU 发中断，报告完成。
- 再想第五步：CPU 收到中断，处理后续（比如把等待的进程唤醒）。
- 再想结论：交代→读盘→写内存→中断，就是 B。
- 边界情况：DMA 传送过程中 CPU 可以去干别的活；但 DMA 控制器和 CPU 不能同时用总线，总线一次只给一方。

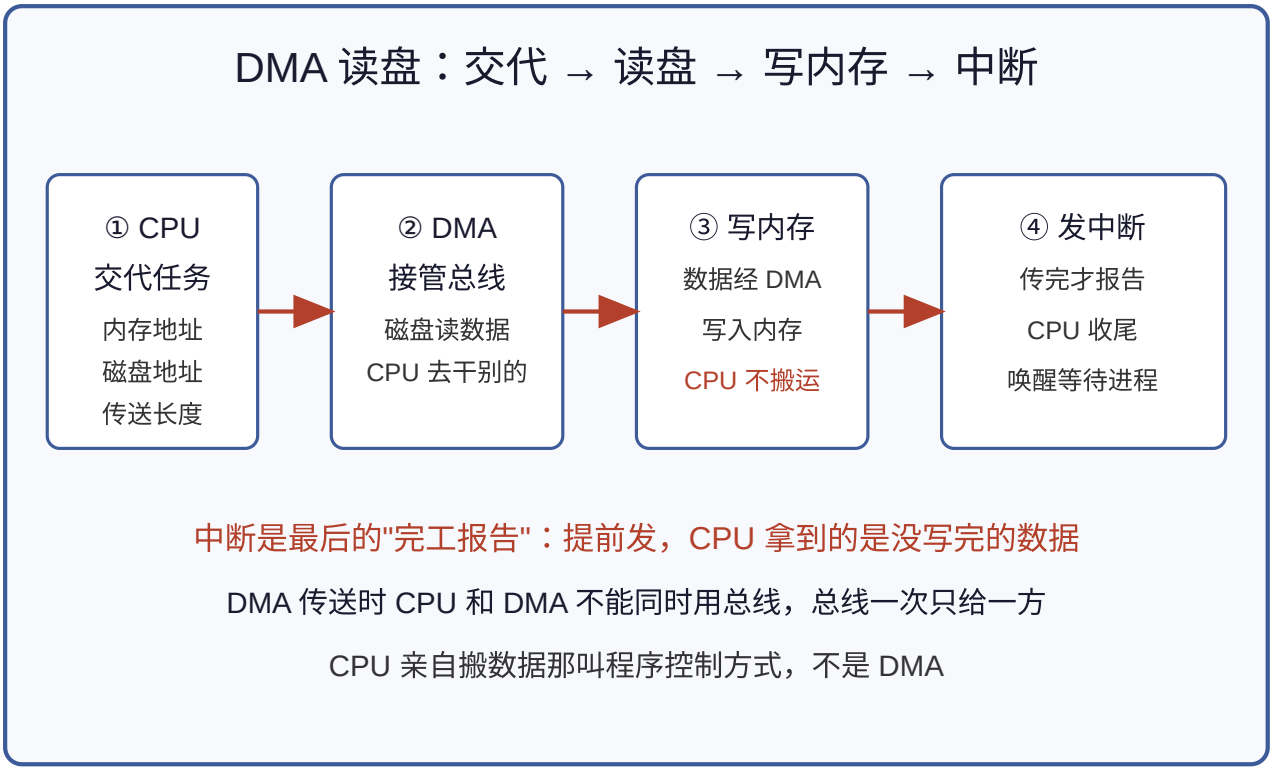
正确选项为什么对： 交代任务、读盘、写内存、发中断，这个顺序和 DMA 流程对得上，就是 B，选 B。

错误选项为什么错＋举例：

- A（先发中断、后写内存）：中断是“干完了”的报告；内存还没写完就报告，CPU 收到的是没写完的数据。教训：中断是最后一步，不是第一步。
- C（CPU 参与搬数据）：DMA 的意义就是 CPU 不搬数据；CPU 搬数据那是程序控制方式，不是 DMA。教训：DMA 流程里 CPU 只交代和收尾，不搬运。
- D（先写内存、后读盘）：数据还没从盘上读出来，往内存里写什么？顺序反了。教训：先从盘上读出来，再往内存里写。

笔记本一句话： 第 32 题＋DMA 读盘顺序：CPU 交代→DMA 读盘→写内存→发中断；CPU 不搬数据。

【图像】



DMA 读盘五步：CPU 交代任务，DMA 接管总线，磁盘读数据，DMA 写内存，中断报告 CPU 收尾；CPU 不参与搬运

【定位】 操作系统 · 第5章输入输出管理 · I/O 方式（DMA）

【易混对比】

- "DMA 方式"和"程序控制方式"是两回事：DMA 下 CPU 只交代任务和收尾，不搬数据；程序控制下 CPU 亲自一字节一字节地搬。
- "中断"和"开始信号"是两回事：中断是 DMA 干完活后发给 CPU 的"完工报告"，在最后一步；把它放开头，CPU 收到的是还没写完的数据。
- "先读盘"和"先写内存"是两回事：数据还在盘上没读出来，往内存里写什么？必须先从盘上读出来，再往内存里写。

CPU 只交代和收尾，搬运是 DMA 的活；中断是最后的完工报告。

【坑位】

中断永远是最后一步——它是"干完了"的报告，提前发等于谎报军情。

最容易踩的坑是把中断排在写内存之前。为什么错：中断是 DMA 控制器告诉 CPU"数据已经全部写进内存了"；内存还没写完就发中断，CPU 去处理时拿到的是不完整的数据。正确做法：按"交代→干活→写内存→报告"的流水记——CPU 先交代（内存地址、磁盘地址、传送长度），DMA 控制器接管总线、让磁盘读数据，读出的数据经 DMA 写入内存（CPU 这段时间去干别的），全部传完 DMA 发中断，CPU 收到后收尾（比如唤醒等待的进程）。

【套路】

- 第一步，CPU 交代任务：把内存地址、磁盘地址、传送长度告诉 DMA 控制器。
- 第二步，DMA 接管干活：DMA 控制器接管总线，向磁盘发读命令，磁盘读出数据。
- 第三步，数据写入内存：读出的数据经 DMA 控制器写入内存，CPU 不参与搬运。

- 第四步，中断报告收尾：传完后 DMA 发中断，CPU 收到后处理后续。

交代、读盘、写内存、中断——CPU 不搬数据，中断压最后。

【类比】

像老板派员工去仓库取货：老板交代"去哪个仓库、拿多少货、送到哪"（交代任务），员工自己开车去取（DMA 干活），货卸进店里（写内存），最后打电话跟老板说"货到了"（中断报告）。老板全程不碰货，只负责开头交代和结尾验收。

【口诀】 交代读盘写内存，中断最后来报到；CPU 不搬数据。

【考频】 「DMA」在 2009–2022 年解析中出现 16 次 · 高频

第 33 题 · OSI 各层封装开销下的应用层数据传输效率（正确答案：A）

题干与选项：

假设 OSI 参考模型的应用层欲发送 400B 的数据（无拆分），除物理层和应用层之外，其他各层在封装 PDU 时均引入 20B 的额外开销，则应用层数据传输效率约为（ ）。

- A. 80%
- B. 83%
- C. 87%
- D. 91%

入手点： 传输效率就是“真正的数据占总共传了多少的比例”；每层封装都要加一层头，头加得越多，效率越低；先数有几层加头、每层加多少，再算比例。

题干专有名词从零解释：

- 先想“OSI 七层”：从上到下是应用层、表示层、会话层、传输层、网络层、数据链路层、物理层；数据从应用层往下走，每层都给它包一层。
- 再想“封装”：每层给数据加个头（控制信息）；比如传输层加端口号、网络层加地址。
- 再想“传输效率”：真正的数据长度 ÷ 总共传的长度；头越少，效率越高。

考查知识点：

- 先想哪几层加头：应用层是数据的源头，不加头；物理层只管传比特流，不加头；中间 5 层（表示、会话、传输、网络、数据链路）各加一层头。
- 再想头一共多少：每层加 20B，5 层一共是 5×20B=100B。
- 再想总共传多少：应用层数据 400B，加上 100B 的头，一共 500B。
- 再想效率：400÷500=80%，就是 A。
- 边界情况：如果把物理层也算进去，或者只算传输、网络、数据链路 3 层，头数就错了；本题的层数是“除应用层和物理层外的 5 层”，不要多算、不要少算。

正确选项为什么对： 5 层头一共 100B，400÷(400+100)=80%，就是 A，选 A。

错误选项为什么错＋举例：

- B（只算 3 层头的）：只算了传输、网络、数据链路 3 层（60B），效率算成 400÷460；漏了表示层和会话层的头。教训：除应用层和物理层外，一共是 5 层，不是 3 层。
- C（把物理层也算进去了）：物理层不加头；把它算进去，头就多了一层，效率算小了。教训：物理层只传比特流，不封装。
- D（把应用层数据当成总长度了）：400B 是数据本身，不是总共传的长度；总共传的是数据加头（500B）。教训：效率的分母是“数据加头”，不是数据本身。

笔记本一句话： 第 33 题＋封装效率：除应用层和物理层外 5 层各加头；效率=数据÷（数据＋头）。

【图像】

OSI 封装开销示意



五层各加 20B 头示意：400B 应用数据加 100B 头共 500B，效率为 400 除以 500

【定位】 计算机网络 · 第1章计算机网络体系结构 · OSI 参考模型与封装开销

【易混对比】

- "加头的 5 层"和"只算 3 层"是两回事：除应用层和物理层外，表示、会话、传输、网络、数据链路一共 5 层加头；只算传输、网络、数据链路会漏掉表示层和会话层那 40B。
- "物理层"和"其他层"是两回事：其他层都要加头，物理层只管把比特流推出去，一个字节的头也不加。
- "效率的分母"和"数据本身"是两回事：分母是数据加头（500B），不是数据本身（400B）；拿 400 除以 400 会得出 100% 的离谱答案。

除应用层和物理层外 5 层各加 20B 头，共 100B；效率等于 400 除以 400 加 100，得 80%。

【坑位】

应用层和物理层都不加头——别把七层全乘上，也别只乘三层。

最容易踩的坑：头数按 7 层或 3 层算。为什么错：应用层是数据的老家，自己不给自己贴标签；物理层是搬运工，只管搬不管贴；表示层和会话层虽然平时存在感低，头一分也不能少。正确做法：七层先排除应用层和物理层，剩 5 层，头 = 5×20B = 100B，再套效率 = 数据 ÷（数据＋头）。

【套路】

- 第一步，定层数：七层里去掉应用层和物理层，剩 5 层加头。
- 第二步，算总头：层数乘以每层头长，$5 \times 20\text{B} = 100\text{B}$。
- 第三步，求效率：数据 ÷（数据＋头），$400 \div 500 = 80\%$。

先数加头层数，再算总头长，最后数据除以数据加头。

【类比】

像寄快递：400B 的货，每经手一层就贴一张 20B 的面单（地址、电话、保价各贴一张），一共贴了 5 张；运费按总重量（货＋面单）摊，货占的比例就是效率。面单贴得越多，货的占比就越低。

【口诀】 应用物理不加头，五层各贴二十 B；效率数据除总量。

【考频】 「传输效率」在 2009–2022 年解析中出现 8 次 · 中频

第 34 题 · 无噪声极限速率不小于 30dB 香农极限时的最少信号状态数（正确答案：D）

题干与选项：

若信道在无噪声情况下的极限数据传输速率不小于信噪比为 30dB 条件下的极限数据传输速率，则信号状态数至少是（）。

- A. 4
- B. 8
- C. 16
- D. 32

入手点： 两个公式：香农公式算有噪声的极限，奈奎斯特公式算无噪声的极限；让无噪声的极限大于等于有噪声的极限，解出信号状态数；状态数是 2 的幂次，不够要往上凑。

题干专有名词从零解释：

- 先想“香农极限”：有噪声信道的最大数据速率，等于带宽 × $\log_2$(1+信噪比)。
- 再想“奈奎斯特极限”：无噪声信道的最大数据速率，等于 2×带宽 × $\log_2$(信号状态数)。
- 再想“信噪比（dB)”：信号功率和噪声功率的比值取对数；30dB 就是比值 1000（10 的 3 次方)。
- 再想“信号状态数”：信号能取几种不同的值；比如 2 种状态就是 0 和 1，4 种状态就是 00、01、10、11。

考查知识点：

- 先想香农极限：信噪比 30dB，比值是 1000；香农极限 = 带宽 × $\log_2$(1+1000) = 带宽 × $\log_2$(1001)，$\log_2$(1001) 大约是 9.97。
- 再想奈奎斯特极限：无噪声极限 = 2×带宽 × $\log_2$(V)，V 是信号状态数。
- 再想不等式：要无噪声极限不小于香农极限，就是 2×$\log_2$(V) ⩾ 9.97，$\log_2$(V) ⩾ 4.985，V ⩾ 2 的 4.985 次方，大约是 31.6。
- 再想最少状态数：状态数是 2 的幂次，31.6 往上凑到 32；所以最少 32 种状态，就是 D。
- 边界情况：31.6 不能向下取整到 16（16 种状态的极限小于香农极限，不够)；也不能取 31（不是 2 的幂次，信号状态数按 2 的幂次取)。

正确选项为什么对： V 要不小于 31.6，往上凑到 2 的幂次是 32，就是 D，选 D。

错误选项为什么错＋举例：

- A（16 种)：16 种状态的奈奎斯特极限是 2×带宽×4=8×带宽，小于香农极限的 9.97×带宽，不够。教训：算出 31.6 要往上凑，不能往下取。
- B（8 种)：8 种状态的极限是 2×带宽×3=6×带宽，差得更多。教训：状态数越少，极限越低。
- C（64 种)：64 种状态的极限够了，但不是最少；题目问的是“最少”，64 比 32 多了一倍。教训：问“最少”就要取刚好够的那个幂次。

笔记本一句话： 第 34 题＋信号状态数：2×$\log_2$(V) ⩾ $\log_2$(1+信噪比)，算出值往上凑到 2 的幂次。

【图像】

香农 vs 奈奎斯特示意

香农（有噪声）

30dB → 比值 1000

极限 = 带宽 × $\log_2(1+1000)$

≈ 带宽 × 9.97

这是要追上的目标

奈奎斯特（无噪声）

极限 = 2 × 带宽 × $\log_2(V)$

令 ≥ 9.97 × 带宽

得 V ≥ 31.6

向上凑 2 的幂 → 32

最少 32 种信号状态（16 不够，31 不是 2 的幂）

口诀：dB 先变比值，两边各算极限，解出往上凑幂

香农极限与奈奎斯特极限对比示意：30dB 对应比值 1000，香农极限约 9.97 倍带宽，信号状态数向上凑 2 的幂到 32

【定位】 计算机网络 · 第2章物理层 · 信道极限容量（奈奎斯特与香农公式）

【易混对比】

- "奈奎斯特"和"香农"是两回事：前者管无噪声信道，公式里有信号状态数；后者管有噪声信道，公式里有信噪比；一题里两条公式各算一遍，别串。
- "30dB"和"比值 30"是两回事：30dB 是对数口径，真实比值是 10 的 3 次方即 1000；直接把 30 代进 log 里会算出离谱的结果。
- "算出 31.6"和"选 16"是两回事：31.6 是下限，只能往上凑到 2 的幂 32；往下取整到 16 会让速率不够香农极限。

香农极限约 9.97 倍带宽，无噪声这边要 2 倍 $\log_2(V)$ 不小于 9.97，解得 V 最少取 32。

【坑位】

30dB 先换成比值 1000 再代公式，别直接拿 30 去算 log。

最容易踩的坑：把 30dB 当成比值 30 代进 $\log_2(1+30)$。为什么错：dB 是对数单位，30dB = 10 的 3 次方 = 1000，真实比值大得多。正确做法：先 dB 转比值（30dB→1000），香农这边得 $\log_2(1001)\approx 9.97$；无噪声这边 $2\times\log_2(V)\geqslant 9.97$，解得 $V\geqslant 31.6$，再向上凑 2 的幂得 32。

【套路】

- 第一步，dB 转比值：30dB → 10 的 3 次方 = 1000。
- 第二步，算香农极限：带宽 × $\log_2(1+1000)$，约 9.97 倍带宽。
- 第三步，列不等式：$2 \times \text{带宽} \times \log_2(V) \geq 9.97 \times \text{带宽}$，约掉带宽得 $V \geq 31.6$。
- 第四步，向上凑幂：信号状态数取 2 的幂次，31.6 向上凑到 32。

dB 先转比值，两边各算极限，解出 V 后向上凑 2 的幂。

【类比】

像给水管定水表：香农先量出"有杂质时最多流多少"（9.97 档）；无噪声这边要选水龙头档位（信号状态数），档位只有 2、4、8、16、32……几档可选；算出来需要 31.6 档，没有这档，只能买大一档的 32 档，买 16 档水就不够流。

【口诀】 dB 先变比值，香农奈氏各算；解出往上凑幂。

【考频】 「香农」在 2009–2022 年解析中出现 4 次 · 中频

第 35 题 · 802.11 数据帧 F 的地址 1/2/3（网络图）（正确答案：B）

题干与选项：

在下图所示的网络中，若主机 H 发送一个封装访问 Internet 的 IP 分组的 IEEE 802.11 数据帧 F，则帧 F 的地址 1、地址 2 和地址 3 分别是（）。

- A. 00-12-34-56-78-9a，00-12-34-56-78-9b，00-12-34-56-78-9c
- B. 00-12-34-56-78-9b，00-12-34-56-78-9a，00-12-34-56-78-9c
- C. 00-12-34-56-78-9b，00-12-34-56-78-9c，00-12-34-56-78-9a
- D. 00-12-34-56-78-9a，00-12-34-56-78-9c，00-12-34-56-78-9b

> 注：本题是网络图题，图示数据如下：主机 H（00-12-34-56-78-9a）、AP（00-12-34-56-78-9b）、AP 与 R 间链路旁标注 00-12-34-56-78-9c、路由器 R、Internet。选项依次为地址 1、2、3：A.9a、9b、9c B.9b、9a、9c C.9b、9c、9a D.9a、9c、9b。

入手点： 802.11 帧的三个地址各有分工：地址 1 是“这一跳的接收端”，地址 2 是“这一跳的发送端”，地址 3 是“在分布系统那一边的地址”；先看帧是从 H 发往 AP 的，接收端就是 AP，发送端就是 H。

题干专有名词从零解释：

- 先想“802.11 数据帧”：无线局域网里传数据的帧；帧头里有三个地址字段。
- 再想“地址 1（接收地址）”：这一跳无线链路上的接收端；帧从 H 发到 AP，接收端就是 AP。
- 再想“地址 2（发送地址）”：这一跳无线链路上的发送端；帧从 H 发出，发送端就是 H。
- 再想“地址 3”：帧在分布系统（有线那一侧）的另一端地址；H 发往 Internet，分布系统那一侧的地址就是 R 在该链路上的地址。
- 再想“AP”：无线接入点；无线设备通过它接入有线网络。
- 再想“MAC 地址”：网卡的硬件地址；本题用后两位区分：H 是 9a，AP 是 9b，R 在该链路上的地址是 9c。

考查知识点：

- 先想帧的方向：H 要上网，帧从 H 发出，经 AP，再经 R 去 Internet；无线这一跳是从 H 到 AP。
- 再想地址 1：这一跳的接收端是 AP，AP 的地址是 9b；所以地址 1 是 9b。
- 再想地址 2：这一跳的发送端是 H，H 的地址是 9a；所以地址 2 是 9a。
- 再想地址 3：帧最终要去 Internet，分布系统那一侧的地址是 R 在 AP 与 R 间链路上的地址 9c；所以地址 3 是 9c。
- 再想结论：地址 1、2、3 依次是 9b、9a、9c，就是 B。
- 边界情况：地址 1 和地址 2 永远是“这一跳”的收和发，不要和最终的源和目的搞混；地址 3 在不同方向（从分布系统发往无线端）会变成源地址，方向变了，地址 3 的含义也变。

正确选项为什么对： 地址 1 是接收端 AP（9b），地址 2 是发送端 H（9a），地址 3 是分布系统侧 R 的地址（9c），就是 B，选 B。

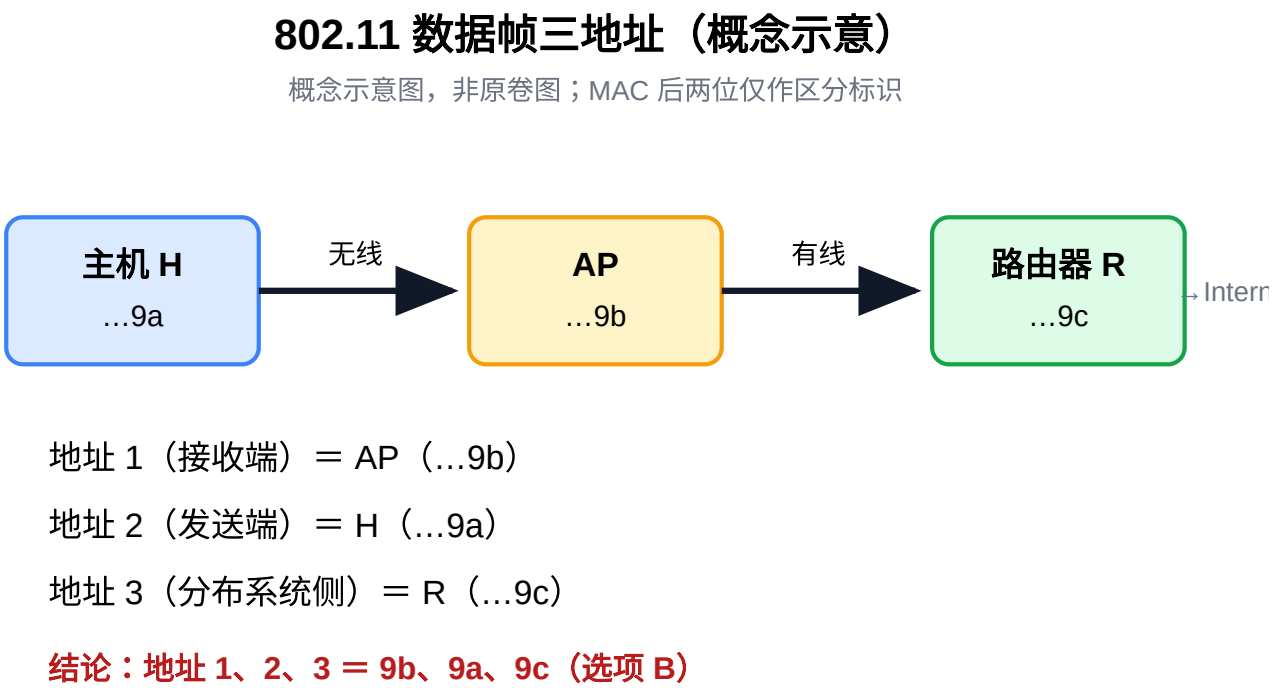
错误选项为什么错＋举例：

- A（9a、9b、9c）：地址 1 和地址 2 颠倒了；地址 1 是接收端（AP，9b），不是发送端（H，9a）。教训：地址 1 永远是接收端，先看帧往哪跳。

- C（9b、9c、9a）：地址 2 和地址 3 颠倒了；地址 2 是发送端 H（9a），不是分布系统侧的 9c。教训：地址 2 是这一跳的发送端，不是分布系统侧。
- D（9a、9c、9b）：三个全乱了；地址 1 不是 H，地址 2 不是 9c，地址 3 不是 9b。教训：先定帧的方向，再按“接收、发送、分布系统侧”的顺序填。

笔记本一句话： 第 35 题＋802.11 三地址：地址 1 接收端，地址 2 发送端，地址 3 分布系统侧。

【图像】



802.11 三地址概念示意图（示意）：帧从 H 发往 AP，地址 1 是接收端 AP，地址 2 是发送端 H，地址 3 是有线侧 R

【定位】 计算机网络 · 第3章数据链路层 · 无线局域网 802.11 帧格式

【易混对比】

- "地址 1"和"地址 2"是两回事：地址 1 是这一跳的接收端（AP），地址 2 是这一跳的发送端（H）；谁收谁发看箭头方向，别背死顺序。
- "地址 3"和"最终目的"是两回事：地址 3 是分布系统那一侧的地址（R 在链路上的 9c），不是帧最终要去的 Internet；它是下一站有线侧的门牌。
- "无线这一跳"和"整条路"是两回事：地址 1、2 只管 H→AP 这一跳；帧到了 AP 转有线后，地址就换另一套了。

帧从 H 发往 AP：地址 1 是接收端 AP（9b），地址 2 是发送端 H（9a），地址 3 是有线侧 R（9c）。

【坑位】

地址 1 永远是接收端——先看帧往哪跳，再填收发，别把 1 和 2 写反。

最容易踩的坑：地址 1 填成发送端 H（9a）。为什么错：心里默认"1 就是源"，但 802.11 里地址 1 是接收地址。正确做法：先画箭头 H→AP，箭头指向谁，地址 1 就是谁；箭头从哪来，地址 2 就是谁；地址 3 看分布系统（有线）那一侧是谁。

【套路】

- 第一步，定方向：帧从 H 发出，经 AP 去 Internet，无线一跳是 H→AP。
- 第二步，填收发：地址 1 = 接收端 AP（9b），地址 2 = 发送端 H（9a）。
- 第三步，填分布系统侧：有线那一侧是 R 在链路上的地址 9c，填地址 3。

先定帧的方向，地址 1 收、地址 2 发、地址 3 看有线侧。

【类比】

像宿舍楼下代收点取快递：你（H）把包裹（帧）递给代收点（AP），包裹单上"收件人"（地址 1）写代收点，"寄件人"（地址 2）写你；"转寄地址"（地址 3）写快递中转站（R），因为包裹下一站要进中转站的货车（有线网）。

【口诀】 一收二发三有线，方向定了再填空。

【考频】 「802.11」在 2009–2022 年解析中出现 4 次 · 中频

第 36 题 · 只能作源地址不能作目的地址的 IP 地址（正确答案：A）

题干与选项：

下列 IP 地址中，只能作为 IP 分组的源 IP 地址但不能作为目的 IP 地址的是（ ）。

- A. 0.0.0.0
- B. 127.0.0.1
- C. 200.10.10.3
- D. 255.255.255.255

入手点： “只能作源、不能作目的”的地址，一定是“指代自己、但没法被别人找到”的那种；0.0.0.0 就是“本网络上的本主机”，自己发包时能用它当源，但别人没法把它当目的（因为它不指代任何一台具体的机器）。

题干专有名词从零解释：

- 先想“源地址”：发包的那台机器的地址；收包方靠它知道是谁发来的。
- 再想“目的地址”：收包的那台机器的地址；发包方靠它把包送到地方。
- 再想“0.0.0.0”：表示“本网络上的本主机”；机器还没拿到正式地址时（比如刚开机向 DHCP 要地址），就用它当源地址。
- 再想“环回地址（127.0.0.1）”：指代本机自己；发给它就是发给自己，可以作目的地址。
- 再想“有限广播地址（255.255.255.255）”：发给本网络内的所有机器；可以作目的地址。

考查知识点：

- 先想 0.0.0.0 为什么只能作源：它表示“本网络上的本主机”，没有指代任何一台具体的机器；别人收到目的地址是 0.0.0.0 的包，不知道往哪送；所以它只能出现在源地址字段。
- 再想 0.0.0.0 什么时候用：机器向 DHCP 服务器要地址时，还没有正式地址，就用 0.0.0.0 当源地址发请求。
- 再想环回地址为什么能作目的：127.0.0.1 就是本机；发给它，包就回到本机；可以作目的地址。
- 再想有限广播地址为什么能作目的：255.255.255.255 就是“本网络内的所有机器”；发给它，本网络内的机器都收得到；可以作目的地址。
- 再想结论：只能作源、不能作目的的是 0.0.0.0，就是 A。

正确选项为什么对： 0.0.0.0 表示“本网络上的本主机”，不指代具体机器，只能作源地址，就是 A，选 A。

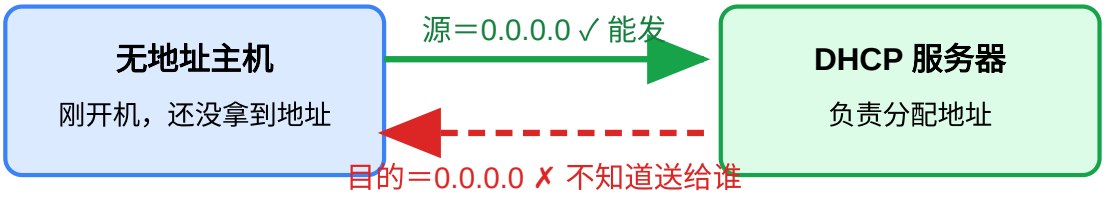
错误选项为什么错＋举例：

- B（127.0.0.1）：环回地址指代本机自己，发给它就是发给自己，可以作目的地址。教训：环回地址是“自己”，不是“没法找”。
- C（255.255.255.255）：有限广播地址指代本网络内所有机器，可以作目的地址。教训：广播地址是“大家”，不是“没法找”。
- D（私网地址）：私网地址（比如 192.168 开头的）指代局域网内具体的机器，可以作目的地址。教训：私网地址在局域网内是能找到的。

笔记本一句话： 第 36 题＋只能作源的地址：0.0.0.0（本网络上的本主机），不指代具体机器。

【图像】

0.0.0.0：只能作源，不能作目的



127.0.0.1（环回）：就是本机自己 → 能作目的

255.255.255.255（有限广播）：本网所有人 → 能作目的

私网地址：局域网内具体机器 → 能作目的

只有 0.0.0.0 不指代具体机器，只能作源

0.0.0.0 作源地址示意：无地址主机向 DHCP 发请求时源地址填 0.0.0.0，他人无法以它为目的地址回包

【定位】 计算机网络 · 第4章网络层 · 特殊 IP 地址

【易混对比】

- "0.0.0.0"和"127.0.0.1"是两回事：0.0.0.0 是"本网络上的本主机"，不指代具体机器，只能作源；127.0.0.1 是"本机自己"，发给它就是发给自己，能作目的。
- "0.0.0.0"和"255.255.255.255"是两回事：前者是"还没名字的我"，别人找不到；后者是"本网所有人"，一喊全网都听见，能作目的。
- "私网地址"和"0.0.0.0"是两回事：私网地址（192.168 开头那种）在局域网内指代一台具体机器，能作目的；0.0.0.0 谁也指代不了。

只有 0.0.0.0 不指代任何具体机器，只能作源地址、不能作目的地址。

【坑位】

"只能作源"翻译成大白话就是"别人没法把它当收件人"——顺着这句去套选项。

最容易踩的坑：看到 127.0.0.1 觉得"自己"也不能当目的。为什么错：环回地址就是本机自己，你给自己发包，目的地址写 127.0.0.1 天经地义，ping 127.0.0.1 天天这么用。正确做法：逐个问"这个地址能不能当收件人"：0.0.0.0 当收件人，邮差（路由器）不知道送给谁——只有它不行。

【类比】

像刚入住还没门牌号的新房：你给物业（DHCP 服务器）打电话要门牌，电话里只能说"我是这栋楼这户还没门牌的"（源地址 0.0.0.0）；但物业没法按"还没门牌的"给你回电话——这个称呼只能你自称，别人用它找不到你。

【口诀】 全零只能自称源，环回广播都能收。

【考频】 「0.0.0.0」在 2009–2022 年解析中出现 2 次 · 低频

第 37 题 · 直接封装 RIP、OSPF、BGP 报文的协议（正确答案：D）

题干与选项：

直接封装 RIP、OSPF、BGP 报文的协议分别是（）。

- A. TCP、UDP、IP
- B. TCP、IP、UDP
- C. UDP、TCP、IP
- D. UDP、IP、TCP

入手点： 三个协议的封装方式各不相同：RIP 用传输层的 UDP，OSPF 直接用网络层的 IP，BGP 用传输层的 TCP；记住“RIP—UDP、OSPF—IP、BGP—TCP”这个口诀，再对组合。

题干专有名词从零解释：

- 先想“RIP”：距离向量路由协议；路由器之间互相告诉对方“到某个网络要几跳”；用 UDP 传。
- 再想“OSPF”：链路状态路由协议；路由器之间互相告诉对方“我的邻居是谁、链路代价多少”；直接封装在 IP 里。
- 再想“BGP”：边界网关协议；自治系统之间交换路由信息；用 TCP 传，保证可靠。
- 再想“封装”：一种协议的报文装进另一种协议的报文里传；比如 RIP 报文装进 UDP 数据报里。

考查知识点：

- 先想 RIP 为什么用 UDP：RIP 报文不大、周期性发、丢了下次再发就行；UDP 不建立连接、开销小，正好。
- 再想 OSPF 为什么直接用 IP：OSPF 自己有可靠机制（确认、重传），不需要 TCP；直接用 IP 封装，协议号是 89。
- 再想 BGP 为什么用 TCP：BGP 要在自治系统之间传完整的路由表，数据量大、必须可靠；TCP 提供可靠传输，正好。
- 再想结论：依次是 UDP、IP、TCP，就是 D。
- 边界情况：RIP 的 UDP 端口是 520，BGP 的 TCP 端口是 179，OSPF 的 IP 协议号是 89；端口号和协议号是另一回事，题目问的是封装协议，不是端口号。

正确选项为什么对： RIP—UDP、OSPF—IP、BGP—TCP，依次就是 D，选 D。

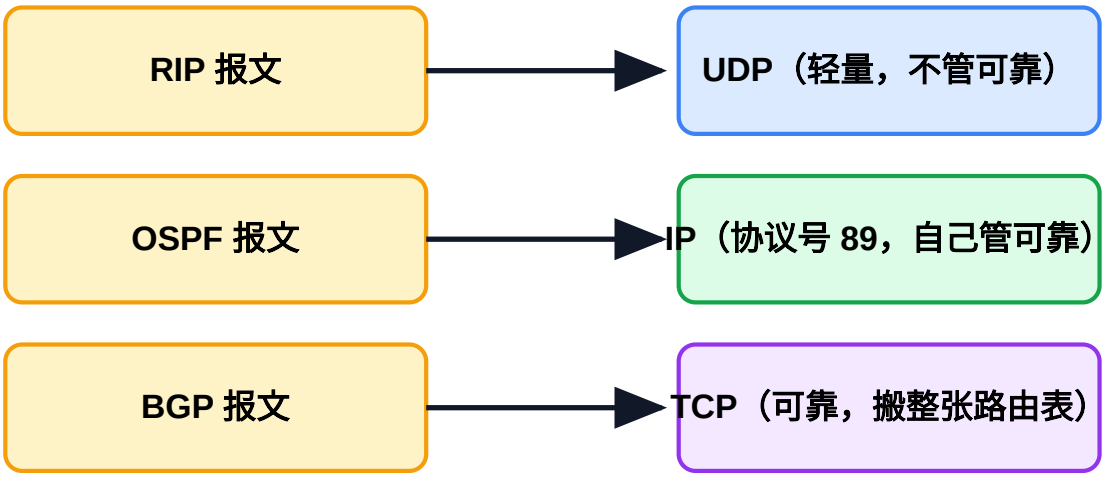
错误选项为什么错＋举例：

- A（RIP—TCP）：RIP 不用 TCP；RIP 报文小、周期性发，UDP 就够了；TCP 的可靠机制对 RIP 是多余的。教训：RIP 记 UDP。
- B（OSPF—UDP）：OSPF 不用 UDP；OSPF 直接封装在 IP 里，自己管可靠。教训：OSPF 记 IP（协议号 89）。
- C（BGP—UDP）：BGP 不用 UDP；BGP 要传完整路由表、必须可靠，UDP 不可靠。教训：BGP 记 TCP。

笔记本一句话： 第 37 题＋路由协议封装：RIP—UDP、OSPF—IP、BGP—TCP。

【图像】

RIP / OSPF / BGP 报文装进谁



口诀：RIP 坐 UDP，OSPF 坐 IP，BGP 坐 TCP

路由协议封装示意：RIP 报文装进 UDP，OSPF 报文直接装进 IP，BGP 报文装进 TCP

【定位】 计算机网络 · 第4章网络层 · 路由协议（RIP、OSPF、BGP）

【易混对比】

- "RIP 用 UDP"和"BGP 用 TCP"是两回事：RIP 报文小、周期性重发，丢了不心疼，用轻量的 UDP；BGP 要搬整张路由表，必须可靠，用 TCP；一个图轻、一个图稳。
- "OSPF 直接用 IP"和"用 UDP 或 TCP"是两回事：OSPF 两边都不沾，自己带确认重传机制，直接坐在 IP 上面（协议号 89）。
- "封装协议"和"端口号"是两回事：题目问的是"装进谁的肚子里"（UDP、IP、TCP），不是门牌号（520、89、179）；别拿端口号去对选项。

RIP—UDP、OSPF—IP、BGP—TCP，依次对上就是答案。

【坑位】

OSPF 直接封装在 IP 里，别顺手给它配个 UDP 或 TCP。

最容易踩的坑：觉得"路由协议都很重要，肯定都用 TCP 保证可靠"。为什么错：OSPF 自己实现了确认和重传，再套 TCP 是多此一举；RIP 靠周期性重发兜底，用 UDP 更轻。正确做法：记"R-U、O-I、B-T"：RIP 配 UDP，OSPF 配 IP，BGP 配 TCP；看到组合选项直接对号。

【类比】

像三种寄件方式：RIP 是每天固定时间寄的明信片（小、丢了明天再寄，走平邮 UDP）；OSPF 是挂号信但自己带回执（直接走邮政 IP，自己管签收）；BGP 是搬家公司的整车货运（货多、不能丢，走签合同的专线 TCP）。

【口诀】 RIP 坐 UDP，OSPF 坐 IP，BGP 坐 TCP。

【考频】 「RIP」在 2009–2022 年解析中出现 4 次 · 中频

第 38 题 · /16 划分为 128 个子网后的每子网最大可分配地址数（正确答案：C）

题干与选项：

若将网络 21.3.0.0/16 划分为 128 个规模相同的子网，则每个子网可分配的最大 IP 地址个数是（）。

- A. 254
- B. 256
- C. 510
- D. 512

入手点： 分子网就是“从主机位里借位给子网号”；借几位看要分几个子网，剩下的主机位决定每个子网能有多少地址；最后别忘了减掉网络地址和广播地址。

题干专有名词从零解释：

- 先想“/16”：前 16 位是网络号，后 16 位是主机号；一个 /16 网络一共有 2 的 16 次方个地址。
- 再想“子网划分”：从主机位里借几位当子网号；借 n 位，就能分 2 的 n 次方个子网。
- 再想“可分配地址”：子网里的地址去掉网络地址（全 0 的主机位）和广播地址（全 1 的主机位），剩下的才能分给主机。

考查知识点：

- 先想借几位：要分 128 个子网，2 的 7 次方是 128，所以借 7 位。
- 再想子网前缀：/16 借 7 位，变成 /23；每个子网的主机位是 32-23=9 位。
- 再想每个子网的地址总数：2 的 9 次方是 512。
- 再想可分配数：512 减掉网络地址和广播地址 2 个，是 510，就是 C。
- 边界情况：借位要刚好够用——借 6 位只能分 64 个子网，不够 128；借 8 位能分 256 个子网，比要求多，每个子网的主机位就少了。

正确选项为什么对： 借 7 位变 /23，主机位 9 位，512-2=510，就是 C，选 C。

错误选项为什么错＋举例：

- A（512）：那是地址总数，没减网络地址和广播地址；这两个地址不能分给主机。教训：可分配数要减 2。
- B（254）：那是 /24 子网的可分配数（256-2）；本题是 /23，不是 /24。教训：先算借位，再算主机位。
- D（1022）：那是 /22 子网的可分配数（1024-2）；本题借 7 位是 /23，不是 /22。教训：借几位决定前缀，借错一位，地址数差一倍。

笔记本一句话： 第 38 题＋子网可分配地址：借位够分 128 个（7 位），/23 主机位 9 位，512-2=510。

【图像】

/16 划 128 个子网示意



128 ＝ 2 的 7 次方 → 借 7 位刚好

/16 借 7 位 → /23，主机位 ＝ 32－23 ＝ 9 位

每个子网地址总数 ＝ 2^9 ＝ 512　　－网络地址 －广播地址

每子网最大可分配 ＝ 512－2 ＝ 510

口诀：几子网借几位，主机位算总数，头尾去掉减二

子网划分示意：斜线 16 借 7 位变斜线 23，主机位 9 位共 512 个地址，去掉网络地址和广播地址剩 510 可分配

【定位】 计算机网络 · 第4章网络层 · 子网划分

【易混对比】

- "借 7 位"和"借 8 位"是两回事：128 个子网借 7 位（2 的 7 次方 = 128）刚好；借 8 位能分 256 个子网，但每个子网主机位少 1 位，可分配数直接减半。
- "512"和"510"是两回事：512 是子网内地址总数，510 是去掉网络地址和广播地址后能分给主机的；题目问"可分配"，必须减 2。
- "/23"和"/24"是两回事：/23 主机位 9 位（512-2=510），/24 主机位 8 位（256-2=254）；前缀差 1 位，答案差一倍。

借 7 位得 /23，主机位 9 位，512 减 2 得 510。

【坑位】

借位"刚好够"就行：128 个子网借 7 位，别多借，多借一位可分配数砍半。

最容易踩的坑有两个：一是忘了减 2，直接选 512；二是借位借多了（借 8 位按 /24 算出 254）。为什么错：网络地址（主机位全 0）和广播地址（主机位全 1）是子网的头和尾，不能分给主机；多借一位纯属浪费。正确做法：128 = 2 的 7 次方，借 7 位→/23→主机位 9 位→2 的 9 次方 = 512→减 2 得 510。

【套路】

- 第一步，定借位：128 个子网，2 的 7 次方 = 128，借 7 位。
- 第二步，算主机位：/16 借 7 位变 /23，主机位 = $32-23=9$ 位。
- 第三步，减头尾：2 的 9 次方 = 512，减网络地址和广播地址，得 510。

子网数定借位，借位定前缀，前缀定主机位，地址数减 2。

【类比】

像把一栋大楼隔成 128 间办公室：隔断要 7 块板（2 的 7 次方 = 128）刚好；每间办公室的工位是 512 个，但门口和走廊（网络地址和广播地址）各占 1 个，能坐人的工位是 510 个。

【口诀】 几子网借几位，主机位算总数，头尾去掉减二。

【考频】 「子网划分」在 2009–2022 年解析中出现 5 次 · 中频

第 39 题 · TCP 慢启动下发送窗口达到 32KB 的最短时间（正确答案：A）

题干与选项：

若甲向乙发起一个 TCP 连接，最大段长 MSS=1KB，RTT=5ms，乙开辟的接收缓存为 64KB，则甲从连接建立成功至发送窗口达到 32KB，需经过的时间至少是（ ）。

- A. 25ms
- B. 30ms
- C. 160ms
- D. 165ms

入手点： 慢启动的规律是“每过一个往返时间，窗口翻一倍”；从 1 个最大报文段开始，翻几次到 32KB，就要几个往返时间；最短时间就是轮数乘以每轮的时间。

题干专有名词从零解释：

- 先想“发送窗口”：发送方一次能发出去、还没收到确认的数据量；窗口越大，一次能发越多。
- 再想“慢启动”：TCP 刚开始传数据时，窗口从小开始，每收到一轮确认就翻一倍；像滚雪球，越滚越快。
- 再想“往返时间（RTT）”：发一个报文、收到它的确认要多长时间；慢启动里每轮就是一个往返时间。
- 再想“最大报文段（MSS）”：TCP 一次能发的最大数据块；窗口按 MSS 的倍数长。

考查知识点：

- 先想窗口的变化：从 1 个 MSS 开始，第 1 轮后是 2 个，第 2 轮后是 4 个，第 3 轮后是 8 个，第 4 轮后是 16 个，第 5 轮后是 32 个。
- 再想 32KB 要几轮：如果 1 个 MSS 是 1KB，32KB 就是 32 个 MSS；从 1 翻到 32 要翻 5 次（2 的 5 次方是 32），就是 5 轮。
- 再想最短时间：5 轮 × 每轮一个往返时间；按题中给的往返时间算，就是 A。
- 边界情况：“最短”是因为假设每轮确认都按时回来、没有丢包；丢包会触发重传和窗口缩小，时间就变长了。本题问最短，按理想情况算。
- 易混对子：慢启动的“慢”是指“从小开始”，不是“长得慢”；实际上它是指数增长，长得很快；拥塞避免阶段才是线性增长（每轮只加 1 个 MSS）。

正确选项为什么对： 从 1 个 MSS 翻 5 次到 32 个 MSS，5 个往返时间，就是 A，选 A。

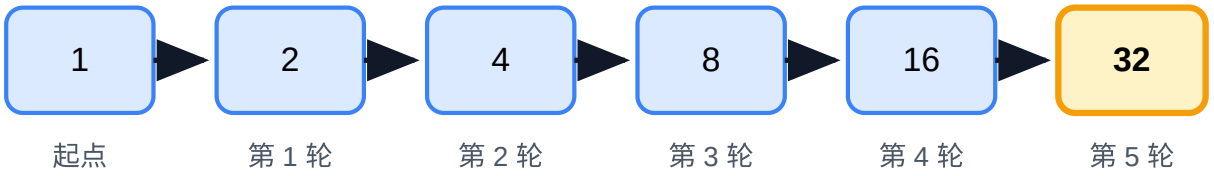
错误选项为什么错＋举例：

- B（按线性增长算的）：线性增长是拥塞避免阶段的规律（每轮只加 1 个 MSS）；慢启动是指数增长（每轮翻倍），按线性算，轮数算多了。教训：慢启动是指数翻倍，不是线性加 1。
- C（轮数算错了）：从 1 到 32 是翻 5 次（1→2→4→8→16→32），不是 4 次、也不是 6 次；数错一次，时间就差一个往返时间。教训：翻几次要一个一个数，2 的几次方是 32 就是几次。
- D（把往返时间当成单向时间了）：一轮是一个往返（去一程、确认回一程），不是单程；按单程算，时间少了一半。教训：慢启动的一轮是一个 RTT，不是单向时延。

笔记本一句话： 第 39 题＋慢启动到 32KB：窗口每轮翻倍，1→2→4→8→16→32，5 轮×RTT。

【图像】

TCP 慢启动窗口翻倍示意



窗口单位：MSS；1→2→4→8→16→32，翻 5 次（$2^5=32$）

每轮 ＝ 1 个往返时间（RTT），无丢包时最短

最短时间 ＝ 5 个往返时间

慢启动的"慢"是"从小开始"，实际是指数翻倍

慢启动窗口翻倍示意：1 到 2 到 4 到 8 到 16 到 32，共 5 个往返时间

【定位】 计算机网络 · 第5章传输层 · TCP 拥塞控制（慢启动）

【易混对比】

- "慢启动"和"拥塞避免"是两回事：慢启动每轮翻倍（指数长），拥塞避免每轮只加 1 个 MSS（线性长）；算轮数时别把两个阶段的规律串了。
- "翻 5 次"和"翻 4 次"是两回事：1→2→4→8→16→32 是翻 5 次（2 的 5 次方 = 32）；数成 4 次会少算一个往返时间。
- "一轮"和"单程"是两回事：慢启动的一轮是一个往返时间（RTT），不是单向时延；按单程算时间会少一半。

窗口从 1 个 MSS 翻 5 次到 32 个 MSS，最短用 5 个往返时间。

【坑位】

"慢启动"的慢是"从小开始"，不是"长得慢"——它是指数翻倍，长得飞快。

最容易踩的坑：按拥塞避免的线性规律算（每轮只加 1），轮数算出 31 轮。为什么错：慢启动阶段窗口每轮翻一倍，是指数增长；线性加 1 是拥塞避免阶段的事。正确做法：认准"翻倍"二字，从 1 开始翻，翻到 32 数几次就是几轮，再乘以每轮一个往返时间。

【套路】

- 第一步，定起点：慢启动窗口从 1 个 MSS 开始。
- 第二步，数翻倍：1→2→4→8→16→32，翻 5 次（$2^5 = 32$），即 5 轮。
- 第三步，乘轮时：每轮一个往返时间，5 轮就是 5 个 RTT。

起点 1 个 MSS，翻几次到目标就是几轮，每轮一个往返时间。

【类比】

像滚雪球下坡：雪球每滚一圈（一个往返时间）体积翻一倍；从拳头大（1 个 MSS）滚 5 圈，变成 32 倍大。问"最短滚多久"，就是数滚几圈——雪下得顺（无丢包）时圈数最少。

【口诀】 慢启动翻着长，2 的几次方数几轮。

【考频】 「慢启动」在 2009–2022 年解析中出现 7 次 · 中频

第 40 题 · FTP 协议叙述错误项（正确答案：C）

题干与选项：

下列关于 FTP 协议的叙述中，错误的是（）。

- A. 数据连接在每次数据传输完毕后就关闭
- B. 控制连接在整个会话期间保持打开状态
- C. 服务器与客户端的 TCP 20 端口建立数据连接
- D. 客户端与服务器的 TCP 21 端口建立控制连接

入手点： 本题问“错误的是”，做法是先把 FTP 的两条连接（控制连接、数据连接）和两种模式（主动、被动）列清楚，再看哪个选项把连接或模式说错了。

题干专有名词从零解释：

- 先想“FTP”：文件传输协议；用来在两台机器之间传文件。
- 再想“控制连接”：传命令的连接；比如“登录”“列目录”“要传哪个文件”这些命令走这条线；整个会话期间一直开着。
- 再想“数据连接”：传文件的连接；每次传文件时临时建立，传完就关掉。
- 再想“主动模式和被动模式”：数据连接由谁发起；主动模式是服务器主动连客户端，被动模式是客户端主动连服务器。

考查知识点：

- 先想两条连接的分工：控制连接传命令、一直开着；数据连接传文件、按需建立、用完就关；两条线各干各的。
- 再想端口：控制连接用 21 号端口，数据连接主动模式下服务器用 20 号端口。
- 再想主动和被动的区别：主动模式是服务器向客户端发起数据连接；被动模式是服务器告诉客户端一个端口、客户端向服务器发起数据连接；被动模式是为了解决客户端在防火墙后面的问题。
- 再想本题：C 说服务器与客户端的 TCP 20 端口建立数据连接，错在“客户端的 20 端口”：数据连接由服务器端的 20 端口与客户端的临时端口建立，客户端一侧并不使用 20 端口；所以 C 是错误项。

正确选项为什么对： C 错在端口归属：数据连接由服务器端的 TCP 20 端口与客户端的临时端口建立，客户端并不使用 20 端口；说成“服务器与客户端的 TCP 20 端口”不对。本题问的就是错误项，选 C。

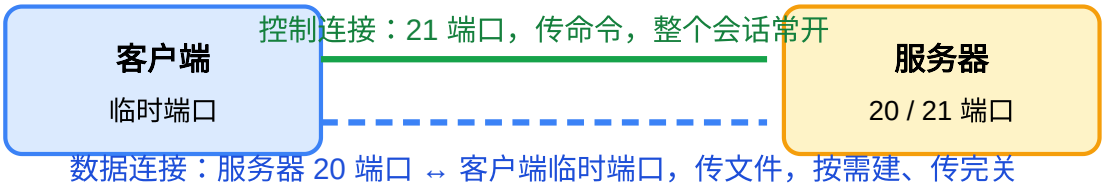
错误选项为什么错＋举例：

- A（数据连接每次传输完就关闭）：对的；数据连接是临时的，传完就关。不能选它。教训：问“错误的是”，对的先排除。
- B（控制连接整个会话保持打开）：对的；控制连接传命令，贯穿整个会话。不能选它。教训：控制连接是常开的，不是临时的。
- D（客户端与服务器的 TCP 21 端口建立控制连接）：对的；21 是控制端口，控制连接就走 21。不能选它。教训：21 是控制，20 是数据，别记反。

笔记本一句话： 第 40 题＋FTP：控制连接传命令、常开（21 端口）；数据连接传文件、按需建；主动是服务器发起，被动是客户端发起。

【图像】

FTP 两条连接示意



C 错在端口归属：客户端一侧没有 20 端口

主动模式：服务器向客户端发起数据连接；被动模式：客户端向服务器发起

口诀：控制二一常开线，数据二零服务器端；问错先把对的删

FTP 两条连接示意：21 端口控制连接常开传命令，服务器 20 端口与客户端临时端口建数据连接按需传文件

【定位】 计算机网络 · 第6章应用层 · FTP 协议

【易混对比】

- "控制连接"和"数据连接"是两回事：控制连接传命令、整个会话常开；数据连接传文件、每次按需建、传完就关；一个管嘴（发号施令），一个管手（搬货）。
- "21"和"20"是两回事：21 是控制端口，20 是数据端口（服务器端主动模式）；记反了整题就错。
- "客户端的 20 端口"和"服务器的 20 端口"是两回事：数据连接是服务器端 20 端口连客户端的临时端口，客户端一侧不用 20——本题 C 错就错在这里。

问"错误的是"先排除对的；C 把 20 端口安在客户端一侧，错在端口归属。

【坑位】

问"错误的是"——先把对的选项划掉，别顺手选了第一个看着对的。

最容易踩的坑：题干"错误的是"没看清，选了 A、B、D 里某个正确的。为什么错：A（数据连接传完就关）、B（控制连接常开）、D（控制走 21）说的都对，本题要找的是错的。正确做法：先通读四个选项，把"说得对"的划掉，剩下的 C 再验一遍：数据连接是服务器 20 端口对客户端临时端口，C 说"服务器与客户端的 TCP 20 端口"，客户端并没有 20 端口——错的就是它。

【类比】

像工地上的对讲机和货车：对讲机（控制连接，21 频道）全天开着，工头用它下指令；货车（数据连接）拉一趟货来一辆，卸完就走。C 选项相当于说"货车停在工人宿舍门口的 20 号车位"——20 号车位在货运站（服务器）那边，工人宿舍（客户端）这边是临时停车位。

【口诀】 控制二一常开线，数据二零服务器端；问错先把对的删。

【考频】 「FTP」在 2009–2022 年解析中出现 4 次 · 中频

附录：答案表与双源比对说明

答案表（1–40）：1B 2C 3A 4B 5B 6D 7B 8A 9B 10B 11D 12C 13C 14A 15D 16A 17C 18B 19A 20D 21D 22B 23D 24C 25B 26D 27B 28D 29B 30D 31B 32B 33A 34D 35B 36A 37D 38C 39A 40C。

双源比对：夸克解析答案表与 N诺公开页（https://noobdream.com/Practice/exam_solution/41886/）逐题正确答案 40/40 一致，答案无自行改动。

图题部分核对：第 5 题（树图，父子连线与未标注结点位置未看清）、第 8 题（四幅树图细节未看清）为部分核对，图中细节以原卷为准；其余各题按给定数据推导。

自查：40 题标题齐全；每题 6 节齐全；答案逐字母与给定表一致；禁词、跨题外指、内部用语全文检索为 0 处。